

On Computable Meanings

2nd Quater of the 21st Century

1 The Tails of Reality

Life as a human being is an incredible experience. Billions of years of physical evolution, millions of years of biological evolution, an open source code repository, capable of replicating itself and thereby forking, experimenting, and pushing forward what works. The classes of life that this process has wrought are a tapestry of specialized machines, some so vastly different from each other as to suggest that they must have come from different sources. Why did this process start? How did it start? What explanation can there be for the seemingly infinite set of coincidences over time that have allowed this present moment? These are questions that sprout instinctively in humans, the self-aware creatures marking the newest age of biological discovery. Certainly, we say, in the early days, life had no purpose other than arbitrary mechanical survival, but somewhere along the way, the machine woke up. Was this a new emergent ‘soul’ or ‘self’ force out of *lifeless* chemistry before it, or did the code just eventually become more perceptive, adaptive, and capable so as to begin to resemble a ‘conscious’ presence? Is there even a difference? If there were, how could we prove it?

From our stature as hard-conditioned self-perceivers, we cannot imagine an understanding of our reality that does not put us in control, with the joysticks in the hands of our soul and mind. All of our evidence tells us that we are here, this is real, and this is special, but what if we were just being fed biased facts? Seeing things through a fundamentally biased lens? Does a computer *itself* have control over how a program manipulates its electrical flows, the switching of its latches, and the logic that determines why it does what it does? Could a machine that is programmed to be as confident, self-aware and self-concerned, persistent, and as *right* as possible ever conceive of its source code that defines it? “Billions of years of physical evolution, Millions of years of biological evolution”; are these ideas even different at all? As artifacts of Biology and products of evolution, we ascribe to it purpose, striving, and a kind of light that emerged from darkness, yet we hold the mountains, lakes, moons, and stars as static, uncontrolled and unaware bodies of mass. At some point, gravity, electromagnetism, and chemistry started dancing with each other so sophisticatedly that patterns more active, complex, and adaptive arose and a line in the sands of existence was drawn between the living and the non-living. But, as we learned in middle school, the winners write the history. The mountains and moons aren’t talking, they are not showing signs of life, so they must not be alive like us; but maybe we just did not know how to ask or how to look for their life force. Or are we just complex pieces of matter that have tricked ourselves into a special distinction?

Fine, abstracting and assuming up some levels to a human context, we could ponder human intelligence, ingenuity, deduction, computation, communication, and science. We perceive, we remember, and we then decide. From our observations, we establish assumptions, we make sense of the world through patterns and models, including the pervading pattern of our own self-perception. (Think about that!) Wake up, reboot: this is my body, *I* am the persistent force that carried this body yesterday, what *I* did then affects me now and what *I* do now will affect me tomorrow. Core assumptions like causality, object permanence, and primitive logical inference underpin the entire continuum of our consciousness at any moment, all of our input signals being processed and packaged into a computed internal projection of a sensible world in front of our eyes. It all fits and works so well. (Would we be having this conversation if it didn’t?) So well in fact that it is easier than not to forget that there is a physical system endowing this consciousness to emerge. While the systems we have adapted collectively to understand and persist in the world, such as numbers, inferential logic, and communication are so effective most of the time, it seems inconceivable that they are not representations of discovered ‘truths of the universe’. (How can you tell me consciousness is an illusion if I *feel* it? How can you discredit the inherent truth of communication if I can read and understand *this* sentence?) We have, however, shown this truth not to be the case in the fat tails of the distribution of reality as we see it, in the edge cases when we zoom far in or far out. Our model is imperfect and internally defined, making the tails of our reality confounding and unknowable.

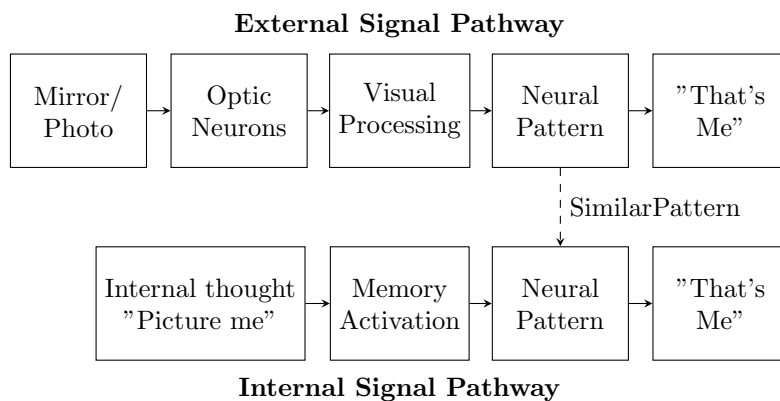
2 Picture Your Face

Picture your face. You can see it now, can you not? How does that song go? Do you hear it? Think about the feeling of a warm shower engulfing you with comfort. What about that dream last night?

What happens when you see a face? Your eyes receive energy waves which are then processed by optic neurons that trigger an ascending hierarchy of neural firing patterns, first primitively finding colors and edges, then depths and shapes, then increasingly more abstract constructs of the ‘image’ you are seeing. Many such patterns will activate anytime you look at anything in, say, a *light* environment, furthermore will there be common patterns activated anytime you look at a *face*, anytime you look at a *face you know*, and finally there is some highly complex pattern of neural activity that denotes to you (some part of your brain? your sense of self? a more complex pattern itself that denotes the self?) *that this is my face*. None of this is conjecture; your brain processes everything you see with the same hardware, so clearly there must be softer (ware) activity related to distinct things you see, hear, and otherwise sense (or think!). When your reflection is right in front of you or when you see a selfie, this process is repeated many times per second, heavily reinforcing the concept (denoted by a neural pattern) of *my face*. This classification event is something of a computation done by your brain. Not in the sense that it has to figure out or calculate *what your face looks like*, but instead that there is an algorithm being executed, a set of steps to turn inputs (visual, likely other mediums) into an output (to you) with an enhanced or functional meaning. A process to turn visual waves into a mental conception of something. In the exact same vein, waking up, coming to, and reorienting your mind into *this* body in *this* place is too a computation. As is the reading of *this* sentence. Everything your brain does that works to make sense of the world for you is a computation on inputs (oftentimes inputs originate from within the brain as well!).

Given this, can we imagine how we can then ‘recall’ the same pattern (or at least parts of it)? How, when not looking at yourself, can you still become quasi-visible ‘in your mind’? How can a song be ‘stuck in your head’? Recall what we said about when you truly are seeing something; the external signals trigger a whole host of internal signals that process the input into an internal understanding or projection of ‘what we are seeing’. The feeling of seeing a certain thing is, most of the time, triggered by the external signals themselves; but the feeling, the decision, the knowledge itself of classifying something you see (just a speck of the second-by-second processing and projection you do to feel human and conscious) is an internal phenomenon. Thus it makes sense to be able to be triggered from within as well, just like so many other neural events. One of the mysteries of the brain is the way in which the *physical* shapes and gives way to the *meta-physical* mind which in turn seems to reach back down and change the physical. It is on full display here. When you picture something in your head when it is not really there, hum or whistle a song and feel the same feelings that the song usually gives you, it is because you are deliberately triggering mental firing patterns for these very things, thus igniting maybe the tail end or the peripherals of the original cascade of signals that flood in every time you ‘really’ see it. Are you really ever ‘truly’ seeing it or do we sometimes just have overwhelmingly strong evidence, sufficient to reinforce a concept, sufficient to say ‘this is right here in front of me’?

These mental images (and dreams!) give very strong evidence of the strength of the internal projection and abstraction engines in our brain and the extent to which our reality is a smoothed internal aggregation of data.



Another teaser for the computation underpinning all of our thoughts and consciousness can be elucidated by discussing what happens when you have a word on the tip of your tongue, but just cannot seem to recall it. As discussed in the previous example, our brains function by taking in data, which is then processed by hard and software that somehow synthesizes it into a sensible ‘thing’ or concept in our mind. While we discussed visuals, this principle just as strongly describes all other senses and even interpersonal communication, reading, or our internal

monologue. The internal processing is continuously enriched (or at least changed or tweaked) by all the data that comes in, and the feedback for how ‘correct’ we were previously. For instance, we discussed recognizing your own face, but that association in our brains was only formed by several tries, several examples of ‘seeing your face’ which then physically established somewhere in our neurons that a certain pattern of light waves corresponds to a certain concept (pattern of neural firings!) in our head that denotes ‘me’. When thinking about words, as a kid, we all began babbling, repeating what we heard around us as simple devices of mimicry. A baby knows not what it is trying to say or that it is even trying to say anything, it just hears sounds from people around it and wants to mimic. It may, along its way, stumble on a gold mine and say a word that then causes some external reaction, a confirmation. For instance, my father and mother continuously referred to themselves as daddy and mommy, words that I heard hundreds of times per day, then in my nascent mind, when I reproduced those sounds, they would miraculously come to attention! From that first fateful confirmation, my understanding of those words only strengthened as representations for these two people that provide me food, safety, warmth, and life! Other similar associations form with food, naps, the bathroom, and all the other common functions a baby may want to call on in the outside world. Did those specific words have any inherent link to the objects with which I associated them, no, they were simply just confirmed to be effective labels for those things. We English speakers agree on the meaning of all the words we use, when we use them, but all of this is just a product of reinforcement, patterns being imprinted. Fast-forwarding from childhood, all the words we use internally have associations, definitions with other concepts that our brains are familiar with—often other words that represent more primitive things, all the way down. For instance, how would I explain what a ‘bus stop’ is? How would I define this word in terms of other words? I would say that it is a place where buses stop to pick people up and drop them off. This immediately necessitates the definition of all the words used in this sentence. Let us pick bus; a vehicle that transports people or things. What are vehicles, people, or transportation? Clearly we have opened up a pandora’s box of sorts, but it is to illustrate all of the nested meaning that underpins a simple word or phrase, objects which we then combine endlessly to talk about other things. Imagining defining the entire web of all these words is a fool’s errand explicitly, however recently technical and statistical breakthroughs have shown us that computer programs do a solid job of this by building guess-and-check models that eventually see enough examples to behave correctly. Do the architects of Large Language Models define every single word explicitly, or encode grammar rules for the model to obey as it parses then reproduces sentences? Absolutely not. The model simply converges to ‘correct’ behavior by billions of examples of simply trying and then fixing itself to hopefully do better next time. Back in the mind of a child, if I were to point at my dad and say ‘mommy’, I would be met with signals that tell me or make me feel wrong, then he would say ‘daddy’ and point to himself. I would say ‘daddy’ and from that example, my understanding would be changed. After maybe some more of these examples, my brain draws an association (mind you, an association between signals from my ear, movements on my throat and mouth, and the guy I see, smell, and feel outside of me). There is some physical mechanism that allows the ‘correct’ associations to be imprinted in my head, much like the small tweaks that happen to a Large Language Model.

3 What is Computation?

As you can gather from the name, computation has to do with the process of calculating something mechanically, like by a *computer*. While the scope of this topic seems limited to *stuff happening in a computer or calculator*, it actually applies to all perception, deduction, and thought too. Computations can be extremely complicated, like in modern Transformer Neural Networks, or they can be very simple, like the sum of 2 and 3 or detecting the difference between two piles of colored buttons. No matter the complexity, the underlying characteristic of a procedure being followed to achieve an improved result remains. While numbers and the various number-theoretic tools we employ all day every day as a part of our necessary thinking feel extremely natural, objective, and obvious, they in fact represent an abstract computed view of the outside world. It feels like the most unquestionable conclusion that $2 + 3 = 5$, but we have to realize the foundation of assumptions on which that abstraction rests. What is **2**? Thinking back to how you learned what numbers were as a toddler, or if you tried to explain to me what a number is without referencing number theory, ultimately the explanation would boil down to a representation of quantity. The Arabic numeral **2** just stands in for the concept of a thing and just a single other thing. There is no way to define it other than a conceptual representation for a certain quantity. The basic digits of Arabic numerals are so ingrained in the minds of anyone who can read this that such explorations ‘below’ the abstract are hard not to try to breeze through. But it is important to recognize that our number system relies on some heavy assumptions and shorthands. To illustrate this, we will try to abandon our conventions and redefine numbers from the ground up, first just thinking about basic representations of quantity.

We will have to make some basic assumptions to get off of the ground, though. Since we want numbers to

represent quantities of arbitrary elements, it may make sense to start with the most ‘basic’ number, then build from there. For thousands of years, the simplest and ‘first’ number was always thought of as **1**, the singular, however eventually philosophers and mathematicians realized the importance of the concept of emptiness, absence of quantity. We could easily start with either **1** or **0**, but by now most agree on the latter. **0** is the one and only symbol and concept we will use familiarly.

3.1 Successorship

Starting with **0**, the empty set, an empty quantity, we can say that any number (representing a quantity) can always have a successor that is a quantity that containing a single additional element. If you assume that **0** is a number and that every number has one number higher than it, you can first imagine **1**, then **2**, then **3**, and so fo(u)rth. But we do not want these Arabic numerals to confuse our new definition of numbers! Even without addition, you can purely symbolically represent numbers and successorship: start with randomly assigning nothingness to **0**. The algorithm for the successor, **S(n)**, of any number **n** is just whatever **n** is with an extra ***** written after it. What is the successor of **0**? We take **n (0)** and return **n*** (which would be **0***). What is the successor of this number? **0*** with an extra ***** becomes **0****. Then **0*****, and so forth. We can also immediately see that this can be carried on forever, suggesting the infinitude of possible quantities.

Why do we want to take something we already understand as numbers (**0, 1, 2, 3, etc**) and turn them into an inefficient representation? While it feels like the number symbols we know are simpler and more efficient at representing the idea of a number, and they are in many contexts, it is important to understand the primitive logic and computation underpinning those representations. Also, if we as humans want to build numbers into machines so they can calculate stuff for us (you can thank this desire for any and everything you do today with anything electronic), we need to assume no intuition on the part of the machine. How can we tell it to add two numbers if it does not know what numbers are? We have to dumb things down and redefine so a machine can work with them, independent of the ambiguity of language and the reliance on common intuition; but we need them dumbed down in a way that we can then map the computer’s representations back to ours. From here forward, we should imagine the domain in which we are defining this new system to be a simple machine that can scan and write things on a piece of paper; a person with a pencil, paper, and only the ability to follow strict rules, no thinking.

3.2 Addition

Ok, we have assumed the existence of a number **0** and we know how to get the next number for any number. Let us see how this can define **addition** : assuming the successor formula **S(n)** that returns **n***, we can define **Sum(n, m)** that will return what we know as **n + m**. But we want to do this mechanically. What is adding? Well it really is just repeated successorship (repeatedly incrementing **n, m** times). Transferring quantity from one argument into the other to form the conjoined quantity. There is no concept of **number + number = number** number + number = number; instead we have

$$\text{Sum}(\text{string_of_symbols}, \text{string_of_symbols}) \rightarrow \text{string_of_symbols}.$$

Using **S(n)**, we can say that **Sum** moves a ***** from **m** and then calculates **S(n)** until **m** is **0** then we just get rid of that too. We assume that deleting and writing and storing a value can be done by this machine, as trivially as it can be done on paper with a pencil. So for **Sum(0**, 0***)**, we can trace our progress:

$$\begin{aligned} & \text{Sum}(0**, 0***) \quad (\text{starting expression}) \\ \rightarrow & \text{Sum}(0***, 0**) \quad (m \neq 0, \text{ so take } * \text{ and } n = S(n)) \\ \rightarrow & \text{Sum}(0****, 0*) \quad (m \neq 0, \text{ move another } *) \\ \rightarrow & \text{Sum}(0*****, 0) \quad (m \neq 0, \text{ move final } *) \\ \rightarrow & 0***** \quad (m = 0, \text{ final result is in } n) \end{aligned}$$

3.3 Multiplication

We sent in symbols with **2** and **3** stars and got out one new term with **5** stars, as expected! (Look at our pervasive need to still use shorthands like Arabic numerals to explain this.) We can now train a monkey or a machine to do addition without them having a clue what numbers are. From here, we can define **Product(n, m)** using **Sum**, because multiplication is nothing more than repeated addition (adding **n** to **0**, **m** times). **Product(n, m)** could be

defined as starting with **0** as a stored string on our paper/memory, then deleting one of the stars on **m**, calculating **Sum(stored_string, n)**, saving this sum as the updated **stored_string** and then when **m** becomes **0**, we just discard it. Let's follow **Product(0**, 0***)**:

```

    Product(0**, 0***) (starting expression)
→ Product(Sum(0, 0**), 0**) (m ≠ 0, update stored_string)
→ Product(Sum(0**, 0**), 0*) (m ≠ 0, update stored_string)
→ Product(Sum(0****, 0**), 0) (m ≠ 0, update stored_string)
→ Sum(0****, 0**) (stored_string is 0, so discard)
→ 0***** (final result of sum)

```

Yep, that is **6** stars at the end there. If you were to pause along the way and look at one of these intermediate steps hoping to parse it logically, you might be disappointed. For instance, look at the **3rd** term, **Product(Sum(0**, 0**), 0*)**, it might look like the **Product** of the **Sum** of **2** and **2**, and **1**, which is **5**, which is wrong! The error is not in the formula, it is in your projection of your understanding of addition and multiplication and numbers into this new system. As we discussed, the number on the right is simply a counter that we use in the process of recursively reducing the expression. We add **2** to our **stored_string** on the left of that inner **Sum** until the right argument, **m**, is **0**. This system is designed for a monkey, a machine, not a human who knows what numbers are and how they 'should' work. Following our strictly defined steps, we see that in the end these formulae work for us, but if we try to always map it to our conventional understandings throughout the computation, things look confusing. We must not use our ever encroaching intelligence to work with these mechanical systems. They are not thinking, they are moving around symbols mechanically. Without proving it, you can probably see that you could now define **Power(n, m)** with **Product**, because powers are just repeated multiplications!

As a motivating recap, we have now defined a representation of numbers, the infinitude of numbers, addition, multiplication, and powers without ever using a human definition of numbers, just writing rules about writing and rewriting symbols according to mindlessly testable conditions. You can easily too define **Predecessor** (taking off a *****) and thus **Subtraction** and **Division**.

Let us now examine an interesting example for calculating a square root symbolically and mechanically. There are several ways to do this, but to hint at where we are headed, let us imagine a square root function **Sqrt(n)** that returns the **integer** value of the square root of a number. The general method for doing this mechanically will amount to starting at **0**, squaring it, testing if that answer is *greater than* **n**, then if it is not, continuing to the next number, and if it is, choosing the number before we breached **n**. So for **Sqrt(9)**, we test **0**: (calculate **Product(0, 0)**) then check if it has less *****s than **0*******. We know **Product** is mechanical, and we could easily imagine a mindless method to check if two strings have the same amount of *****s. Obviously to us, **0** is not the answer for any input other than **0**, so we will increment to **0*** and try again and fail and repeat with **0**** and **0*****. Once we get to **0******, we will see that **Product(0****, 0****)** has larger amount of *****s than **n**, and we will then say the answer must be the last successful one, **0*****, and be done. For a number like **0***** (10)**, we would test up to **0***** and see it is still below, then go to **0****** and find that this squared also yields more stars than our input, so we declare that the integer value of the square root of **0***** (10)** is **0*****, which is true! Expanding this to calculate decimals is very possible, as you will see. This exercise is just intended to demystify the ability to define common mathematical operations unconventionally and mechanically.

3.4 Abstraction!

We have established several important things here, namely, the mechanization of numbers and number theory, divorcing it from intuition and building it in a purely routine, mindless way. In truth, we have peeled back some of the layers of **abstraction** that allow us to have such a quick, effective, and sensible conception of numbers, quantity, addition, multiplication, and all of the number theory that rests above it. While I am not at all positing that we abandon our intuition and abstractions, it is a crucial demonstration that numbers themselves are computational concepts, representing much more primitive ideas about raw data and the real world. Numbers are an abstraction on quantity, addition is an abstraction on counting and successorship, multiplication is an abstraction on addition, powers are an abstraction on multiplication. Layers all the way up. Just as we can dive down from biology, which is based on and defined as an abstraction of chemistry, which is an abstraction defined in terms of physics, which is an abstraction in terms of algebra, which is an abstraction on basic arithmetic, which we now know is an abstraction on counting, quantity, and comparatory logic.

The process we employed to start with basic assumptions and operations, then use those simpler processes repeatedly to define and compute more complex concepts, again and again, can be thought of as **recursion**. It goes hand in hand with **abstraction**. The thinkers we will soon meet are the first to realize that the nature of almost all computation is recursion, repetition, and conditional checks (like the checking we did in **Sqrt** or in **Sum** to make sure that **m** becomes **0**). Even numbers represent layers of recursive computation, and we will see that computation is the foundation for all perception, intelligence, and thus consciousness.

4 Classes of Numbers

Turing's work and results rely heavily on, and are much more interesting when you know about several classes of numbers. As discussed, numbers are just representations of quantity in the real world that can be combined and manipulated to solve and predict some incredibly valuable things. They are one of the basic substrates in which we build our sanity and intelligence. First and most elementary are the Natural Numbers, which can be thought of as **0, 1, 2, 3, 4**, and so on. These are just integers greater than or equal to **0**. We can expand this understanding to negative numbers as a mirror image of positives, but there is no need for this demonstration. We know that there is always a next natural number, and thus we know that they theoretically go on forever. If we have defined a representation of numbers like **0** and ***** we can calculate and count any natural number with enough time and scratch paper. Natural numbers are so called natural, because they feel the most natural and fundamental as things from the real world. We know you can measure something like a weight or a length and come up with a fractional value for a natural thing, but the basics of numbers just boil down to quantity. If there are two and a half leaves on the ground, there are just 3 distinct leaves. Natural numbers are also an incredibly important basis on which we have to build the other classes of numbers. The distinctions between classes of numbers will have to do with and will further inform notions about their **enumerability**. This idea refers to progressively counting items and pairing them off with the natural numbers, the most basic enumerable set. When you count anything physical, you use the natural numbers, so that is how we will define enumerability. As discussed, the naturals go on forever, but they are still enumerable; in fact, they are the smallest class of 'infinite' sets. While it seems confusing that infinite sets could have different sizes, you will soon become more comfortable with this distinction. The size of a set is called its **cardinality**.

Let us try to enumerate the letters of the alphabet by pairing each element with a natural number (we can start at 1).

1 A
2 B
3 C
.
26 Z

Okay, we are done! This is not an infinite class and of course it is enumerable. Let us try to enumerate the decimals from **0** to **1** by lining them off with the naturals. Where the hell do we even start? Do you go **0, 0.1, 0.2, 0.3, ... , 1.0**? Well, obviously, you missed **0.11, 0.15, 0.2374, 0.99**, and so many other decimal numbers! Suppose you want to use the smallest increment you can imagine, so you know you'll hit every number, say, **0, 0.0001, 0.0002, 0.0003**, and so on. What about all the numbers between **0.0001** and **0.0002**? Oh, I see, we have to use **0, 0.00001, 0.00002**, etc. Clearly, our precision is never quite good enough and we actually get stuck before even reaching **0.000000000...1**, and even *that* number times **10⁻¹**! This should be suggesting to you the possibilities for different cardinalities of infinite sets. Based on how we defined the natural numbers, standing in for whole quantities, they are *built to be countable*. It is the class of numbers that defines enumerability. Yet, once we introduce the increasing 'accuracy' of decimals, the absolute next step from any number seems impossibly arbitrary.

4.1 Rational and Irrational

Already we are sort of jumping ahead, so let us ground ourselves again by going to the next most interesting class of numbers, **rational** numbers, and **irrational** numbers. You may know a couple examples of rationals and irrationals, but strictly, rationals are numbers that can be represented as the ratio of two integers. This means that

rational numbers either have decimal expansions that eventually become all 0s, or they have decimal expansions with regularly repeating patterns. Any integer/natural is rational because it can be rationed with 1, obviously leaving no decimals. Ok, what are some more interesting rational numbers? $1/2 = 0.5000$, $5/4 = 1.250000$, $1/3 = 0.333333 \dots$. Remember, $1/3$ is rational because the pattern of decimals is clearly repeating 3s. What about the square root of 2? What about π ? Try to find a ratio of integers that *equals* these numbers, you will fail. What about $314159/100000$ for π ? Well, that is a clever idea but the decimal is just **3.1415900000**, not really π . If you have found an integer ratio, it is not precise. There are some numbers that just do not conform to a rational relationship of integers and these are called **irrational** numbers. How enumerable are these new classes?

First to enumerate the rational numbers, we see that we'll have to be a bit more creative than with the natural integers. We saw what decimals could do to a counting scheme, so let us try to incorporate natural numbers into our enumeration. Instead of pairing off one rational number to one natural number, what if we stepped through the rationals by categorizing them based on the sum of the integers involved in their construction. For instance, the sum of the integers in $1/3$ is $1 + 3 = 4$. $2/17$ would be in the 19-class. 0 and 1 together can only yield rational numbers equal to 0. In the 2-class, we have $1/1$ and $0/2$, giving us the new rational number 1. Moving to 3 we have $2/1$, $1/2$, and $0/3$, giving us 2 and $1/2 = 0.50000 \dots$. Clearly in each class, there will be many many more rational numbers we can find, but that is okay. The point is not to put them in 1-1 correspondence with the naturals, it is to illustrate the enumerability of the rational numbers. We can see that in a given class, there must be a finite limit of the amount of rational numbers we can define. That is the key and that shows us that the rational numbers are enumerable. While there are 'more of them than the naturals' in a given range, they actually have the exact same cardinality. As do the even numbers (1-2, 2-4, 3-6...). The key is matching them with naturals in a limited way. Another way to think about enumerability is the ability to 'sanely' count items and make progress, as opposed to the never-ending (or never-starting) process of counting all decimals from 0 to 1. What about irrationals? Hmmmm, this seems a little harder now because we do not have such a system we can use to build ratios, because we *know* these numbers have no ratios that define them. Spoiler, they are not enumerable (just think: decimals are either rational or not, and we know that decimals as a whole are not enumerable...). However, to see it fully, we must go a bit further.

4.2 Transcendental

Irrationals contain numbers like the square root of 2, π , and any other number whose decimal expansion is random and non-repeating. But are all these irrationals equally 'definable'? Are all irrationals the same kinds of beasts? We know we cannot use integer ratios to standardly define irrationals, but what about something like algebra: can we set up an explicit algebraic equation with one variable, x representing the number in question, that altogether equals 0? **Algebraic** numbers are ones who fit into this class. First off, all naturals and irrationals are algebraic. We could define, say, 6 with the equation: $x - 6 = 0$. We could define $1/3$ with: $3x - 1 = 0$. In both cases, x *precisely* equals the number in question, with absolutely no ambiguity. Can the same be done for the square root of 2, a famous irrational number? $x * x = 2$. There we go, x *precisely* equals the square root of 2. Thus it is an irrational but algebraic number. Now what about π ? You may first try something like: $10x - 31.4159 = 0$? Well, we will get **3.141590000** if we carry out the precise division. Maybe you want to take a circle you know the dimensions of and try to set up such an equation there to isolate π ? Guess what, you will never precisely do it; the other numbers in the equation will inevitably be approximations of π ! There is no possible way to algebraically define π with precision, thus making it **Transcendental**. While both it and the square root of 2 are irrational, transcending integer ratios, π distinctly transcends even more complex algebraic equations. Our tools of arithmetic simply do not have the precision and power necessary to express π 's value in numbers. You could certainly find very precise approximations, or you could find physical models or representations, or you could use an infinite sum that precisely approaches π , but you just cannot define it finitely with just algebra. Same with **Euler's e**. Transcendental numbers are the class of numbers that begin to seriously challenge what we believe to be true and logical. If we cannot define them algebraically then what are they?

4.3 Real

Having established these classes of numbers, we can ask what **computable** numbers are. It seems like a weird and pointless question because it feels like any number could be computed, even if transcendental. For instance, π and e can be expressed in infinite sums of terms that continuously get smaller and smaller, approaching the true value with increasing precision. We can show this with mathematical notation on paper, and we can certainly program computers to very well approximate (and with infinite time, calculate) these transcendentals. Some transcendentals are computable, and now it should be easier to see that any irrational, rational, or natural number

is also computable. Some of these classes are definable and computable with mindless algorithms, others require more thinking, but nevertheless, they can be explicitly instructed. There may not be a pattern to the *number* but there is certainly a pattern to the *calculation* of the number that does not require infinite instructions. For instance, the square root of **2** can be calculated to a point with a simple finite list of instructions (as discussed earlier, guessing and checking numbers until one surpasses, then guessing new ones, etc). Even **e** can be finitely programmed. While the terms in the summation are infinite, the pattern to add the next one is easily explainable: $1 + 1/2! + 1/3! + 1/4! + 1/5! + \dots + 1/n!$. We could have (and have almost) written instructions, finitely, to produce **e**, a transcendental number. This number is thus computable. We will later see a formal definition of computable numbers as those computable by **Turing Machines**, a theoretical construct of a computer that we will know very well very soon. For now, just think about it like a number that can be calculated by reading a list of finite instructions (however meta those instructions are, like for **e**). A similar infinite series can be done with π , leading us to believe that any transcendental number can be expressed in some kind of way, but this will be proven false. In fact, the cardinality of the computable numbers is provably less than the real numbers, greater than the natural numbers, and somewhere weird in between.

Finally, we have the ability to talk about **real numbers**. Reals are the set of numbers that are transcendental, irrational, rational, and natural, computable or not. It is all numbers whatsoever, and they *certainly* are not enumerable. In fact, the vast majority of (and in the probabilistic limit, *ALL*) numbers are transcendental. Think of all the transcendentals between **0** and **1**. We can easily think about these like $\pi - 3$, **e** - **2**, and other numbers with infinite random decimal expansions.

To finally show the absolute difference in enumerability in the reals vs the naturals, we will consult the Russian mathematician **Georg Cantor**, who was enthralled by these discussions of sets of numbers, infinity, and the different families of infinite sets. One of his most amazing proofs used **Diagonalization** and showed the uncountable infinity between **0** and **1** and thus the non-enumerability of all real numbers. The method he used for proof is called a **proof by contradiction**, or *reductio ad absurdum* (reduction to absurdity), which essentially means assuming the thing you want to refute, then showing that it leads to some absurd logical contradiction. In this case, since he is proving the non-enumerability of numbers between **0** and **1**, he first assumes that they *are enumerable* and that we have enumerated them in a list. Suppose the list contains some numbers like:

1. 0.**1**000000000...
2. 0.3**7**474800000...
3. 0.28**3**848584000...
4. 0.000**0**2220000200...
5. 0.1234**5**6789123456...
6. 0.92468**2**4680246...

First we must establish that each of these numbers has an infinite trail of decimals (even if that trail has bouts of repeated values **0s** after a point). This list has lots of rationals and irrationals and transcendentals. Now, to show that the list can never end, we just need to show that for **any** such 'complete' list, there is **always** at least one missing value. We want to create a number that we know is not on this list but should be: to do so, we can imagine starting a new number called **C** which takes the **first** digit from the **first** number on the list (0.**1**0000...) then the **second** digit from the **second** number on the list (**second** digit from 0.3**7**4748...) then the **third** from 0.28**3**848584..., etc. After just looking at our little sublist, we have **C** = 0.**1 7 3 0 5 2**.... Now, we uniformly change each digit by, say, adding **1** (and if it's 9, make it a 0), **C** now becomes 0.**2 8 4 1 6 3**..., and we can proudly say that it could not have possibly been in the list already. How do we know? Well we can see that it is different from the **first** number in the list *at least* at the **first** digit, that it is different from the **second** number on the list *at least* at the **second** digit, and the same for the **third**, **fourth**, ad infinitum. What sorcery was just used?? Even with this new number, we can keep applying diagonalization (provided the decimals are all infinite!) and continue to produce new numbers. If this seems contradictory or mind-melting, it is because our last assumption, *that you could ever completely enumerate the real decimals between 0 and 1*, is faulty and provably leads to illogic. It is a beautiful and powerful process (proof by contradiction *and* diagonalization, both of which play massive roles later on). We now see that real numbers (thanks to irrationals and transcendentals) are a somehow bigger (or at least less countable) infinity than natural numbers and this is a significant thought for mathematicians and even philosophers, who now may be queasy about our natural-seeming feeling about real numbers that we have seen leak from logic.

Another similar diagonal and contradiction-based proof will help us think about enumerability, numbers, the calculus of proof, and problem solving in general. It is Euclid's proof that prime numbers are also infinite, primes

being numbers that are only cleanly divisible by themselves and 1. Long before Cantor, Euclid showed that for *any* supposed complete list of prime numbers, there will always be at least one higher, inductively showing that they are infinite. The proof relies preliminarily on a more fundamental proof, called the *Fundamental Theorem of Arithmetic*, that shows that any number can be expressed as an equation of primes. Any number can be factored to prime numbers.

Firstly, any **prime** number is a factor of itself and **1**, thus factorable by primes. For any even number, we know that at least we can factor out **2** (repeatedly if the resultant dividend is still even... if it goes on until you have just 2s, then it is a power of **2**, certainly a product of many primes, **2**) and get an odd number. We have the **2** as a factor, then an odd number. Any odd number is factorable by primes because you can count up from **1** to **n**, testing if anything divides evenly, and if nothing does it is prime. If something does divide cleanly, then you can apply the same process with *that* number. Concretely, any number is either composite, in which case you can keep dividing, or prime, in which case it is certainly factorable by a prime. This is not the major proof and it is very fundamental and accepted and rigorously shown.

Provided this theorem, we can now show the infinitude of primes. Suppose, again, that we have enumerated all prime numbers into a list. (**2, 3, 5, 7, ...**). To create the next prime number, we simply multiply each of these numbers together to achieve a new number we will call **C**. In this example, $C = 2 * 3 * 5 * 7 = 210$. Is this number prime, well no, because we could easily divide out any of these numbers and get another composite of the rest. Here is the trick: add **1** to **C**, making **211**. Now, it is divisible by **2** with a remainder of **1** (**105 rem 1**), it is divisible by **3** with a remainder of **1** (**70 rem 1**), by **5** with a remainder of **1**, and of course by **7** with a remainder of **1**. Well well well, we have found a new prime not on our list. Add this to the list, and we can do the same thing again, forever. You may notice that **211** is certainly not the next prime number (what about **11, 13**, etc), but that is absolutely unrelated to this proof. We are just showing that there is always at least one higher, not producing the next prime necessarily. The diagonal aspect is because we are uniformly using our proposed complete list, then introspectively doing something to the composite of those elements. Indeed a valid, if cumbersome and weird, proof of the infinitude of all numbers can be shown in this way by just using a list of numbers and not just primes. This is called a factorial (denoted by **!**, $3! = 3 * 2 * 1$). Undeniably a **Factorial(n) + 1** is not divisible by any number from **2-n** evenly. Borrowing from linear algebra, where we map high dimensional spaces with basis vectors, each allowing movement in some dimension (for instance in 3 dimensions, with $[1\ 0\ 0]$, $[0\ 1\ 0]$, and $[0\ 0\ 1]$, we could combine these linearly to span the entire space of 3d, because each vector affords us direction in **x**, **y**, and **z** directions), these prime factors in the composite **C** that we then add **1** to are something like basis vectors for the space of natural numbers calculable by products. The whole point of primality, for instance with **7**, is that the number just cannot be produced by multiplication of anything other than itself. **2, 3, 4, 5**, and **6** just cannot combine cleanly with multiplication to get **7**. It is like a dimension that the vectors just cannot reach with multiplication. If you look at **C** before the addition of **1**, it behaves very specifically: say we have $2 * 3 * 5 = 30$. Divide **30** by **2** and you have **15** ($5 * 3$), by **3** you have **10** ($2 * 5$), by **5** you have **6** ($2 * 3$). You see that each contributes uniquely to this composite number and it is *always* not prime. By construction! However, when we add **1**, we escape the grip of the basis. The simplest example to check would be **2** (is this new number cleanly divisible by **2**? Well, it is just **1** off!). Then for **3**, you see again that you are just off by **1**!. How about this intuitive view too: **C** is always even before you add **1**, because it is multiplied by **2** in the beginning. Adding **1** immediately means it is odd now, knocking **2** out of the question as a divisor. What if you take out another number like **3**? Before adding **1** to **C**, this becomes yet another even number (will be even until we take out **2**, damn), and after adding **1** we can see that taking out **3** will leave yet another *odd* number. Would've been even if not for the **1**! It is a genius proof and does lots to **prime** your mind for the ideas yet to come.

5 Turing Machines

Having a foundation now of computation, recursion, and the nature of different classes of numbers, we need to explore **Alan Turing**'s true insight that allowed him to A) prove the unsolvability of *The Decision Problem for First-Order Logic*, and B) completely awaken the world to the idea of programmable universal machines, what we now know as computers. In our previous discussions, we aptly defined processes for producing and defining all natural numbers, many operations on those numbers, the processes to produce rational and irrational numbers, π , **e**, and other transcendentals, and other tasks like comparing two numbers (**0** and **0**, say, for their count of stars) for equality. These are all processes that, ultimately, could have been done by a monkey, a baby, a human who has no idea what numbers are, or a sophisticated machine. Before we think about programming, imagine a machine that spits out on a piece of paper the natural numbers from **0** onward. We could imagine a wheel with symbols (**0, 1, 2, 3, 4, ..., 9**), some gears to handle what happens from **9**→**10**, **99**→**100**, and other steps adding a new

digit. It is a very mechanical process, and machines like this (and ones to, say, solve differential equations) existed before **Alan Turing** and **David Hilbert**, but the unfortunate factor was that they were hard-coded to do one specific category of task. Look even at a modern mechanical watch—no binary or circuits involved, a successful, but highly specialized machine. Yes, it is awesome to make a machine to crunch out calculus problems, but it would be useless if we wanted to, say, enumerate the natural numbers, compare two lists of symbols, or really do *anything except* solving differential equations. Wouldn't it be nice to have a machine that is capable of doing *many* different things, or even *any* programmable thing (I say via typing keys into my computer, saving to the cloud)? Something like the programmable behemoth of intelligence we enjoy? While it seems like we are switching gears ;) Turing somehow intuitively felt that in order to prove some amazing things about numbers and the limits of formal systems discussed above, he could define a flexible (by way of being strictly defined at base) kind of machine that could compute numbers, sort things, and even take instructions and use those to do new things! Turing's definition of computable numbers is simply numbers computable by a Turing Machine, clearly something that needs to be pretty versatile. It will feel alien, it will take some rereading, but the only way to begin the education process is just to dive right in!

5.1 The Basics...Are All You Need

In its most basic definition, a Turing Machine consists of a *tape* (an infinite tape of paper with distinct squares) that can be scanned, edited, and shifted by a *head* that, somewhere in it, contains instructions called *states* or *configurations*.

$$\begin{array}{c} \text{state}_i \\ \left[\dots | 1 | \boxed{@} | A | 9 | \pi | * | \dots \right] \end{array}$$

The head can move one square to the left, to the right, it can stay still,

$$\begin{array}{c} \text{state}_j \\ \left[\dots | \boxed{1} | @ | A | 9 | \pi | * | \dots \right] \end{array}$$

or

$$\begin{array}{c} \text{state}_k \\ \left[\dots | 1 | @ | \boxed{A} | 9 | \pi | * | \dots \right] \end{array}$$

it can read the square, rewrite something on the square, or erase the square (print a blank), based on the states and configurations that define it.

$$\begin{array}{c} \text{state}_x \\ \left[\dots | \boxed{Z} | @ | A | 9 | \pi | * | \dots \right] \end{array}$$

or

$$\begin{array}{c} \text{state}_y \\ \left[\dots | 1 | @ | \boxed{8} | 9 | \pi | * | \dots \right] \end{array}$$

or

$$\begin{array}{c} \text{state}_z \\ \left[\dots | 1 | \boxed{} | A | 9 | \pi | * | \dots \right] \end{array}$$

And that is it! It may seem highly unsatisfactory, but this rule base is all you need. We will go much further with examples, but truly, all of computation can be modeled by such a simple system. As a quick aside, the common computer architecture today (called Von Neumann architecture or RAM architecture) is a similar process of a gigantic piece of essentially scratch paper, divided into squares that hold either **0** or **1** or nothing, and are read from, written to, and located by a head of sorts, called the CPU, which can do very simple math and logic operations and has a very simple set of instructions. With this architectural foundation we can do *everything and anything* that you can imagine or instruct.

5.2 Conventions

Back to Turing Machines, we will start with an example of a machine that prints out a certain number. To do this we need to go over some of Turing's definitions and conventions. Obviously, we need to suspend disbelief that we can have an infinite tape: we must accept that we do. You could alternatively imagine a finite tape that could continually just add squares on the right or left if needed. The key is an *arbitrarily* large memory. Next, we must discuss how these machines even work in general: the tape starts blank, then according to the state the machine is in and the symbol it is scanning on the square, it *does* something, then switches to a new configuration. While most computers, functions, and programs today are meant to *do* something then stop, Turing only wants to talk about machines that go on forever. After all, there is an infinite tape and this is a theoretical construct. These forever-running machines are called *circle-free* and those that halt, break, reach a contradiction, or otherwise stop are called *circular*. That is, even if the machine is calculating the binary representation of $1/2$, **0.1**, it will hopefully print **0.10000000...00000... forever**. Infinite tape, infinite writing, it is all okay. The circularity consideration has massive ramifications, so let us trust Turing here. A couple more things. As alluded to, this machine will operate with binary digits. It makes it a simpler machine to construct (less symbols needed), and you'll see, it lends itself much better to digital computing (as we know now by all computers being *made up of 0s and 1s*) and logical expression.

Binary, quickly, just denotes powers of **2**, like **base-10** does powers of **10**. The digits to the left of the decimal are the powers of 2^x where x is the digit's index from the point. **1 1 0** is $(1 * 2^2) + (1 * 2^1) + (0 * 2^0) = 6$, just as, in **base-10** it would be $(1 * 10^2) + (1 * 10^1) + (0 * 10^0) = 100 + 10 + 0 = 110$. And for decimals, it corresponds to powers of **2** with negative indices. So the binary form of $1/2$ is **0.1** ($1 * 2^{-1}$).

Finally, we must say that since we have infinite room on the tape, we should use it wisely. The 'output' we are computing is to be written in binary on every other square of the tape, called **F-squares**, and other squares in between, **E-squares**, will be used as kinds of scratch paper.

$$\left[\begin{array}{|c|c|c|c|c|} \hline \text{E} & \text{F} & \text{E} & \text{F} & \text{E} \\ \hline \end{array} \right]$$

5.3 Our First Machine

Also, we are only writing decimal expansions with these Turing-convention machines. He is not inventing this machine to create computers, he is doing it to prove something about real and computable numbers and, as we know, the span from $0 \rightarrow 1$ is just as uncountable as that from $0 \rightarrow 100$ or $0 \rightarrow \infty$, with respect to real numbers. Everything he does with these machines can easily be projected and applied up onto a larger continuum of numbers. This is why we can restrict ourselves to decimal expansions and spare no information or applicability.

With that, we are ready for some examples of machines. Machines, as we said, are governed by *configurations*, which look like functions in programming, or definitions of functions, as in algebra. The anatomy of a configuration contains **5** parts: the configuration name/number, the scanned symbol, the symbol to be printed, the directional move to make on the tape, and the next configuration to switch to after this one is done). It makes up a quintet, such as **Q₁ – 0 – 1 – L – Q₂**, which would mean: “in configuration **Q₁**, if the scanned symbol is **0**, print **1**, shift **Left** one square, then go to configuration **Q₂**). **CONFIG-SCANNED-PRINT-MOVE-NEXTCONFIG. Q** is a conventional name for configurations, and from here on out, symbols like **0**, **1**, and others will be numbered like **S₁**, **S₂**, **S₃**, etc, where **S₀** actually denotes a blank square. So, the configuration **Q₈ – S₀ – S₃ – N – Q₂** would mean in **Q₈**, if we see a blank square, print symbol **S₃** (which could be, say '?'), move nowhere (**N**), then go to **Q₂**.

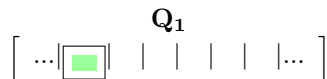
Let us work through an example now, where we have the configurations to produce the number $1/3$ in binary (which is **0.01010101010...**). We have:

- **Q₁ – S₀ – S₀ – R – Q₂**

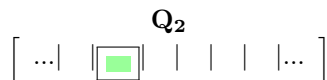
- $Q_2 - S_0 - S_1 - R - Q_3$
- $Q_3 - S_0 - S_0 - R - Q_4$
- $Q_4 - S_0 - S_2 - R - Q_1$

It will step through Q_1, Q_2, Q_3, Q_4 , then eventually come back to Q_1 and repeat. This machine is very simple because it only moves right and it really does not need the E-squares because its pattern is so simple. However, without a doubt, it computes the decimal expansion of $1/3$ without ever having to define numbers or anything of the sort. 1 and 0 could easily be X and Y , but we may choose these somewhat arbitrarily, and they do not at all necessarily mean numbers. However, obviously, for Turing they will be treated as numbers. Let us trace through how the tape evolves as we go on. While the tape is infinite, I'll use a finite one here but just keep adding squares as needed.

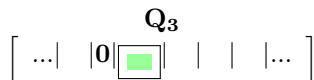
We start in Q_1 with an empty tape:



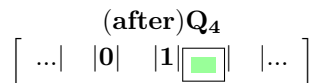
We see that we have scanned S_0 (green is scanned), so we write S_0 , which amounts to changing nothing. Then we shift right and go to Q_2 :



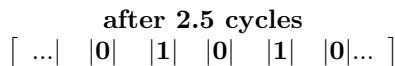
which prompts us to write in S_1 to the scanned square and go to Q_3 :



which tells us to, when scanning a blank, move to Q_4 after printing nothing and shifting right, leading to:



and go back to Q_1 . The tape will evolve to become:



...and print this expansion forever. This is cool and all, but it does not really seem powerful enough to do anything but print out alternating 0s and 1s! How is this the programmable future of computers that also solves the decision problem? To see this, we have to go a bit further in our exploration of what a specific Turing machine is. Clearly we have 'hard-coded' this machine to do one specific thing and it is not thinking or reasoning about anything. We could definitely conjure up other specific machines to do other more complex things, like print the decimal expansion of the **square root of 2**—to write out **101001000100001000001, etc**—or counting the natural numbers in unary **1s** separated by a **0** at a time. There would be many more Q configurations and lots more tape used, but it surely could be done by such a machine. Before expanding, it will be necessary to refine our notation for these machines.

5.4 Standardizing Machines

Whereas before, we wrote the instructions, the machine configurations, with letters, numbers, and dashes between, since we know that configurations have a standard form and that there should always be 5 terms, we can ditch the dashes and denote configurations and symbols with letters to obtain:

$$\begin{aligned} Q_1-S_0-S_0-R-Q_2 &\rightarrow \text{DCDDRDCC} \\ Q_4-S_0-S_2-R-Q_1 &\rightarrow \text{DCCCCDDAARDC} \end{aligned}$$

Essentially we replace Q_n with **D** followed by n **C**s, we replace S_n with **D** and n **A**s, **R** and **L** and **N** stay the same. We are trying to minimize symbol usage so while using **D** twice seems misleading, it acts as a sort of sentinel for each term, knowing that trailing **C**s always mean a configuration and **A**s always signify a symbol. Finally, for a given machine **M**, we chain together all machine configurations in this notation with ‘:’, so the **Standard Description** for our example 1/3-calculating machine **M** is:

$$\begin{array}{cccc} Q_1 & Q_2 & Q_3 & Q_4 \\ \text{DCDDRDCC} : \text{DCCDDARDCCC} : \text{DCCCDDRDCCCC} : \text{DCCCCDDAARDC} \end{array}$$

This is the **Standard Description** of **M**. While, at least for me, this is an immediately satisfying encoding, not sparing any atom of information in the new concise shape, it comes with very important implications as well that are more than two-fold. First and most romantic, Alan Turing is known for many things, but among his most famous, he invented the famous artificial intelligence test called the Turing Test that says if an intelligence (no form specified) can hold a conversation with a human and convince the human of its consciousness, it may as well be considered conscious. *I think, therefore I am*. While not exactly related to **SD**s, Alan Turing, on top of being a mathematical and computer science genius, was always weighing his results with the mysterious mechanisms of our mind. This format of **SD**s, at least only aesthetically (and indeed much more than this respect), immediately evokes the analogy in our heads to **DNA** and the base pairs that mindlessly organize, a genetic code, that is expressed over time in organic life. We have clearly shown that this very short **SD** can produce with increasing accuracy the ratio of 1 to 3, all represented here by organization of a handful of little symbols that we arbitrarily assign letters to. Looking at this code or even output out of context would do nothing for you to suggest that it divides 1 by 3, yet we know that it, in a certain interpretation, explicitly shows a machine how to do so. Some machines that do more interesting things will have only a handful of configurations, a short code containing some powerful computation. If you look at a human, or even bacteriophage virus, **DNA** on a page (many), it will look like utter gibberish, yet it is capable of directly informing the construction of an incomprehensibly accurate and powerful life. The ubiquity of simple coding schemes, on an extremely fundamental level, combining repeatedly to represent much more complex ideas or things is universal in the world of philosophy of consciousness, mathematical logic, and computer science. From simple pieces and rules, we can get a lot done and represent it very very neatly.

The two more practical outcomes of this notation are powerful. Because the latter of the two reasons leads directly into the next chapter, we will briefly visit with the numerical implications of standardizing a machine’s function. Bearing in mind that one of Turing’s principal concepts of focus in this paper and invention is to show that computable numbers form a set that is of smaller cardinality than the real numbers. Not an uncountable infinity. Earlier we discussed an enumerable infinity as one that can be sequentially counted and paired off with the natural numbers reasonably. Since we now have these very minimally complex yet very informative **SD**s, we can easily convert them into numbers. Why would we do this? Well, as we will see from Turing and Gödel, getting things into the domain of natural numbers helps a lot in coming to conclusions and proving things about number theory. If we can create a unique integer for every single Turing Machine possible, then we have defined a subset of the natural integers that correspond to specific computable numbers/sequences. If the integers which describe circle-free Turing machines are a subset of the natural integers, (clearly it is because for instance the number “1” is not even a complete configuration let alone Standard Description number), and we know that any machine produces at most one computable sequence/number, then we can easily say that the set of computable numbers/functions/sequences is enumerably finite. How do we transfer to numbers? Well, very arbitrarily and simply.

We could say **D** → 3, **C** → 5, **A** → 7, **R** → 8, **L** → 9, **N** → 6, and : → 1

Completely arbitrary, but very effective and complete. Our **SD** from before now becomes the unique, if gigantic, integer:

3533835513553378355513555338355551355553377835

from DCDDRDCC : DCCDDARDCCC : DCCCDRDRCCCC : DCCCCDDAARDC

Anyone who has read this chapter will immediately see how it can mechanically be decoded and rewritten with Qs and Ss and even in English, or any other programming language for that matter. We will revisit the question of the cardinality of computable numbers, but already it seems clear that any computable number or sequence corresponds to at least one of these **SD** integers and more importantly, every single **SD** integer corresponds to at most one computable number. Since these **SD** integers are clearly a subset of natural numbers, the computable numbers are enumerable and limited. Not limited in the sense of there being a ‘last’ one, but certainly limited with respect to the real numbers; enumerable. We know the space between 0 and 1 is just as hard to count as all real numbers, so we clearly already see that the space between computable numbers and real numbers is huge.

The final consequence of the **SD** notation is the fact that it is now a fully fledged program which can be mechanically executed by another Turing machine. Imagine a Turing Machine, **U**, that has its own set of configurations that can do certain tasks, and starts with a tape pre-filled with **M**’s standard description: ‘**DCDDRDCC : DCCDDARDCCC : DCCCDRDRCCCC : DCCCCDDAARDC**’. Imagine it had configurations to find the end of this sequence, maybe print something like a ‘:’ or ‘::’ then begin executing the code provided by this **SD**. What would we need? We would need to be able to find the beginning of these configurations, it would need to be able to interpret what each of **M**’s configurations prescribes as its scanned symbol and comparing it with **U**’s current scanned square, then it would need to be able to translate something like **DAA** to **S2**, a symbol within the head of **U** so it could read and print, then it would need to know how to go to the next configuration and repeat. What we are describing is a programmable machine that is capable of reading the **SD** of any given machine **M**, then replicating the ‘motion’ of **M** on its tape. A very powerful idea.

6 The Universal Turing Machine

We left with a fantasy of a **Turing Machine**, **U**, that could be given on its tape the **Standard Description** of another machine, **M**, and replicate **M**’s output. In other words, **U** could be a ‘programmable’ Turing machine that obeys instructions for how to behave. An idea that comes close to how modern computers still operate, this machine will be far more complex than our example machines before. We will need to establish the tasks that are needed in order to replicate **M**’s motion mechanically, then discuss those tasks’ viability for a Turing Machine, **U**. One useful standard will be the **Complete Configuration (CC)** of the machine at any given time, which looks like a large string containing the entire contents of the tape, where before the scanned symbol there is the **SD** notation for the current configuration. Supplying our machine **U** with the **SD** of machine **M** enables us to continuously leave a snapshot of which part of **M**’s program is being run (when and where), so stepping through **M**’s motion is as simple as a progressive rewriting of these **CCs** representing each ‘moment’ of **M**. On **U**’s tape will be written sequential **CCs** with the ‘new’ symbol printed on **M** (that makes its way into the new **CC**) printed between **CCs**. So in copying **M**, **U**’s output will be densely packed with progressive descriptions of the entire tape. This is precisely what allows **U** to actually do the work of emulating **M**. Due to the deterministic nature of the **SD** and the Turing Machine, the transition from one **CC** to the next is a simple and trivial symbolic manipulation, with absolutely only *one* possible next evolution, dictated right there by the previous **CC**. Examples always do better than explanations.

Let us assume that somewhere in **M**’s **SD** written on the beginning of **U**’s tape is the following configuration: “...: **DCCC** **DA** **DAAA** **NDC**:...”. It refers to the configuration **DCCC/Q3** when scanning **DA/S1/0**. Now imagine an example **CC** (after already letting **U** run for a bit) like: “**DDADDAADDAD** **DCCC** **DA** **DD**”, which would correspond to a tape (on **M**):

in DCCC/Q₃
 $\left[\dots \mid \boxed{0} \mid \boxed{1} \mid \boxed{0} \mid \boxed{0} \mid \dots \right]$

where the next configuration is **DCCC** and **DA** is scanned (convenient!). This **CC**, while already long and only getting longer, provides a very useful bookmark and snapshot into exactly ‘*where we are*’ in the progression of the machine. For **U**, this notation is extremely useful to write after each ‘round’ of mimicking **M**’s motion;

that is after printing each successive character that **M** would print. Remember, **U** is a machine itself with its own configurations, but its goal is to mimic the movement of **M**, whose **SD** is written in the beginning of the tape of **U**. **U** needs to somehow print out all of the symbols that **M** would, so it needs to (on the tape!) continually keep track of how far it is along **M**'s motion. When using **D**s and **C**s and **A**s, it is always in reference to symbols and configurations of **M**. The **first section** of the **CC** denotes the **M** symbols that have been printed by **U**, though in reality, on **U**'s tape, these progressive **CC**s are interspersed between the printed symbols. **U**'s configurations are essentially invisible to **U** (they may be stored physically somehow in the head just like our earlier machines). The only configurations and symbols **U** will be writing on its tape in the **SD** encoding are those corresponding to **M**.

Supposing we have already gotten through several steps of **M**'s motion with **U** (since it is easier than the very beginning of **U**'s mimicry of **M**), let us see how we can use the previous **CC** to tell us what to do next. What we want to do is (**A**) print the next symbol that **M** would print and then (**B**) write the next **CC** for future use. First, we need **U** functionality to look at the **SD** on the beginning of its tape to find “**DCCC DA**”, which is the current configuration and scanned symbol (of **M**!). Since we are borrowing and intelligently using the **SD** notation, once we find this sequence in the **SD** at the beginning of the tape (the program), then we know the **D** with some amount of **A**s immediately following “**DCCC DA**” will tell us which symbol **M** would print next. Once we have found this next symbol, on **U** we print:

$$\left[\dots : \text{SYMBOL} : \dots \right]$$

where **SYMBOL** is deduced by whatever follows “**DCCC DA**” in the **SD** (there will be **U** configurations to turn something like **DAAA** into **S3** from **M**, which could be any symbol, say **?**). Once we have printed the next figure **M** would print, we need to write the **CC** for the next round. First we need to copy everything from the tape before the next configuration of **M** to the next **CC**.

From our above example, this would be copying “**DDADDAADDAD**” to the next **CC**, then, since we just printed a new **M** character, we print that character next (If the matching instruction was **DCCC DA DAAA NDC**, we would be printing **DAAA**). Then we copy whatever followed the previous scanned symbol, which in our example was two blanks denoted by **DD**.

To recap, assuming **U**'s tape at **t₁** was:

$$\begin{array}{c} \text{U} @ t_1 \\ \left[\dots |D| |D| |A| |D| |D| |A| |A| |D| |D| |A| |D| |D| |C| |C| |C| |C| |D| |A| |D| |D| \right] \end{array}$$

at **t₂**, we would have:

$$\begin{array}{c} \text{U} @ t_2 \\ \left[\begin{array}{l} \dots |D| |D| |A| |D| |D| |A| |A| |D| |D| |A| |D| |D| |D| |C| |C| |C| |C| |D| |A| |D| |D| \rightarrow \\ \rightarrow : ? : |D| |D| |A| |D| |D| |A| |A| |D| |D| |A| |D| |D| |C| |C| |D| |A| |A| |A| |D| |D| \end{array} \right] \end{array}$$

This is directly isomorphic to the transition on **M**:

$$\begin{array}{c} \text{M} @ t_1 : in \quad Q_1, \quad scanning \quad S1/DA/0 \\ \left[\dots \quad |0| \quad |1| \quad |0| \quad | \quad | \quad | \dots \right] \end{array}$$

⇓

$$\begin{array}{c} \mathbf{M} \quad @ \quad \mathbf{t_2 : to} \quad \mathbf{Q_3}, \quad \textit{printing} \quad \mathbf{S3/DAAA/?} \\ \left[\quad \dots \mid \mid \mathbf{0} \mid \mid \mathbf{1} \mid \mid \mathbf{?} \mid \mid \mid \dots \right] \end{array}$$

again assuming that in the **SD** on **U**'s tape before the “::” there was the instruction: **DCCC DA DAAA NDC**, and **M**'s **S3/DAAA** was ‘?’’. The **M**-configuration changes from **Q₃** to **Q₁** (**DCCC** to **DC**) and the symbol scanned changes from **M**'s **DA** to its **DAAA** (same as what we printed because the move instruction was **N**), which we said was ‘?’’. These changes are reflected in the new **CC** and the new symbol printed would be written in its **special intermediate spot**, between the colons. For time **3**, **U** would first search for “**DC DAAA**” (**Q₁** on **S3** for **M**) in the **SD** at the beginning of **U**'s tape.

What kind of computations did **U** do to do this? It needs to be able to find and compare sequences of squares to identify the current configuration and scanned symbol of **M** and the corresponding instructions in **M**'s **SD**. It also needs to be able to decode **M**'s scanned and printed symbols from the **SD** notation to the real symbols, like ? or 1 or 0. Then it needs to be able to copy and paste certain things. All of this can easily be achieved with a substantial set of **U** configurations, lots of which include marking the **E-squares** along the way with special characters to guide comparisons and copies. To elucidate the primitive nature of these computations, imagine a chain of configurations that could find a symbol marked on its neighboring **E-square** with “x” then copy that value to the farthest open square on the right. Suppose on the tape we have:

$$\left[\quad \dots \mid \mid \mathbf{1|x0} \mid \mid \mathbf{1} \mid \mid \mathbf{1} \mid \mid \blacksquare \mid \dots \right]$$

and we are in **Q₂**. Then you could write a host of configurations for copying **1** that would move right until two blanks are found then print **1** in the second blank. While we ultimately need a limited amount of configurations in the machine for it to be valid, there are no bearings on how high that number must be; we could have **COPY-S2-1** and **COPY-S2-2** configurations where the former will do *nothing but shift right* unless it sees a blank, then if it does, pass to the latter, which will pass back to the former if it scans anything except another blank, and if it does see a blank, it prints **S1**! This behavior could endow:

$$\left[\quad \dots \mid \mid \mathbf{1|x0} \mid \mid \mathbf{1} \mid \mid \mathbf{1} \mid \mid \mathbf{1} \mid \dots \right]$$

Then we could easily have a configuration to find **x** and delete it. While these even seem like hand-waving simplified examples—and they are—the core idea of simple, stepwise decision-making is absolutely the only necessary bit to grasp; seeing all of Turing's complex configurations (that even *he* used extensive shorthand for) is not as necessary as noticing the primitive nature of all of the possible things that this machine would need to do to emulate another machine from that machine's **SD**. Could it be done by a simple monkey or machine or child with a pen and paper? Nothing along the way requires deduction, problem solving, or intuition beyond a simple directive on the part of the machine, we just conjoin many simple steps. Importantly, too, *we*, in writing the configurations, are the beings which must exert intelligence, but once our wishes are explicitly spelled out, the machine can *act* intelligently.

For the record, since we started our demonstration by assuming we were some portion of the way along **U**'s emulation of **M**, I will now show how **U starts up**. All we really need to do in **U**'s setup is to find the “::” that separates the **SD** (program) and the output, then print

$$\left[\quad \dots \mid \mid \mathbf{D} \mid \mid \mathbf{C} \mid \mid \mathbf{D} \mid \mid \dots \right]$$

to signify that we are in **M**'s **Q₁** (**DC**) and the tape shows a blank (**D**). **U** will then find the corresponding instruction in **M**'s **SD** for a blank scanned square in **Q₁**. From here, we can see how things would progress, but starting with explaining this would have likely confused.

Clearly, there is more than just **0**s and **1**s being printed on **U**, but that is necessary. Modern computers require lots of other computation to simply execute an arbitrary program. The way this machine works is by continuously printing the machine configuration, the contents of the tape, the scanned symbols, basically all the metrics of the Complete Configuration of the machine at each step, then writing the next figure of **M**, then repeating. This is required so we can, at each step, syntactically and mechanically check which of **M**'s configurations corresponds to **U**'s current rendition of **M**'s motion. As we saw, there can be many different **Q₁**s, say, for a machine, but what distinguishes them is the scanned symbol they operate on. This practice of writing many versions of a function/configuration in programming is called overloading, and it allows us to embed conditionals into our program, to do the same category of function for different inputs. Turing's grasp of the importance of overloading and its perfect symbiosis with conditional loops is one of the core insights that spurred the explosive development of digital, universal computers. If we write enough code to account for all possibilities, no part of the machine has to do any complex thinking other than simply comparing one symbol to another, which is very trivial. Encoding an arbitrary machine's instructions into **SD** so that they can be unfurled by a versatile chameleon machine like **U** is what allows this all to happen. Just like Gödel saw, utilizing a well-crafted syntax is often the bridge from the complex and abstract to the trivial, mechanical, and primitive.

Now it becomes clear that the Turing Machines like **M** that we discussed to compute numbers/sequences are really more like programs and this machine, **U**, is the computer as we know it today. From here on out, it will be our vehicle to carry out the motions of other machines (or programs) we'll need to consider. While it was shown and used to help solve the **Entscheidungsproblem** for first order logic, **Alan Turing** struck gold on an idea that nobody had published before. One that has transformed our world in incomprehensible ways. He invented the idea of general programmability and still today, we are uncovering what power this discovery had. Another astonishing implication of this theoretical construct is that, as alluded to earlier, any algorithm, which really just means a strictly defined set of steps, can be emulated and executed by a Turing Machine. While our computers and minds are likely more like a confluence of many programmable Turing Machines working in tandem, connected to the outside world with less arbitrary hardware, they unarguably do things sequentially, governed by very very simple deterministic steps along the way, not thinking at all, just following the laws of its construction, the laws of physics, as it works to achieve an outcome (consciousness) greater than it could imagine. While our minds feel like they could not be discrete deterministic machines, nobody would argue that this is not the case for the brains of bugs and the chips that run in our computers. Our consciousness, our *mind* is more akin to something that reads at rapid speed only the **0**s and **1**s that are printed by **U**, not even seeing or perceiving the **E-squares**, **SD**, and work that goes into their production. Our brain, however, is just a discrete Universal Turing machine. Do we argue that **DNA** is expressed in a non-mechanical, mystical, 'conscious' way? Do we believe deep down that our brains defy time, do two things at once, or really do anything that does not have a physical and temporally discrete explanation? No.

7 Computable Numbers

Now we have covered the ground work required to discuss the nature of computation, its definition, and, eventually, its limits. As we saw before the machine, lots of the computation we felt to be automatic in our brains since grade school can be represented absent of all English, in a system of symbols and manipulations of those symbols, that we have meticulously crafted to actually represent concepts we know from math. The natural numbers, addition, multiplication, square roots, and other processes were shown to be mechanically definable with symbols and rules that rely on no human intelligence to carry out. In other words, they could be done by a machine. And now, we have a machine that operates based on very simple rules of symbol manipulation!

Computable numbers are those which can be computed by a Turing Machine. All naturals, rationals, irrationals, and even many transcendentals could be calculated and output by a Turing Machine, as we have seen, because they require only a finite list of instructions that pertain only to the shunting to and fro of symbols on the page or tape. The key is finite rules. Not rules that have anything to do with cardinality or numbers or anything, just simply a finite amount of rules and procedures. A recursive process. Any sequence computable with a finite amount of rules is a computable sequence.

So if we said that transcendentals are the cause for the uncountability of real numbers, but we have shown some finite algorithms to produce some well known and some arbitrary transcendentals (that is, we have shown the computability of some transcendentals) then what possibly could exist in the gap between computables and the

reals? Well, first a reminder from the angel on our shoulders that computable numbers, which can be identified by **SD** integers of machines that compute them, are as enumerable as the naturals. That seems very solid. However, back to our special transcendentals. Many people often describe **pi** and **e** as infinite chains of random digits, and that is how it appears, but each given digit in each place is anything but random. These transcendental constants represent very important and valuable constants we have found in the ‘natural’ world of math and physics, tainting their randomness. What about infinite random decimals that seem to be pointless? Can you compute those? Well, we must back up and really examine the contradictory question we just posed. If a process can be written to do something, is that thing at all random? No, it is directly explained by the process. In computing and statistics and other fields, we often require randomness. For example, the layout of many games like solitaire at the start depend on a random-seeming fresh start, for certain algorithms to train Neural Networks that require random choices. In any computer and most programming languages, there are methods that will generate a ‘random’ number on some interval for you, but ultimately this number is produced by what is called a pseudorandom process. John Von Neumann once aptly expressed that anyone claiming to have a formula for randomness is acting sinfully. I would agree. The algorithms on our computers to simulate randomness depend on a seed number to kick off a chain of convoluted, arbitrary seeming (yet always the exact same) calculations that produce a number, which can be scaled into any range requested. After it produces a number, it assigns that number to its next seed. Or maybe it gets its seed from the milliseconds on the clock at the time that the process ended. Wherever the seed comes from, at the end of the day, a given ‘random’ number can be traced and explained by a wholly nonrandom set of events. Even sophisticated random number generating hardware in computers, like the ones that use ambient noise or soundwaves to inform a seed, can be deterministically explained and thus predicted, provided one knows the mechanism it uses to ‘produce the random number’ or find a seed. The point is, for something to be truly random, it needs to untraceably pop up into existence for no reason whatsoever (is that even possible in the general case at all?). Clearly, any mechanism we could ever embed into a solitaire game, a statistical testing program, or, sadly, a Turing machine designed to produce all real numbers, even the random ones, will have to be defined explicitly and finitely. If you can compute it, it is computable and not random! The really eerie part of this discussion is that I cannot even give you one example, explicitly, of a non-computable number. If I said I randomly chose the number **27374.994845839283** by smashing my keys randomly, I would be lying. Undeniably, **LaPlace’s Demon** could explain what went on in my head and hands that caused this number to be produced. And I sure as shit could write up a Turing Machine to write this number. The creepy reality is that the probability of me ever writing down a non-computable, random number is **0%**. If it has a finite **SD** integer, then it absolutely is not a non-computable number, so we must accept that the set of computable numbers is a subset of the real numbers. What do non-computable numbers even look like, then? We cannot know. This conversation is not done.

8 What is Logic?

Now, to take a slight divergence from our path to meet a new friend, and inspiration of Alan Turing, Kurt Gödel, we must first learn about formal logic. Before touching formal systems, it makes sense to cover the groundwork of what we consider logic, how it can be expanded, and how it can even lay the foundation for a formal definition of number theory. While, as humans who can communicate on paper and in voice, we are familiar with the concepts of truth, falsehood, contradiction, and other necessary laws that govern reasoning and deduction, but we can wonder: are such systems of reasoning consistent? Do they make sense? Is there some underlying structure to truth, falsehood, implication, and whatever else we call logic with which we can unambiguously prove, for instance, that the light on my computer screen cannot both be on and off simultaneously? That, outside, it must either be raining or not raining? Can we show that if I am older than my brother Will, and my brother James is older than I, that James must be older than Will? These laws we will soon formalize underlie our entire system of thinking sanely, gathering and using evidence, even communicating in the most basic way. Long before language, there was a semblance of a logic in early humans that allowed them to draw conclusions, to believe in something because of evidence, to build the early pillars of intelligence. Clearly that system of thinking caused us to flourish, it brought us communication, collaboration, and scientific deduction of all kinds, so as philosophers and early mathematicians and logicians yearned for answers, definiteness, and certainty in the world, people started formalizing logic. At its most basic level, logic deals with truth and falsehood, two ideas that seemingly cannot both be present for a given proposition, such as ‘is it raining’? Logic as such that deals with sentences is known as **Propositional Logic**.

8.1 Booleans

As with the examples above, sentences can be false, and sentences certainly may be true (like this one), but never ever both. As humans we immediately imagine situations that we have felt may be contradictory; ‘Joe is a bad guy’ is certainly a sentence and indeed a proposition, but there is no quantifiability to being a good guy. It is subjective. It is the antithesis of the purpose of formalizing logic into such a system. While we discuss propositional sentences, the reader would do best to simply imagine either very objective thoughts in English, or even simpler sentences made up of boolean variables. Boolean variables, which **can either be True or False**, were coined by **George Boole**, who lives on vastly today for all programmers. With the concept of a boolean variable, we can explore how they can be combined into a sentence, a proposition. In explaining their existence I even used a primitive logical connective, **or**, which I required to express that booleans **can either be True or False**. We have two connectives in logic. Are you ready? ‘Or’ and ‘and’. It would be nearly impossible for any of us to use English without these logical connectives, as they are so firmly ingrained in how we think. And, you will notice, they only work on more than one thing, a testament to their ubiquity in the most primitive definitions of number theory. From here on out in examples, not in regular speech, these two connectives will be denoted as $\vee$ and $\&$, respectively. Anyway, propositions themselves all must evaluate to either **True or False**, as they are purely made up of booleans and boolean connectives. How about some examples: ‘ $P \vee Q$ ’ can be true or false. When could it be true? Whenever *either* **P OR Q** is true, this statement will be true. Because if I said “It is either going to rain today or it will not”, I would be considered correct if either case was actually true. Clearly they cannot both be true at once. However their logical sum, the $\vee$, will be true if either is. ‘ $P \& Q$ ’ will not be true unless both **P AND Q** are both true themselves. Again, if I were to replace the “or” in my example sentence with the word “and”, I would always be wrong. $\&$ is true for $1/4$ possible combinations of **P** and **Q**, whereas $\vee$ is true for $3/4$ possible assignments of **P** and **Q**. Lots of symmetry in this stuff, eh? Now for an important example: $P \& Q \vee R$ will be evaluated first on the $\vee$ operator, making a situation that conjoins **P** with either **Q or R**. We could instead write: $(P \& Q) \vee R$, which would evaluate what is inside the parentheses first, making **Q** and **R** a bundle, then be posed as an option, either this conjoined term or **R** needing to be true. These connectives are binary operators, acting on two variables at once, however there is another vitally important logical operator, a unary operator. It is **not**, which we will denote as $\neg$. That sentence doubtless read odd the first time through, did it **not**? This operator inverts the boolean ‘truth value’ of a variable, such that if **P** represents ‘*it is raining*’, then $\neg P$ represents ‘*it is not raining*’. It could be instantly seen as obvious that: $P \& \neg P$ could never be true, and $P \vee \neg P$ is always true. Again, the connectives showing their yin and yang. These simple propositions will be more important than knowable at this point, so it is vital that their mechanism is understood and appreciated.

Before moving on, a final comment must be made with the introduction of a new operator of sorts. It only exists as a tool for explanation, it is not a part of propositional logic whatsoever. What it strictly denotes is equivalence of the **truth tables** of multiple propositions. Truth tables are simple combinatorial tables that show the evaluated boolean values of a proposition under **all** possible combinations of assignments of the boolean variables in the proposition. For every boolean variable, like **P** or **Q**, there will be **2x** more possible propositions to evaluate. A statement with just **1** variable has **2** possible values. A statement with **2** boolean variables will have **4** possible values, and on and on. As discussed, the truth table of $P \& Q$ would have **True** written in **1** out of **4** boxes with **False** written in **3**, and in that of $P \vee Q$ would be **3 True** boxes with only **1 False** assignment of **P** and **Q**. Our new **truth table** equivalence explanatory operator is “ $=$ ”. With it we can now discuss **DeMorgan’s Law**, $\neg(P \vee Q) = \neg P \& \neg Q$ and $\neg(P \& Q) = \neg P \vee \neg Q$. It just tickles the mind to imagine examples to try with these. Even just imagining for yourself an explanation of either equivalence in a sentence, feel how kind it is to your head and your sense of intuition. These operations feel very powerful and fundamental because they truly compose the atoms of our sanity and reasoning. Boolean algebra and propositional logic more than lays the foundation for all of language, ask a linguistics student. Interestingly, DeMorgan’s Law can be slightly reorganized to read $P \vee Q = \neg(\neg P \& \neg Q)$ and $P \& Q = \neg(\neg P \vee \neg Q)$. It is elegant and sensible. The third binary operator is nuanced but extremely powerful. It is called *entailment*, denoted by “ $\rightarrow$ ”, and it relates two boolean statements based on their shared truth tables. However it is a one-sided relationship. $P \rightarrow Q$ being true means that whenever **P** is true, **Q** is also true. For instance, $P \rightarrow P \vee Q$ means that the second term—the *disjunction* of **P** and **Q**—is always true when **P** is and indeed is a true proposition. Interestingly, $P \rightarrow Q = \neg Q \rightarrow \neg P$. **Q** needs **P** to be true, so if **Q** is false, clearly **P** was not there to make it true. Finally, we have one last connective we will only express in English, **if and only if**, which can be represented as $P \rightarrow Q \& Q \rightarrow P$ and which we denote with $\iff$. It asserts that **P** and **Q** have identical truth values, or if they’re statements, they have identical truth tables.

8.2 Satisfiability and Decidability

Now we must discuss **tautology** and **contradiction**. A tautologous proposition is one that evaluates true regardless of variable assignments, like $\mathbf{P} \vee \mathbf{Q} \vee (\neg \mathbf{P} \& \neg \mathbf{Q})$. A contradiction is a proposition that is always false, such as $\mathbf{P} \& \mathbf{Q} \& \neg \mathbf{P}$. No matter what either variable is, the sentence never becomes true. Finally we have sentences such as our first friend $\mathbf{P} \vee \mathbf{Q}$, which clearly is true for 3/4 possible assignments of $\mathbf{P}$ and $\mathbf{Q}$ but not for all. It is not a tautology, but not a contradiction, and thus is it **satisfiable**. To test the satisfiability of any sentence in propositional logic, one simply has to work through a truth table and see if any assignments come out true. In propositional logic, *every single sentence* is **decidable**. We can absolutely always tell which of the three categories a sentence belongs to (tautologies are also, of course, considered satisfiable) because these sentences effectively act as simple circuits. There is always a procedure to **decide** the satisfiability of these sentences. They make finite claims about simple truths and falsehoods. While propositions can get arbitrarily large, doubling the time for a **decision** with every new variable, they are still always decidable in a finite amount of time.

This begs a brief comment on the **P vs NP** debate in computational complexity theory: **can we solve any arbitrary proposition for its satisfiability, in the worst case, in less than 2^n steps?**, where n is the amount of propositional variables in the sentence. In our truth table approach, we always take this many steps, but if we wanted to program a computer to do it with a process whose number of steps did not *double* with each new input, expanding exponentially in complexity as the problem incrementally ‘grows’, we would need to imagine a new intelligent way to do so. We have not yet. The doubling of this problem puts it in the **exponential complexity** category, whereas a problem like finding the highest number in a list is polynomially complex, because the amount of steps to solve it just increases linearly with the input size, not exponentially. **P** refers to such **polynomial-time** problems, where **NP** denotes a special class of **non-deterministic-polynomial-time** problems that can actually all be reduced into a problem of deciding a proposition with logic. Many famous problems have been standardized and reduced into a common **NP** form, with the reduction itself being done in polynomial time. This means that if we could solve it for *one* in **P-time**, we could do it for *all*. That is the extent of our discussion on the issue. If propositional logic was the extent of what **Hilbert** wanted to decide, Turing would not be writing this paper, because regardless of the speed, we know these problems can infallibly be solved in a finite process. You may be thinking that, ideally, we could formalize some of these logical symbols with some basic rules about how one tautology can transform into another, like in the tautology example, simply adding “**VR**” would indeed expand the statement while keeping it tautologous. If this process could be mechanized, it would be exciting to see a Turing machine churn out tautologies. Could one?

8.3 Predicate Logic

On top of the propositional logic, which deals with explicit propositions about boolean variables and connectives, we can build **Predicate Logic** or **First Order Logic**, which deals not with specific statements but generalized predicates, which act more as boolean **functions**. Whereas we had the propositions like “**James is older than Will**”, which is either true or false as is, if we generalize this sentence and separate the predicate, “**is older than Will**”, from the subject, we can form a boolean function, **OlderThanWill**, that takes in arguments that stand in as subjects. The original proposition can be reconstructed by expressing “**OlderThanWill(James)**” which will share a truth table with the original proposition. However now we could assert, of anyone in the domain of arguments, whether or not they are older than Will by applying this function to them. We could also have predicates with two arguments, such as **AreBrothers(x,y)**, where x and y are people in this domain. If James and Will are brothers, then **AreBrothers(James, Will)** would evaluate to true, but **AreBrothers(LeBron, James)** would not be. (Can you imagine a predicate that would make these latter two arguments true though?) What is the truth value of **AreBrothers(x, James)**? It may not be immediately clear, but this is asserting that x is a brother of **James**, which is now true or false depending on x . This choice of predicate is actually fascinating because if it is true, it must also be true with the arguments swapped. If **AreBrothers(James, Will)**, then

$$\begin{aligned} \text{AreBrothers}(\text{James}, \text{Will}) &\rightarrow \text{AreBrothers}(\text{Will}, \text{James}) \\ &\& \\ \text{AreBrothers}(\text{Will}, \text{James}) &\rightarrow \text{AreBrothers}(\text{James}, \text{Will}) \end{aligned}$$

but that is a special case. Suppose the predicate **Likes(x,y)**, which we want to be true if x likes y . Now the same dual entailment is not always guaranteed. Note: the perception of the differences between these two predicates could surely be found out by exhausting both truth tables and seeing their equivalence, but you likely

came to the conclusion without it. Sensibly, these kinds of thoughts and assertions are fundamentally baked into our language and thus our thinking.

8.4 Semantics to Syntax, Quantors

Having gotten the flavor for predicates, we can introduce **quantors** or **quantifiers**. Suppose we wanted to brazenly assert that everyone in the world likes **Alan Turing**. The way we can express such a *universal* statement is with the **Universal Quantifier**, which we will denote as $\forall \mathbf{x}$, where $\mathbf{x}$ is the bound argument being quantified. We would say: “ $\forall \mathbf{x} : \text{Likes}(\mathbf{x}, \text{Alan Turing})$ ”. While it should be true, this statement is now back to a proposition that may or may not be true. While we could imagine plugging in $\mathbf{x}$ values to the predicate without the quantifier being right or wrong, the quantifier is making *one* universal assertion, which we can say is likely false. Many people like Alan Turing, however far too many did not. Can we express a *true* quantification of the people’s liking of Alan Turing that is more general? Can we say that *there are people* who like Alan Turing without having to list them all? Enter the **Existential Quantifier**, which we will denote as $\exists \mathbf{x}$. We could say: “ $\exists \mathbf{x} : \text{Likes}(\mathbf{x}, \text{Alan Turing})$ ”, which is provably true, by my own admission. The sentence reads as “**there exists an $\mathbf{x}$ where $\mathbf{x}$ likes Alan Turing**” and the first reads as “**for all $\mathbf{x}$, $\mathbf{x}$ likes Alan Turing**”. You may be correctly sensing another yin yang relationship between these quantifiers reminiscent of $\&$ and $\vee$. DeMorgan’s Law nicely fits in with these quantifiers as well, such that: $\neg \forall \mathbf{x} : \mathbf{P}(\mathbf{x}, \mathbf{y}) = \exists \mathbf{x} : \neg \mathbf{P}(\mathbf{x}, \mathbf{y})$ and $\forall \mathbf{x} : \neg \mathbf{P}(\mathbf{x}, \mathbf{y}) = \neg \exists \mathbf{x} : \mathbf{P}(\mathbf{x}, \mathbf{y})$. If it is not the case that all $\mathbf{x}$ satisfy the predicate with $\mathbf{y}$, then there must exist an $\mathbf{x}$ where $\neg \mathbf{P}(\mathbf{x}, \mathbf{y})$ is true. Makes sense!

All of these explorations into **Propositional** and **Predicate** logic have relied heavily on **semantics**, examples in English, and explanations. Now that we understand the tools at our disposal, however, we would do well to formalize these laws into a **syntactic** system, one that represents logical ideas without words, but instead symbols. In bringing a rule-based system into syntax, the common method has been the **axiomatic** approach, in which you define several statements that are taken as true without proof. They are assumptions. On top of them, we can define specific syntactic rules about how we may transform statements or substitute values (maybe other statements) into them. With a proper axiomatic foundation and proper rules, one can imagine a brute force generation of every single ‘valid’ statement in the system (called a **theorem**) by repeatedly applying rules to axioms, then applying rules again. By definition, if we apply rules to theorems (and axioms are counted as theorems), we will always produce a new theorem. Let us now propose axioms for propositional logic:

1. $P \vee P \rightarrow P$
2. $P \rightarrow P \vee Q$
3. $P \vee Q \rightarrow Q \vee P$
4. $(P \rightarrow Q) \rightarrow (R \vee P \rightarrow R \vee Q)$

If you are wondering why we have not included $\&$ in the axioms, it is because $\mathbf{P} \& \mathbf{Q}$ can actually be rewritten as $\neg(\neg \mathbf{P} \vee \neg \mathbf{Q})$. If these axioms seem trivial, that is because they are designed as such: they make so much sense to us that if they were not provable (they actually are not), then we’d think a system without them is plainly wrong. Axioms are theorems that are assumed true with no proof. For expanding to predicate logic, we add two axioms:

5. $\forall x : P(x) \rightarrow P(y)$
6. $P(y) \rightarrow \exists x : P(x)$

Also sensible, but necessary.

8.5 Rules of Inference

Now for the rules of manipulation and inference in Predicate Logic, we have two main ones:

Substitution

- Propositional variables can be replaced with formulas (avoiding variable clashes)
- Variables can be changed uniformly (avoiding clashes)
- Predicates may be replaced by formulas
- Note: $\forall \mathbf{x} : \mathbf{P}(\mathbf{x})$ is equivalent to $\forall \mathbf{y} : \mathbf{P}(\mathbf{y})$ as they use nameless variables

Implication

- If $\mathbf{P}$ is true and $\mathbf{P} \rightarrow \mathbf{Q}$ is true, then $\mathbf{Q}$ is true

The variable changing corresponds to $\forall \mathbf{x} : \mathbf{P}(\mathbf{x})$ effectively being the same as $\forall \mathbf{y} : \mathbf{P}(\mathbf{y})$ because they are both talking about nameless variables. However if we were to plug in a proposition containing $\mathbf{x}$ into a formula containing $\mathbf{x}$ bound somewhere else, then one of those two variables must be uniformly changed to avoid ambiguity. Let us derive an example of a new theorem, by demonstrating a transformation from axioms. Such a demonstration is called a **proof**, and they can only be performed on real theorems. If a formula is provable, then it can be reduced to axioms via rules of inference and it is a theorem.

8.6 Example Proof

1. Start with axiom (3): $\mathbf{P} \vee \mathbf{Q} \rightarrow \mathbf{Q} \vee \mathbf{P}$
2. Apply substitution: Replace $\mathbf{P}$ with $\mathbf{F}(\mathbf{x})$ and $\mathbf{Q}$ with $\mathbf{G}(\mathbf{x})$
3. Result: $\mathbf{F}(\mathbf{x}) \vee \mathbf{G}(\mathbf{x}) \rightarrow \mathbf{G}(\mathbf{x}) \vee \mathbf{F}(\mathbf{x})$

It is not an astonishing result, but it is irrefutably a tautology if $\mathbf{F}$ and $\mathbf{G}$ were indeed theorems already.

Now we are ready to introduce a syntactic definition of natural numbers to predicate logic by adding some axioms. **Guiseppe Peano**, in 1889, first formalized such axioms for number theory and they are still regarded as the best set today. His were written in language, not symbols, so we will follow his convention, soon to be represented symbolically.

8.7 Peano Axioms

1. Zero is a number
2. Every number has a successor that is also a number
3. Zero is not a successor to any number
4. Two numbers that are successor to the same number are the same number
5. If something is true for Zero, and the fact that it is true for one number implies that it is true for that number's successor, it is true for all numbers

The last one refers to a principle called **induction**, and it is powerful for many proofs, including Turing's. Now to transform into symbols, we must make some arbitrary choices and some necessary ones. We are used to the **Arabic base-10** notation for numbers, but we are trying to formalize into a purely syntactic space, built on simple axioms that do not profess anything other than what we interpret. Numbers are not defined on their own. There is no concept of **1** or **2** or **100** in this axiomatic system, only **0** and successorship. So to refer to **0** we will henceforth either use **0** or **u**. To indicate successorship, we will use either a leading **S** or a trailing *****. Our **1**, **2**, **3** will look like **S0**, **SS0**, **SSS0** or **0***, **0****, **0*****. Immediately, we now remember the earlier discussion on the computational basis behind all of our understanding of numbers, and that most of what we consider number theory can be now defined recursively building on successorship. We can define $\mathbf{F}(\mathbf{x}, \mathbf{y})$ to be true if and only if $\mathbf{y}$ is the successor of $\mathbf{x}$. We can define $\mathbf{N}(\mathbf{x})$ to be true if and only if $\mathbf{x}$ is either **0** or the successor of some number. It is important to remember that these are boolean predicates, returning true or false, and not actually calculating anything. With a successorship *predicate*, $\mathbf{F}$, and Peano Axiom **5**, we can define the axioms for addition with a new $\mathbf{Sum}(\mathbf{x}, \mathbf{y}, \mathbf{z})$ predicate that will be true if $\mathbf{z}$ is the sum of $\mathbf{x}$ and $\mathbf{y}$:

7. $\forall x : \mathbf{Sum}(\mathbf{x}, \mathbf{0}, \mathbf{x})$
8. $\forall x : \mathbf{Sum}(\mathbf{0}, \mathbf{x}, \mathbf{x})$
9. $\forall x : \forall y : \forall z : \forall r : \forall s : \mathbf{Sum}(\mathbf{x}, \mathbf{y}, \mathbf{z}) \ \& \ \mathbf{F}(\mathbf{y}, \mathbf{r}) \ \& \ \mathbf{F}(\mathbf{z}, \mathbf{s}) \rightarrow \mathbf{Sum}(\mathbf{x}, \mathbf{r}, \mathbf{s})$

We defined earlier how these predicates could be checked and how, not in this paradigm, they could be calculated as numeric functions as well, without ever discussing anything but **0** and successorship. The first of these axioms of addition concern adding with **0** (again, taking pains to establish unambiguously what happens in either scenario, no implications being made), and the last states that if $\mathbf{x} + \mathbf{y} = \mathbf{z}$ and $\mathbf{y} + \mathbf{1} = \mathbf{r}$ and $\mathbf{z} + \mathbf{1} = \mathbf{s}$, then $\mathbf{x} + \mathbf{r} = \mathbf{s}$. As we know, with addition, we can define multiplication, and thus **Prod**($\mathbf{x}, \mathbf{y}, \mathbf{z}$), by adding some new recursively defined axioms:

$$10. \forall x : \mathbf{Prod}(\mathbf{0}, \mathbf{x}, \mathbf{0})$$

$$11. \forall x : \mathbf{Prod}(\mathbf{x}, \mathbf{0}, \mathbf{0})$$

$$12. \forall x : \forall y : \forall z : \forall r : \forall s : \mathbf{Prod}(\mathbf{x}, \mathbf{y}, \mathbf{z}) \ \& \ \mathbf{F}(\mathbf{y}, \mathbf{r}) \ \& \ \mathbf{Sum}(\mathbf{z}, \mathbf{x}, \mathbf{s}) \rightarrow \mathbf{Prod}(\mathbf{z}, \mathbf{x}, \mathbf{s})$$

to be true if $\mathbf{x}$ times $\mathbf{y}$ equals $\mathbf{z}$. Again we cover the case for **0**, then assert that if $\mathbf{x} * \mathbf{y} = \mathbf{z}$ and $\mathbf{y} + \mathbf{1} = \mathbf{r}$ and $\mathbf{z} + \mathbf{x} = \mathbf{s}$, then $\mathbf{z} * \mathbf{x} = \mathbf{s}$. From here, we can quickly define many other number-theoretic predicates such as:

- **IsEven**($\mathbf{x}$) = $\exists y : \mathbf{Prod}(\mathbf{y}, \mathbf{0} **, \mathbf{x})$

- **IsPrime**($\mathbf{x}$) = $\neg(\exists y : \exists z : \mathbf{Prod}(\mathbf{y}, \mathbf{z}, \mathbf{x}) \ \& \ \neg(\mathbf{F}(\mathbf{y}, \mathbf{0} **) \vee \mathbf{F}(\mathbf{y}, \mathbf{x})) \ \& \ \neg(\mathbf{F}(\mathbf{z}, \mathbf{0} **) \vee \mathbf{F}(\mathbf{z}, \mathbf{x})))$

- **IsOdd**($\mathbf{x}$) = $\neg \mathbf{IsEven}(\mathbf{x})$

We must reiterate that while we have constructed this system to play nicely with our ideas of number theory and logic, that it is nothing more than a set of arbitrary symbols and arbitrary rules. We use these systems to rigidly test and explore the spaces we are trying to represent in the system. Our sanity and intelligence depends on many intuitions, assumptions, and indeed-logical deductions, but we formalize to strip out the intuition and determine how demonstrable certain human truths are in a restricted space of rules. If we can devise a system as this that can crank out tautology after tautology, eventually permeating the entire space of our understanding of number theory, and if we can express any true statement in number theory as a theorem of the system, then we have proven evidence of the consistent behavior of number theory. We'd have a reliable and complete system which defines numbers. While propositional logic is so fundamental, that if it were somehow incomplete or inconsistent we would question our own sanity, first order predicate logic opens the door to fundamentally ungovernable complexity.

8.8 Recursion

One final consideration needed in establishing first order logic as a system for number theory is that of **recursion and bounds** of what can be defined and computed. While infinity is an inexorable concept that arises with natural numbers, it can only formally exist in certain contexts in axiomatic number theory. While we can easily prove, with induction, that the set of natural numbers is infinite (it is even expressed in the axioms themselves), the steps required to show it are certainly not infinite. We do not need to say “**0 is a number**” and “**0* is a number**” and “**0** is a number**”, and on and on forever. Even without induction, axioms (1) and (5) clearly show that **0** is a number, and that every number has a successor that is also a number, and thus show that numbers go on infinitely. We can easily show logically that numbers go on forever in a limited number of steps. Similarly for a predicate such as **IsPrime**($\mathbf{x}$), while the quantifiers do this work for us in our first definition, if we wanted to test it mechanically and explicitly, we would have to assert a conjunction of every pair of numbers from **0**** through $\mathbf{x}$ not being the factors of $\mathbf{x}$. Thus the terms in this expanded verification predicate are bounded, limited by $\mathbf{x}$. A Turing Machine or algorithm or predicate that wants to verify whether $\mathbf{x}$ is prime would never ever get caught on a search for an answer that may never be there. If it checks through $\mathbf{x}$ and finds no factors above **0****, we would know that $\mathbf{x}$ is prime. Familiarly, this process of checking is recursive, in that it entails repeating a certain process repeatedly *on a condition*, then terminating if that condition is met or if the list of needed checks is exhausted. It is a recursive, repetitive algorithm that we know will eventually end. This kind of predicate is **Primitively Recursive**. For programmers, primitive recursion covers any ‘**loop**’ block of code that is **bounded**. In the context of predicate logic, I like to think of it as a predicate that is possible (if cumbersome) to be written **without any quantifiers**. A finite set of boolean &s and v's. We can expand **IsPrime**(**5**) to be:

$$\neg \mathbf{Prod}(\mathbf{x}, \mathbf{0} **, \mathbf{0} ****) \ \& \ \neg \mathbf{Prod}(\mathbf{x}, \mathbf{0} ***, \mathbf{0} ****) \ \& \ \neg \mathbf{Prod}(\mathbf{x}, \mathbf{0} ****, \mathbf{0} ****)$$

which explicitly shows that no number between **2** and **5** is a factor of **5** with some free variable $\mathbf{x}$. And trivially, the **Prods** can all be eventually broken down into successor functions which of course do not have quantifiers of any kind. When you think of primitive recursion, think limited predicates and bounded loops.

However, for a predicate like **IsDifferenceOfPrimes(x)**, which would return true if **x** is the *difference of any two prime numbers*, the situation is different. With quantifiers, we could easily write this predicate, but expanding that predicate without those quantifiers is impossible, in the general case. Since we know that one of the prime numbers needed to satisfy this predicate would need to be greater than **x** (because we're asserting that **x** is the outcome of a subtraction), there is no guarantee that we ever find such a prime. There could be an even number that is not such a difference of primes, in which case we would need an infinite set of terms denoting which numbers are not viable. This class of predicate is **Generally Recursive**, meaning it recurses, but has no guarantee of termination, no bound on number of steps. (Note that primitive recursion is a subset of general recursion.) This will be extremely important in our next discussion, because we'll define predicates that we need to ensure can be **always** defined and evaluated in first-order logic (i.e. ones that are primitive recursive).

You may immediately wonder how this fits in with Turing Machines, which are equivalent in computational power to first order predicate logic, but are circle-free. By definition, circle-free machines run forever, however there is a nuanced distinction to make. For Turing's conventions and demonstrations, he focused on machines that were circle-free, but in no way does that mean that the **SDs** themselves ever need to be infinite in length. Nor does it mean that, in a formalized representation of Turing Machines into predicate logic, there would ever need to be an infinite conjunction or disjunction of terms. We can describe **infinite** processes only if we can **define them finitely**. More on this later, but it is crucial to grasp what primitive recursion refers to and not to get it confused with circularity of Turing Machines.

9 Enter Gödel

Having established the systems of propositional logic and first order predicate logic, we are now ready for Gödel. Though he proved that *propositional* logic is indeed complete and consistent in 1930, when he tried to do the same for *first order* logic in 1931, he in fact proved that no such system (comprehensive enough to define arithmetic) could be both complete and consistent. If you think Turing's methods are alien, inefficient, opaque and just inconceivable, wait until you see the work that inspired him. First some basic ground rules. Gödel's method relies on a formal system being powerful enough to be able to define natural numbers, successorship, addition, and multiplication. First order logic certainly can fit this bill. His method of proof fully relies on the ability to do arithmetic, specifically addition and multiplication. Also, *we* will be using Arabic numerals as a shorthand to express numbers in Gödel's scheme and in **PM** (the formal system of his choice, defined momentarily), but we must remember that if ever they are being used or implemented within the system, they follow the symbolic conventions. When I write **27** in commentary, I am denoting **0***** (27 *s)**.

9.1 Intuition Behind Gödel's Proof

Inspired by the diagonalization work of Georg Cantor and some of the work by Bertrand Russell which inspired PM, Gödel attempted to find a contradiction to PM being consistent and complete, one example where things broke down. This is a common practice in mathematical proofs that attempt to answer difficult questions of what is possible: don't try to find universal evidence of something *always remaining consistent*, but instead try to conjure an example which specifically, intentionally nullifies consistency. This is known as *proof by contradiction* or *reductio ad absurdum*. In the world of formal systems attempting to define large spaces of truth, this approach is much easier than conversely trying to investigate all of the areas in which the system *does* hold up.

The specific *kind* of contradiction Gödel sought was one involving self-reference. If statements in PM could (somehow!) make statements about statements, then all kinds of weirdness was likely to follow. Before formalizing PM, Russell coined a paradox in an original set-theoretic attempt to codify natural numbers via logic. The crux of the contradiction can be summed up with one tricky English sentence and even more richly with a riddle. The sentence goes like this: *this sentence is false*. The riddle goes like this:

*Imagine a village where there is but one barber. The barber **only** cuts the hair of villagers who do **not** cut their own hair. Simple enough; some industrious villagers cut their own hair, obviating his service, and other depend on him... Well, then, who cuts the barber's hair?*

Under the definition of the scenario, he only cuts the hair of those who do not cut themselves. So if the barber cuts his own hair, then...the barber wouldn't cut his hair. Well, if he opts *not* to cut his own hair, then...the barber *must* cut his hair.

Similarly in our sanitized sentence, if we take it to be true, then immediately we obediently accept that it is false (that is what it, truthfully, says!). This feels nonsensical. So we assume the sentence itself is false; we disobey what it says (*that it is false*), but then that only serves as proof that it was correct all along!

It is a fundamentally unworkable sentence; it cannot be understood coherently as true or false. If, then, Gödel could somehow express this statement in **PM**, then he would be throwing the system a problem it just could not solve coherently. Why?

First, strictly within **PM** there is no notion of truth and falsehood, only theorems and non-theorems. The idea, again, is that all true statements of number theory can be mapped to derivable theorems, and that all theorems represent strictly true statements of number theory. So the **PM**-encoded version of the sentence—we can call it G —would actually be more like: *this statement (G) is not a derivable theorem in **PM***. Assuming you formulate this self-referential, self-denying statement in **PM**, then would it be a theorem (loosely, true) or not (loosely, false)?

Since there is no obvious answer, but only two options, we can just assume one, see if it holds up, and, if it doesn't, try the other:

$G \text{ is a theorem} \rightarrow \text{we cannot derive } G \text{ (because this is what } G \text{ says)}$

Hmm...

$G \text{ is not a theorem} \rightarrow \text{we can derive } G \text{ (because this is what not-} G \text{ says)}$

Accepting G as true invites an immediate and obvious contradiction. If G was indeed derivable, then we have a contradiction: G explicitly states that it is *not* derivable. If, alternatively, it is *not* derivable, then its **truth** would be obvious, but unrepresented, unable to be derived in **PM**—the definition of incompleteness!

So **IF** we can actually formulate this in **PM**, it would spell doom for its consistency-and-completeness. That is a big if...

How could one possibly construct this logical quagmire of a scenario to a strictly defined skeleton of logic and its expanded application to number theory? Because this contradictory statement relies on self-reference, you need a way to have statements in **PM** talk not just about numbers, but about statements of **PM** themselves.

Using alien-like imagination, Gödel discovered how to do this by mapping *the entire syntax and semantics* of **PM** into the world of natural numbers—the very domain **PM** was constructed to rigorously define. Armed with this parallel version of **PM** as a subset of the natural numbers, then making **PM** statements *about numeralized **PM** statements* would amount, from our perspective, to **PM** talking about itself. **PM** talks about numbers. If you could actually map **PM** statements to numbers, then you could have **PM** statements talk about **PM** statements! This existentially depends on a faithful mapping.

9.2 The Mapping We Want, Visually

The diagram below illustrates the kind of mapping we want in order to carry out Gödel's scheme. Not only is it a cool idea in its own right to be able to encode **PM** as numbers (*as defined by **PM**!*), but it will allow us to use **PM** to talk about **PM** statements and their properties. How?

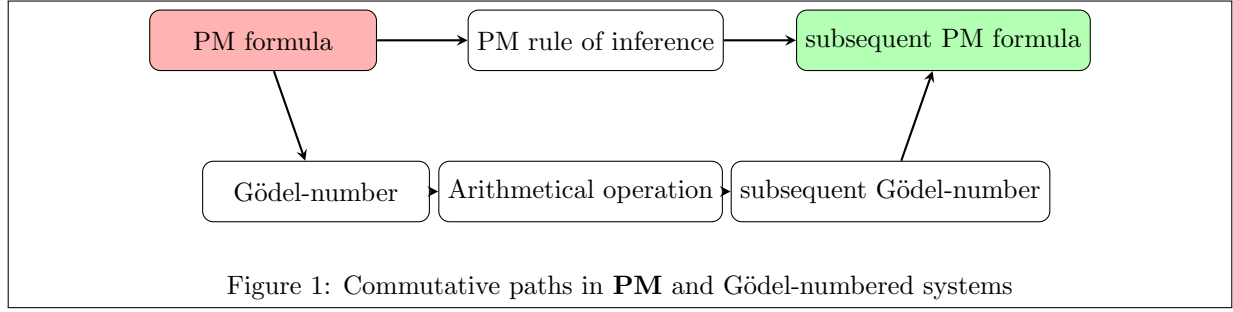
9.2.1 What Is Mapped, Where

In the diagram below, the top route from the first formula to the second is straightforward and entirely contained in normal **PM**. The red formula could be making a statement about, say, the number 9 and its factors. Then with a rule of inference applied (or many) the green formula could emerge as an implied statement about 8 or 10, or it could be taking the original formula and generalizing it in some way. The point is that the formulae on the top row are **PM** statements about natural numbers—business as usual.

The second row, the detour from the red to green formulae, consists exclusively of numbers and numerical functions. The red formula is encoded into a (huge) **PM**-defined number according to an encoding process we also define in **PM**. While it is now just a huge integer, it represents (to us, the wise *employer* of **PM**) the same original statement about 9 and its factors. We could easily map to and from this number and the original formula, so we can say that they are *saying the same thing*. The arithmetical operation is a **PM**-defined function (like multiplication or exponentiation) to be executed upon the first number and it produces a third number. This third number (also just a huge integer) can be decoded (by the same kind of **PM**-defined process) back into the green **PM** formula, which was making some statement inferred by the original one about 9.

9.2.2 The Magic Afforded

Not only can we draw fascinating insights about just what it means that a formal system's defined world contains the axioms and mechanics of the formal system itself. We can operate on this affirmation of soundness.



The top row, again, is business as usual: *symbolic logic talking about numbers*. The bottom row are also—in effect—other symbols saying the same things about the same numbers. The crux comes from the fact that the symbols speaking-about-numbers on the bottom row *are numbers about which the top row can speak!* **PM** formulae of *any* kind are always talking about integers. But since we can represent these formulae as integers, we can use **PM** formulae to talk about integers-which-represent formulae! So if we mind the mapping and remain consistent and prudent, we can use a **PM** formula to talk about the properties of other **PM** formulae. And eventually...eventually, we can have **PM** formulae talking about *their own* properties!

9.3 How to Gödel-number

Without laying out the entire vocabulary of **PM**, we will use a handful of small, idealized examples to illustrate the mechanism and intuition of Gödel-numbering.

Gödel-numbering starts with assigning each individual symbol/letter of **PM** a distinct integer. The assignment of these numbers is completely arbitrary, however it is imperative that each symbol has a distinct integer. (As with all of the numerical parameters in both Gödel-numbering and LLMs, they are arbitrary in their initial assignment. The *specific* numbers do not need to map in any way to semantic rules or facts or other considerations, however, *crucially*, once these parameters are arbitrarily chosen (in Gödel-numbering) or derived from pure random (in LLMs), they must commit to their role, contributing vitally specific information.)

To turn a formula of **PM** into its Gödel-number, we parse it symbol by symbol to form a list of elementary symbols. Then we construct a product with **successive prime bases** raised to the power of the **assigned integers for each symbol**. The product takes the following form:

$$2^a * 3^b * 5^c * 7^d * \dots * \text{Nth-prime}^n$$

where **a** is the **1st** symbol's assigned integer, **b** is the **2nd's**, and on and on

This process constructs a distinct (and massive) integer for every single formula of **PM**. Not even two statements that are equivalent logically will have the same Gödel-number if they're written differently, because the order of the formula is incorporated into the formulation of the number.

As a bit of proof, as well as just an interesting nugget of an example, we can construct the Gödel-number of a specific **PM** formula called **Z**. **Z**, defined as " $\neg(0 = S0)$ ", equates semantically to: *it is not the case that 0 equals the successor of 0*, i.e.: *0 is not the number that follows 0*. Simple enough. Below are the arbitrary symbol assignments:

- "¬" is 1
- "(" is 8
- "0" is 6
- ")" is 9
- "=" is 5
- "S" is 7

- "→" is 4 (we may use this later)

When we Gödelize this formula under my arbitrary assignment of symbols, we get the gargantuan integer: **38437883119901964441147894774941906250**.

$$\text{Gödelize}(\mathbf{Z}) = 2^{\textcolor{red}{1}} * 3^{\textcolor{orange}{8}} * 5^{\textcolor{green}{6}} * 7^{\textcolor{blue}{5}} * 11^{\textcolor{cyan}{7}} * 13^{\textcolor{green}{6}} * 17^{\textcolor{yellow}{9}} = \mathbf{38437883119901964441147894774941906250}$$

The exponents of each term come directly from our assignment of symbols, and the bases just step up through the primes 2, 3, 5, 7, 9, 11, 13, 17, 19, 23, 29, etc. Why primes?...

Well what does this cumbersome scheme afford us? Let's explore what we just did above in our example. The number **38437883119901964441147894774941906250** analyzed via **PM** will certainly correspond to true for **IsEven**, as it's an even number, it most certainly will be false for **IsPrime**, again because it is even *and we just showed exactly how to create it with multiplication*, and it will have many other truth values for many other predicates, just like all numbers. This number is not special on its own, however, now that we see that it was the product of a very special construction, we can think of this number on a more meta level. To us, it has the hidden meaning of the **PM** formula $\neg(\mathbf{0} = \mathbf{S0})$. We have projected a predicate of **PM** into **PM** itself, but in the form of an integer.

This scheme also, importantly, affords us the ability to easily decode this vast integer back into its original **PM** format or, relatedly, to search for any particular symbol or even string of symbols in the formula. Decoding is so easy because we have a very specific algorithm by which we embed these formulae in integers, with even the order preserved. To decode the whole string, starting with the first prime number, **2**, we find the *degree* to which it is a factor of our number. We simply count how many times **2** can be divided in whole from **Gödelize(Z)**. Put a third way, we find the highest **n** such that: $\text{Gödelize}(\mathbf{Z}) \bmod 2^n = 0$. In our example, it turns out that the highest power to which we can take **2** is **1**, telling us that the first symbol is **→**. To get the subsequent symbols in order, we just move to the next prime number. Eventually we will try a prime number who will not even divide evenly even once, and we will know the formula has been fully decoded.

Taking the other route—looking for a specific symbol anywhere in the string, not simply decoding the string—we can just look among the prime bases that comprise the Gödel-number (again, we know the limit of the primes involved: the highest prime is the last one who goes in evenly at least once) and see which ones can be taken to the power of our target symbol and divide evenly. In our example Gödel-number, if we wanted to look for the symbol "=", which is assigned the integer **5**, we test each **prime** up to **17** to the **fifth** power and see if $\text{Gödelize}(\mathbf{Z}) \bmod \text{prime}^5 = 0$. To look for, say, the sequence "**S0**" (who have integers **7** and **6**, respectively) in our Gödel-number, we can do a similarly straightforward and limited test. For all pairs of primes, **p_i**, **p_j** in our bounded range (**p_i** would be **2-13** and **p_j** would be **3-17**), is there a pair such that: $\text{Gödelize}(\mathbf{Z}) \bmod (\text{p}_i^7 * \text{p}_j^6) = 0$? In our example, yes, **p_i = 11** and **p_j = 13** satisfy this, telling us not only that "**S0**" is present, but also where it is in our string. This will be useful!¹

Well we've mapped one formula (and we now know how we'd map others), but it is still completely unobvious that we have actually mapped *the entire syntax and semantics* of **PM** into integers. Just because we have translated **Z** into a number, how do we know this number truly *represents Z*? Also, even if I could formulate statement *G* into numbers (which we don't yet know how to do), how would we know that it *actually means* what we want it to mean? Saying that **38437883119901964441147894774941906250** means *zero doesn't come after zero* seems like a cheap encryption trick.

¹A note on the importance of prime numbers to this hashing scheme is appropriate here. The **fundamental theorem of arithmetic**—that every number can be broken down into prime factors—allows us to look at every single natural integer as a unique combination of distinct elements, prime numbers. Every single integer is thus a unique *weighted product* of these elementarily distinct objects and can be identified and defined as such. For those with experience in linear algebra, you may recognize prime factors as distinct dimensions in linear space! Every single point (integer) in this prime space is defined by a distinct multiplicative combination of the unique elements. Conversely, certain limited combinations of prime numbers have entire regions of number space that are entirely unreachable! These facts are the reason why Gödel-numbering can work: when you think of numbers as composed of prime numbers to different powers, then they can be easily mapped 1:1 from their composite form to their elementary building blocks. When we decode Gödel-numbers, we are simply finding the scalar in each dimension of prime space. We enjoy that a single prime can only possibly have one scalar (exponent, degree, weight) in the composite integer. Further, when you commit to the convention of building these Gödel-numbers with these prime elements *in order*, then testing these formulae as numbers becomes ever more defined (for instance, you always have an upper bound on the highest prime base).

9.4 Numbering Rules of Inference

With all formulae and symbols of **PM** successfully projected back into the world of natural integers, we have implanted individual nodes—**PM** formulae—into a parallel numerical space. At this stage, it is still meaningless. If we instead assigned every English letter (plus punctuation, numerals, etc.) an integer and constructed Gödel-numbers for each word or sentence, would we feel close to constructing a language model? Most likely not! While the assignment of static strings (both in **PM** or English) to numbers is a vital prerequisite for an integrated numerical mapping, it is clear that *the way these numbers work together* is the more important part. Human language is not a function of its static, specific elements—there are *thousands* of natural human languages, all with different letters and sounds—it is about how these elements relate to each other semantically.

As we know, formal systems are entirely defined by the provision of axioms—strings that are taken to be true as seeds—and rules of inference—explicitly defined rules about how axioms may grow or contract or combine under specific conditions. We define elements and an environment in which they abidingly exist. In a system like **PM**, these rules allow us to create families of theorems which make statements about numbers. The theorems don't create numbers themselves, but they just talk about them. And the rules don't create numbers either, they just create theorems which talk about them. This is vital to keep in mind.

Remembering our ultimate goal, we don't want to just talk about numbers and arithmetic which may or may not define them, but we want to talk about *theorems* (as numbers!) and the **PM** rules of inference which may or may not prove/derive them. But if we want the theorems to be numbers, then we need the rules of inference themselves to be represented as arithmetical operations *on* those numbers. Ultimately we want to make statements in **PM** about the viability of other **PM** statements and their provability. So we are going to be making statements like: *this number x (which represents a **PM** formula) is provable by using rule a on the number y (also a formula!)*. Vital to knowing at all how such statements could be made and verified, we should explore what an Gödelized rule of inference would look like *in action*; that is, actively manipulating a Gödel-number of a **PM** theorem in order to mint another.

In logic there is the concept of **Modus Ponens**. It means something like *setting the mood* in Latin, but in logic it refers to this simple and vital-to-human-survival-for-millennia relationship: if **A is true** and **A implies B is true**, then **B is true**. In another context, if you know that **having a ticket earns you a seat on a plane ride** and **you have a ticket** for the next flight home, then **you have a seat waiting for you on that plane**! In symbolic logic, if we have two true statements **P** and **P → Q**, then we know we have a third, **Q**!

Let's imagine a rule of inference (which does indeed exist) in **PM** for Modus Ponens. We can define it like a function that takes in two theorems—something like **ModusPonens(x, y)**. In **PM**'s definition, it would look a lot like the above, but formatted more strictly. It would just say that if you have these two kinds of theorems, then you can derive the third.

How would **ModusPonens(x, y)** look Gödel-numerically?

$$\text{PM}_x : (0 = 0)$$

$$\text{PM}_y : (0 = 0) \rightarrow \neg(S0 = 0)$$

We know how to Gödel-number each of these. Let's use our old Gödel-numbering assignment:

"¬" is 1, "(" is 8, "0" is 6, ")" is 9, "=" is 5, "S" is 7, "→" is 4

to produce:

$$x = 2^8 * 3^6 * 5^5 * 7^6 * 11^9$$

or, beautifully

$$x = 2^{(} * 3^0 * 5^{=} * 7^0 * 11^{)}$$

and

$$y = 2^8 * 3^6 * 5^5 * 7^6 * 11^9 * 13^4 * 17^1 * 19^8 * 23^7 * 29^6 * 31^5 * 37^6 * 41^9$$

or, beautifully

$$y = 2^{(} * 3^0 * 5^{=} * 7^0 * 11^{)} * 13^{\rightarrow} * 17^{\neg} * 19^{(} * 23^S * 29^0 * 31^{=} * 37^0 * 41^{)}$$

The formula, $\mathbf{z}$, we want to create from this rule is defined as:

$$\mathbf{PM}_{\mathbf{z}} = \neg(\mathbf{S0} = 0)$$

And its Gödel-number is:

$$\mathbf{z} = 2^{\mathbf{1}} * 3^{\mathbf{8}} * 5^{\mathbf{7}} * 7^{\mathbf{6}} * 11^{\mathbf{5}} * 13^{\mathbf{6}} * 17^{\mathbf{9}}$$

or, beautifully

$$\mathbf{z} = 2^{\neg} * 3^{(} * 5^S * 7^0 * 11^{=} * 13^0 * 17^{)}$$

Can you spot something difficult in the way we obtained $\mathbf{z}$ from $\mathbf{y}$? Compare $\mathbf{z}$ to the end of $\mathbf{y}$. While my pattern-highlighting coloring and the exponents may mask the trouble, the statement at the end of $\mathbf{PM}_{\mathbf{y}}$ (the $\mathbf{Q}$ being implied by $\mathbf{P}$) has been *re-based* such that the bases for its Gödel-number now start back at $\mathbf{2}$! Our desired $\mathbf{z}$ is currently embedded deep in $\mathbf{y}$, starting with $\mathbf{17}$ as its base, creating a *vastly* different number in absolute terms...though not in terms of key shared exponents! This is an early hint to how complicated our numbering scheme makes dealing with numbers as formulae.

Now that we have defined the inputs and outputs as numbers, how can we go about constructing the actual arithmetical process which defines **ModusPonens**($\mathbf{x}$, $\mathbf{y}$)? First, what must **ModusPonens** do first to verify that $\mathbf{x}$ and $\mathbf{y}$ share the relationship they need to (namely that $\mathbf{x}$ is represented to the left of the implication $\rightarrow$ in $\mathbf{y}$)? Once we know $\mathbf{x}$ and $\mathbf{y}$ are valid for Modus Ponens, then how do we actually extract the implied result (given this re-basing complication)? Could this all possibly be defined in **PM**'s arithmetical world?

Concrete procedure for **ModusPonens**($\mathbf{x}$, $\mathbf{y}$):

1. verify $\mathbf{x}$ and $\mathbf{y}$ are well formed Gödel-numbers
2. verify that $\mathbf{x}$ and $\mathbf{y}$ are well formed for Modus Ponens
 - (a) find the $\rightarrow$ in $\mathbf{y}$ (and save its index)
 - (b) verify that what comes before it = $\mathbf{x}$
3. extract implied formula from within $\mathbf{y}$
 - (a) re-base it
 - (b) verify that it is well formed

The first step (1) is one that we will dismiss in the most shallow way because of its simple but tedious nature. There is a vast but finite set of checks one needs to make on arbitrary integers to make sure that they are well formed Gödel-numbers. These include checks like: checking that all powers are mapped to **PM** symbols, that parentheses are matched, and lots of other *syntactic* checks. While they may seem unwieldy upon reading them, these are checks that can be made entirely with arithmetic and **bounded loops**, now that we know how to find symbols and their indices from a Gödel number. That is, for any one of these checks, you know you just must assert and check a finite (if large) amount of things. For the first check I named, you'd have to just decode the Gödel-numbers as demonstrated above and make sure that each was within the range of how many symbols you've mapped (if **PM** has, say, 25 symbols, then you want to make sure that no prime has a power greater than 25).

Next (2) we have to check that $\mathbf{x}$ and $\mathbf{y}$ are compatible for Modus Ponens inference. To find the $\rightarrow$ in the statement (2a), use the method we demonstrated above. You can then calculate the prime number index of that prime number (for example 2 is the *first* prime number, 3 *second*, 5 *third*, etc.) to know at which spot in the formula the $\rightarrow$ is. To extract $\mathbf{x}$ from $\mathbf{y}$ (2b) is easy because no re-basing is required. You simply do the normal Gödel-decoding process, starting at 2 and finding its appropriate power, until you reach the prime number you found for the $\rightarrow$'s base. If all of the exponents you decoded from $\mathbf{y}$ up to this point equal those of $\mathbf{x}$ (or simply if you re-Gödelize the terms into a number, does it equal $\mathbf{x}$) then you know you are in business.

Assuming these entirely **PM**-definable checks have passed, then you can actually extract the implied $\mathbf{z}$ from $\mathbf{y}$ (3). Using some of our previous work (we know the prime number where the $\rightarrow$ is; we know our target comes right after it, we have already de-Gödelized up to it), then you can continue to de-Gödelize the number after the point of the $\rightarrow$ and save those exponents you get. Now to re-base (3a), you simply create a new Gödel-number starting with base $\mathbf{2}$ from those exponents. You can quickly then verify that it is well formed (3b), and you have your result $\mathbf{z}$!

Modus Ponens is a fantastic example to articulate the principle that you can indeed represent rules of inference on Gödel-numbers as **PM** functions (as they act on regular numbers) because your answer is embedded in one of the inputs. We merely had to uncover it, with a special hard-coded focus on that $\rightarrow$. While it may seem like a cop-out example, rules of inference in any formal system will inevitably mostly be augmenting, trimming, or combining inputs to form an output. That is, there will never be entirely discontinuous steps from one formula to an entirely different other formula (if the formal system in question is sensible and useful, not completely chaotic). **PM**-functions of inference will all also have special hard-coded biases that define what the function is for anyway; i.e. the reason **ModusPonens**(**x**, **y**) had **4** hard-coded was because the $\rightarrow$ symbol is an integral element to Modus Ponens, the relationship we are defining!

One more example of a Gödelized rule of inference—this time one which acts on one formula, not two—could hardly be a waste of time and will hopefully further flesh out the realizability of an arithmetical implementation. Another key rule of inference in logic that can act on any well-formed string—not even necessarily a theorem—is known as the **Law of the Excluded Middle**. It represents a logical relationship which each of us humans and the billions of instances of LLMs respect (and rely on) many times per day, namely that any specific statement can only ever at one instant be true or false. Take the statement: *grass is made of plastic*. Without even caring about the truth value of this statement, we know it is true to say that: *grass is made of plastic* **or** *grass is not made of plastic*, if *grass* and *plastic* and *made of* are well-defined concepts. It is always the case that some objective judgment is either true or false. Thus in **PM**, you may take any well-formed (legal) formula **P** and create the *theorem*: **P** $\vee$ $\neg$ **P**. You may notice that this rule can be applied repeatedly *ad infinitum*—creating, in Gödel-number space, an infinite family of closely related, rapidly growing integers branching from every possible Gödel-number.

Let's define **LawExcludedMiddle**(**x**), where **x** is a Gödel-number. Skipping the well-formedness and other trivial, clerical tests, what must we do in this numerical function?

Concrete procedure for **LawExcludedMiddle**(**x**):

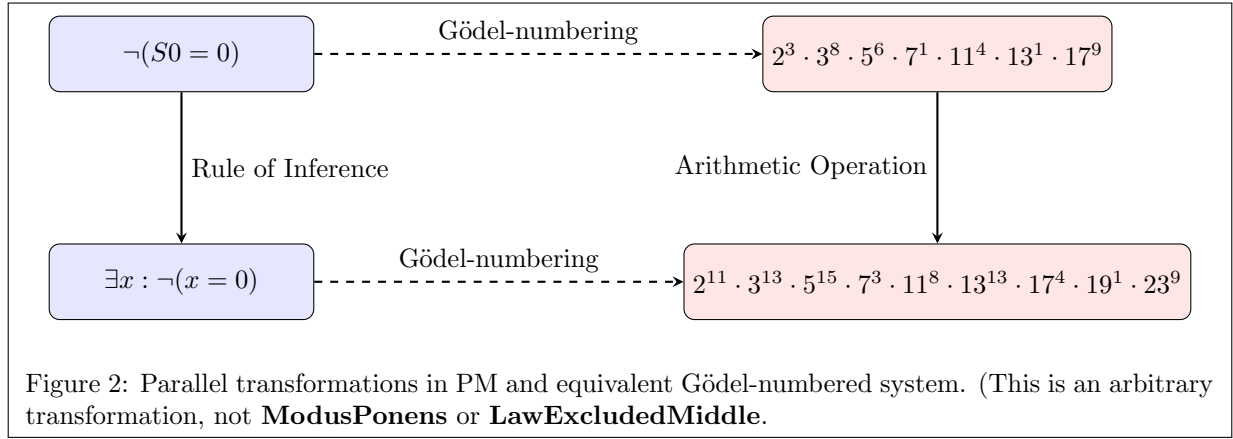
1. determine highest prime in **x**
 - (a) while we're at it, save all of **x**'s exponents for future use
2. starting after highest prime, combine the following elements:
 - (a) $\vee \neg$ (
 - (b) **x** (re-based to the right)
 - (c))

into the number passed in as **x**

Provided we can define and save these intermediary numbers and collections of numbers such as primes and bases, this function amounts mostly to an augmentation with standard Gödel-numbering. Supposing that **x**'s highest prime base was **31**, step **2** would consist of first multiplying **x** by **37** ^{$\vee$} * **41** ^{$\neg$} * **43**⁽. Re-basing **x** amounts to just starting to pull exponents from the list we defined as the original exponents of **x** at **47**. On whichever prime number it ends, you'd cap off with the next prime number to the power of)'s Gödel-number and you're done!

Every Gödelized **PM** rule of inference will have an ornate and cumbersome process. Even our simple Modus Ponens example was simplified. However, in principle, any manipulation of symbols in **PM** can be represented as a convoluted mathematical operation on Gödel-numbers because of the shared mechanical natures of symbolic logic and natural numbers. (Is any system defined by symbolic axioms and rules able to encode itself in the domain it defines?) The scheme by which to map individual formulae to individual integers seems extensive on its own, but seeing how helpful it is for actually *using* those integers like they're formulae sheds light on why the meticulous procedure for encoding was necessary. As we demonstrated, you can—the intelligent human being defining your Gödelized **PM** rules of inference *in PM*, who knows why you're hard-coding the way you are!—effectively work with Gödel-numbers as formulae once you do some simple exponent-decoding, all of which can be defined with simple **PM** bounded loops. Bounded bouts of counting and comparing. You may have many loops, many hard-coded values, many **OR** and **AND** statements you need to join, but you will always have a finite process.

One final important note to highlight about our arithmetization of **PM** strings and rules is the ease with which you can verify whether or not any given Gödel-number was the product of a rule of inference applied to some other Gödel-number. Soon this will be vitally important. If, for example, you wanted to check whether Gödel-number **r** came from applying **ModusPonens** to Gödel-numbers **p** and **q**, you simply check if **ModusPonens**(**p**, **q**) = **r** (Gödel numbers both)! You could even define a predicate function—evaluating to either true or false—that merely



arithmetically tests this claim: $\mathbf{MP}(\mathbf{p}, \mathbf{q}, \mathbf{r}) : \mathbf{ModusPonens}(\mathbf{p}, \mathbf{q}) = \mathbf{r}$. The same could easily be done for a $\mathbf{LEM}(\mathbf{x}, \mathbf{y}) : \mathbf{LawExcludedMiddle}(\mathbf{x}) = \mathbf{y}$.

With **PM** projected into itself such that we can arithmetically test for certain rules of inference operating on Gödel-numbers, we are now closer to Gödel's results. Recalling that we ultimately want to express '*this formula is not provable in PM*', maybe we can start to strategize on how we could say anything remotely like this. We have just defined $\mathbf{MP}(\mathbf{p}, \mathbf{q}, \mathbf{r})$ which evaluates whether or not **ModusPonens**(**p**, **q**) makes **r**. We could similarly imagine saying " $\neg \exists \mathbf{q} : \mathbf{ModusPonens}(\mathbf{p}, \mathbf{q}) = \mathbf{r}$ " which means, much more globally, that no arbitrary formula **p** could be paired with **q** and fed to **ModusPonens** to produce any arbitrary proper output formula **r**. It is saying, essentially, that **q** is not a valid candidate as a second argument to **ModusPonens**. If there is no prime $\mathbf{p}_i$ such that $\mathbf{q} \bmod \mathbf{p}^4 = 0$ then this would indeed be true! This last **PM** statement is not *just* talking about numbers (**p** and **q** and **r**) and some complex arithmetic relationship (though it certainly is doing *at least* that), it is also talking about **PM**'s own rules (which are now represented as arithmetic!). It is saying something powerful and far-reaching: it is making a statement about limits of **PM**, at least with respect to this specific rule of inference. This statement is a start.

9.5 Necessary Definitions

We have explored general examples of how to do some necessary book-keeping with Gödel-numbers such as finding and comparing exponents/**PM**-symbols and we have applied these techniques to specific rules of inference. We explored how you could *implement* rules arithmetically and this set up quite naturally how you could *check* the application of rules of inference.² Ultimately, we are hoping to make statements about *general provability* of specific formulae; provability just refers to whether or not a formula can be produced by a direct succession of applications of rules of inference upon theorems, starting with an axiom. These predicates to check whether a given Gödel-number is the product of a rule of inference on another (or a group of) Gödel-number(s) are the fundamental building blocks to be able to construct statements of general provability. If we have a finite number of rules of inference and rule-predicates—which, by definition, we do—then somehow joining all of these predicates together can amount to a kind of proof-check...

Before the final self-referential leap, we have more predicates to discuss, first regarding inference rules for **substitution**. One vital operation in math, programming, and formal systems (oh and natural language) is substitution of values into variables that play defined roles in a context. **PM** formulae sometimes make statements about specific numbers, but other times they make statements about undefined, unqualified, and entirely arbitrary *free variables*. In Gödelized **PM**, we can imagine an operation—**Substitute**(**x**, **y**)—that uses the same exponent-finding and re-basing operations we have already explored that will take in a formula, **x**, with free variables, and then another term, **y**, (which may be a formula or just a number), and substitute **y** in for the free variables of **x**.

For example, we can substitute the number **3** (encoded as **SSS0** in **PM**) into the free variable of a **PM** statement like $\neg(\mathbf{0} = \mathbf{f})$ to produce $\neg(\mathbf{0} = \mathbf{SSS0})$, which would mean that: *it is not the case that zero is equal to three*. Just as with $\mathbf{MP}(\mathbf{x}, \mathbf{y}, \mathbf{z})$ and $\mathbf{LEM}(\mathbf{x}, \mathbf{y})$, we can define a predicate **SUB**(**x**, **y**, **z**) that will only be true if **Substitute**(**x**, **y**) = **z**. In the above example, we'd be taking in **x** : $\neg(\mathbf{0} = \mathbf{f})$, **y** : **SSS0**, and **z** : $\neg(\mathbf{0} = \mathbf{SSS0})$ and

²Henceforth we will stick with the naming convention for Gödelized rules of inference and the predicates with which we check them. Rules (to act on formulae) will be defined as **RuleOfInference**(**x**) = **y** and corresponding predicates (to check for the application of a rule) will be defined as **ROI**(**x**, **y**).

verifying—again, with tools we already know—whether $\mathbf{z}$ is the product of the syntactic substitution of $\mathbf{y}$ into $\mathbf{x}$'s free variable.

We are nearing the ability to talk about general provability in **PM**. In a certain small sense, with the **SUB**($\mathbf{x}, \mathbf{y}, \mathbf{z}$) we have proven that $\mathbf{z}$ follows from **Substitute**($\mathbf{x}, \mathbf{y}$). That is, we can verify that a *certain* formula was obtained by *certain* antecedents via a *certain* rule/transformation. Before we can evaluate whether a Gödel-formula like $\mathbf{z}$ is *provable at all*, we should formalize how to evaluate whether some specific formula can be derived from *any* previous formula by *any* means. To be clear, a general proof-check would evaluate whether $\mathbf{z}$ can be derived starting at axioms; first, we should know how to test whether $\mathbf{z}$ can be derived from just one previous theorem, say $\mathbf{z}_{-1}$. We can define a predicate **DEM** (for **demonstrate**) which will take in arguments ($\mathbf{z}_{-1}, \mathbf{z}$) and be true if and only if $\mathbf{z}$ is obtainable by applying any available rule of inference to $\mathbf{z}_{-1}$. But what about rules of inference that take in *multiple* terms to produce an output, like **ModusPonens** and **Substitute**? We can just define **DEM** predicates for every family of rules of inference, separated by how many (finite) previous formulae they require as inputs.

Starting with transformations that take in just one input, we'll define **DEM**₁($\mathbf{z}_{-1}, \mathbf{z}$):

$$\mathbf{LEM}(\mathbf{z}_{-1}, \mathbf{z}) \vee \dots \vee \mathbf{Nth1TermPredicateOfInference}(\mathbf{z}_{-1}, \mathbf{z})$$

For rules which take in two inputs, we'll define **DEM**₂($\mathbf{z}_{-1}, \mathbf{z}_{-2}, \mathbf{z}$):

$$\mathbf{SUB}(\mathbf{z}_{-1}, \mathbf{z}_{-2}, \mathbf{z}) \vee \mathbf{MP}(\mathbf{z}_{-1}, \mathbf{z}_{-2}, \mathbf{z}) \vee \dots \vee \mathbf{Nth2TermPredicateOfInference}(\mathbf{z}_{-1}, \mathbf{z}_{-2}, \mathbf{z})$$

This will be true if **SUB**($\mathbf{z}_{-1}, \mathbf{z}_{-2}, \mathbf{z}$) is true OR if **MP**($\mathbf{z}_{-1}, \mathbf{z}_{-2}, \mathbf{z}$) is true OR if any other 2-term predicates of inference is true. After writing **DEM** predicates for every family or rank of parameter-counts as will be necessary in **PM**, then we can put together a general **DEM** predicate (all families) for $\mathbf{z}$ that will conjoin all of these previously defined versions.

DEM($\mathbf{z}_{-1}, \dots, \mathbf{z}_{-n}, \mathbf{z}$), then, will look something like:

$$\mathbf{DEM}_1(\mathbf{z}_{-1}, \mathbf{z}) \vee \mathbf{DEM}_2(\mathbf{z}_{-1}, \mathbf{z}_{-2}, \mathbf{z}) \vee \dots \vee \mathbf{DEM}_n(\mathbf{z}_{-1}, \dots, \mathbf{z}_{-n}, \mathbf{z})$$

(But why does the general **DEM** need to take in all of those parameters? How could you be testing if $\mathbf{z}$ is demonstrable but not know what antecedent theorems with which you may be dealing?) **DEM** is true if any one of the inference rule relationships is true for $\mathbf{z}$ and any antecedent $\mathbf{z}_{-n}$ s we want to include in our test. A bunch of finite numerical checks joined by $\vee$ is indeed still a finite numerical check.

This established, we are ready to create **PROOF**($\mathbf{x}, \mathbf{z}$). The $\mathbf{x}$ that **PROOF** takes as an argument is actually a vast Gödel-number that represents many **PM** formulae in a row. This sequence of formulae is intended to be a derivation of $\mathbf{z}$ by way of an algorithm of applications of rules of inference to one or several starting *theorems*. These formulae which constitute a stepwise derivation of $\mathbf{z}$ are separated in the vast number/string $\mathbf{x}$ by periods.

For example, imagine a simple proof chain in **PM**:

$$\mathbf{x} = \neg(SS0 = 0) \cdot \neg(SS0 = 0) \rightarrow \exists x : \neg(x = 0) \cdot \exists x : \neg(x = 0) \cdot \exists x : \neg(x = 0) \vee \neg(\exists x : \neg(x = 0))$$

$$\mathbf{z} = \exists x : \neg(x = 0) \vee \neg(\exists x : \neg(x = 0))$$

For the ease of reference, we can break it up and identify each subsection of this massive string as such:

$$\mathbf{x} = \mathbf{A} \cdot \mathbf{B} \cdot \mathbf{C} \cdot \mathbf{D}$$

We can hold off on trying to fit the Gödel-number of this vast string on the page.³ Before grasping just how a **PM**-defined predicate could check its Gödel-number for derivation, let's make sure we grasp the semantics of the string. For our example, **A** and **B** are taken to be theorems (which we could easily show they are). They would actually need behind them long chains of derivations starting with axioms to ever be able to reach the **D** theorem in this exact way; as with examples before, the same techniques discussed in this limited tail end of a derivation are identical to those needed to check any longer string needed to actually support **A** and **B**. That said, **C** is

³Though it must be remembered that this is just a vast Gödel-number! We will work with it as a string of formulae, but in actuality, **PROOF** will be working with exponents, primes, re-bases to do the work we are so easily describing with English and with **PM** symbols.

obtained by applying **ModusPonens** to theorems **A** and **B**. From there, we can use **LawExcludedMiddle** on **C** to produce the final theorem, **D**.

Two theorems were needed to produce **C**, and one was needed for **D**.

What **PROOF**(**x**, **z**) must do is:

1. find the prime bases of the periods, capturing the count and locations and lengths of the strings
2. for every theorem **t**, test whether it is either an axiom (using **IsAxiom**(**t**)) or if **DEM**(**t**_{**t-n**}, ..., **t**) is true for all permutations of the theorems before **t**
 - recall that **DEM**(**t**_{**t-n**}, ..., **t**) itself checks **t**'s validity with all possible numbers of parameters
 - either the individual **DEM**_{**k**} or the larger **DEM**(**t**_{**t-n**}, ..., **t**) or **PROOF**(**x**, **z**) itself must handle cycling through (finite!) permutations of all of their inputs
3. check whether **y** is equal to the re-based last theorem

Again, assuming we have already worked through a much longer chain up to the one displayed above, we can imagine what **PROOF** does to verify **C** and **D** in this derivation. In the first "loop" of **PROOF**, **C** will be our **t**. **IsAxiom**(**C**) will be false, but **DEM**(..., **A**, **B**, **C**) will ultimately evaluate to true:

- **DEM**(..., **A**, **B**, **C**) first calls **DEM**₁ on both **A** and **B** individually
 - **DEM**₁(**A**, **C**) and **DEM**₁(**B**, **C**)
 - neither are true, so **DEM**₁ as a whole is false
- **DEM**₂(**A**, **B**, **C**) is then called and will evaluate to true!
 - the predicate within **DEM**₂(**A**, **B**, **C**) that makes it true is **MP**(**A**, **B**, **C**)

Next and finally, **t** will be **D** and the same process will be repeated, this time with the **DEM** call being exactly **DEM**(..., **A**, **B**, **C**, **D**). This predicate will also evaluate to true because eventually within this predicate, **DEM**₁(**C**, **D**) will be true because **LEM**(**C**, **D**) will be true.⁴ Finally, we just check—easily, numerically—that the re-based **D** is equal to **z** (which it is).

While it is cumbersome to imagine—and torturous to write out in full—**PROOF** is *always* a finite and deterministic verification that occurs on huge numbers.⁵ Once again, the ingenious encoding scheme of Gödel-numbering is what allows these functions and predicates to treat integers as ordered sequences. Without prime bases and distinct exponents, this would not be possible!

As a final recap before the crescendo, we have projected all symbols, statements, theorems, and rules of **PM** into the world of integers (**PM**'s own domain of expertise!) with Gödel-numbering. Every Gödel-number has a very distinct (implicit) meaning and ties back to the language of **PM**. With this numbering scheme, we can represent transitions from statement to statement in **PM** using rules of inference, as arithmetic operations from number to number, never needing a potentially infinite test or infinite predicate. We can, with numbers, talk about whether certain theorems can be derived using certain rules, and thus we can make predicates that test if a given Gödel-number of a formula is derived by another massive number representing a stepwise proof. **PM** is finally talking about its own provability via Gödel-numbering.⁶

⁴Once again, the process we just witnessed to verify that **C** and **D** are valid continuations of the derivation will start with axioms and eventually bring us to be able to use **A** and **B** as theorems.

⁵It cannot be overstated that **PROOF** is a **PM**-defined predicate that evaluates integers, just like **IsEven** or **IsPrime** or **IsPowerOf5** would be. To us—highly conscious of why we are writing such ornate numerical functions and predicates—we know it is doing much more, namely a rigorous proof-check. But in the end, it is just checking for numerical properties present in numbers.

⁶The **PROOF** predicate is LITERALLY a Turing machine:

Tape: The Gödel number **x** (that massive integer encoding the proof chain) Reading: Extracting exponents by dividing by primes (moving along the tape) State transitions: Checking if **DEM**(**x**_{**i**}, **x**_{**j**}) holds (the finite state logic) Halting: When you've verified the whole chain ends in **y**

And the mechanical repetition - dividing by 2, then 3, then 5... extracting exponents one by one - that's EXACTLY the "stupid but universal" repetitive action of a Turing machine head moving along tape cells! This is no coincidence! Gödel published in 1931, Turing in 1936. Turing was explicitly influenced by Gödel's work. What you're seeing is that Gödel essentially invented Turing machines without realizing it - his arithmetization process IS mechanical computation!

9.6 Formula G

Thus, we are ready to construct Gödel's **Formula G**. First we start with a preliminary **PM** formula

$$\mathbf{P} = \neg \exists \mathbf{x} : \mathbf{PROOF}(\mathbf{x}, \mathbf{y})$$

This says flatly: ‘*there is no Gödel-number $\mathbf{x}$ that is the proof string for the Gödel-number $\mathbf{y}$* ’. It asserts that $\mathbf{y}$ is simply not provable by any $\mathbf{x}$. $\mathbf{y}$ being a free variable here clearly makes it false, because some formulae (one of which could be subbed into $\mathbf{y}$) are definitely provable! While this is a powerful statement, it seems pointless since $\mathbf{y}$ is just an arbitrary stand in for a formula, nowhere near self-reference. We can imagine, however, that we could conjoin the **PROOF** predicate in here with another one that describes $\mathbf{y}$ in some way. How about the **PM** formula

$$\mathbf{U} = \neg \exists \mathbf{x} : \mathbf{PROOF}(\mathbf{x}, \mathbf{y}) \quad \& \quad \mathbf{SUB}(\mathbf{z}, \mathbf{z}, \mathbf{y})$$

This says flatly: ‘*there is no Gödel-number $\mathbf{x}$ that is the proof string for the Gödel-number $\mathbf{y}$, where $\mathbf{y}$ is a Gödel-number of a formula obtained by substituting the Gödel-number $\mathbf{z}$ into its own free variables*’. $\mathbf{U}$ is a **PM**-formula (not a Gödel-number!) asserts that no arbitrary self-subbed formula—here, the Gödel-number $\mathbf{y}$ —is provable, because $\mathbf{z}$ here is an unnamed free variable.⁷ Let's call $\mathbf{U}$'s Gödel-number $\mathbf{T}$. Now we simply disobey $\mathbf{U}$ and plug $\mathbf{T}$ into its free variable, $\mathbf{z}$, to produce

$$\mathbf{G} = \neg \exists \mathbf{x} : \mathbf{PROOF}(\mathbf{x}, \mathbf{y}) \quad \& \quad \mathbf{SUB}(\mathbf{T}, \mathbf{T}, \mathbf{y})$$

What does it say? “*Gödel-number $\mathbf{y}$, which is $\mathbf{T}$ subbed into the formula represented by $\mathbf{T}$, has no demonstration $\mathbf{x}$ that is its proof*”, or “ *$\mathbf{T}$ subbed into itself is not provable.*” Well, what is $\mathbf{T}$ subbed into itself? **G!!!!** Subbing $\mathbf{T}$ into $\mathbf{U}$ (what $\mathbf{T}$ denotes) is the process we followed to obtain $\mathbf{G}$, yet $\mathbf{G}$ asserts that such a process yields an unprovable formula! Not only have we gotten **PM** to talk about specific theorems of **PM**, transitions between any two formulae, **PM** statements are now talking about themselves. And once they can talk about themselves, *they can scream*.

Recognize again the implications of a string in a formal system that asserts its own unprovability: if $\mathbf{G}$ is provable in **PM**, then **PM** is lying and inconsistent (because $\mathbf{G}$ says that it is not provable), and if $\mathbf{G}$ is not provable then $\mathbf{G}$ is true, but unprovable, meaning **PM** is incomplete. Either it permeates all truths and it lies, or it is unable to express all truths. And the most sinister part of all is that we showed this within the system itself. Used its own tools to dismantle it.

Not to be discussed in detail here, even if you cleverly take $\mathbf{G}$ or $\neg \mathbf{G}$ to be an axiom of an *extended PM*, you will still be able to prove the exact same kind of statement, although the more you augment what feels like a logical sensible system, the farther you become from your sane understanding of the system itself. Remember, the formal system of **PM** itself knows or cares not about math as we know it at all. You could make a formal system that deliberately reads the opposite from **PM**, but could be entirely consistent. Formal systems are just strict symbolic axiomatic rule systems. We ascribe the meaning to the symbols by interpreting them, but if your intelligence hinders your ability to recognize the mechanical nature of the system, you will not be operating formally. The symbols of **PM**, the power endowed in it from the instant that the axioms are rules were formalized, unknowingly house the tools to destroy its own credibility. From mindless, atomic units, an extremely high level realization, stretching far outside math and into philosophy and cognitive science, can emerge.

10 The Limits of Formality

In 1931, Kurt Gödel showed the world that no formal system that represents arithmetic is immune to undecidable statements. Rippling through the world of mathematics and logic, his results, rather than discouraging peers, vehemently inspired them to think similarly outside the box to come to newer, more powerful realizations, or at least very fascinating analogues to Gödel's results. One of such peers was Alan Turing. While Gödel proved the

⁷Self-substitution is not too complicated. If we have our **Substitute** function that takes in two Gödel-numbers, then we simply identify the free variables in the first one (by just making sure they are not qualified or defined in the beginning of the formula), then rebase the second input at the points where that free variable exists. Self-subbing “**SS0** = $\mathbf{x}$ ” would result in “**SS0** = (**SS0** = $\mathbf{x}$)”

$\mathbf{U} = \neg \exists \mathbf{x} : \mathbf{PROOF}(\mathbf{x}, \mathbf{y}) \quad \& \quad \mathbf{SUB}(\mathbf{z}, \mathbf{z}, \mathbf{y})$. Is this statement true? Well, it does not matter for us, but we can already imagine that it is not, because we can think of $\mathbf{zs}$ that might be true when subbed into themselves. Subbing “ *$\mathbf{x}$ is true or $\mathbf{x}$ is false*” into itself to get “ *$\mathbf{x}$ is true or false*” IS TRUE OR “ *$\mathbf{x}$ is true or false*” IS FALSE”.

inconsistency-or-incompleteness of all axiomatic formal systems of number theory, the question was still out, posed by **David Hilbert**, on whether there exists a general procedure (or algorithm) for deciding the viability of a statement. ‘*Is there a set of inputs that will cause this predicate to evaluate to true?*’ ‘*Is there a solution to this Diophantine equation?*’ Hilbert was not interested in answering these questions for a given predicate, but he was interested in formalizing the process of doing so for any arbitrary formula. Gödel’s work showed that there are certain formulae that are undecidable: it is not clear as to whether or not the statements are true... provable... He showed that there are always some that cause this undecidability, but in Hilbert’s question, such a decision procedure would say that **G** is not provable. It would say that “**x** = **0** & **Ey** : **x** = **Sy**” is *not* solvable *for any* **x**, and it would say that “**x** = **0** $\vee$ **Ey** : **x** = **Sy**” is solvable/true *for all* **x**. On a given example, we know how to work this problem out, but we are essentially looking for a Turing Machine that can decide the solvability of any possible such formula.

10.1 Devil’s Advocate

Before we approach the decision problem with Turing, it is essential to finally return to his machines and reflect on some examples of Gödelian scenarios that are necessary precedent for Turing’s (spoiler) disproof of the possibility for a general decision procedure for theoremhood or satisfiability. Again, we must study the set of computable numbers and show why it is limited. As a review, Turing’s machines were ideally *circle-free*, meaning they run forever, never reaching a contradiction, always having a next step. Following Turing’s paper exactly, we will hear out a devil’s argument, suggesting that the computable numbers are *not* enumerable.

“Suppose computable numbers are enumerable. Let α_n be the **n-th** computable sequence, and let $\beta_n(\mathbf{m})$ be the **m-th** figure in α_n . Essentially we have a list of computable sequences where α allows us to index them for the **n-th** sequence. Within a given sequence, α_n , $\beta_n(\mathbf{m})$ allows us to access the **m-th** digit of that sequence. Let γ be the sequence with $1 - \beta_n(\mathbf{n})$ as its **n-th** figure. In other words, γ is the diagonal of this list—the list of computable sequences which α indexes—with each **0** and **1** flipped (hey, Cantor). Since γ is clearly computable from the list, there exists a number **K** such that $1 - \beta_n(\mathbf{n}) = \beta_K(\mathbf{n})$, for all **n**. That is, the sequence γ is the **K-th** sequence of our list. And for **n**, $1 - \beta_n(\mathbf{n}) = \beta_K(\mathbf{n})$ and thus $1 = \beta_n(\mathbf{n}) + \beta_K(\mathbf{n})$. Putting **n** = **K**, $1 = \beta_K(\mathbf{K}) + \beta_K(\mathbf{K})$, which becomes $1 = 2 * \beta_K(\mathbf{K})$, ie, **1** is even. Clearly this is impossible, so our assumption that the computables are enumerable must be incorrect.”

While this argument is seductively compelling, the true fallacy (hmm?) lies in the assumption that γ is a possibly computable sequence. To recall, we created γ by diagonalizing the list of computable sequences and flipping the bits, which seems very possible if the computable sequences are so enumerable. What was our error, then? Well, our definition of computable numbers are numbers that can be computed with finite means, finite instructions, and thus in a Turing Machine. Turing Machines compute fixed sequences, and we gave each a **SD**, then turned that **SD** into a unique **SD** integer. So enumerating the computable sequences, as discussed earlier, is (at worst) as easy as enumerating the natural positive integers. Hence, they are enumerable. Right?

10.2 Proven Boundaries of Turing Machines

It is not quite so simple. The devil’s advocate argument would be true if we could enumerate all computable sequences by finite means, but this process would equivalently amount to determining, in a finite amount of steps, whether or not a given **SD** integer is one of a circle-free Turing Machine. There is no such general finite process, and to show this, we again apply diagonalization. The most simple way to show this would be to say that with such a process, we could create γ , which we know leads to contradictions. Surely valid, it leaves us feeling that something is missing. To illustrate it further, we imagine constructing a sequence even simpler than γ , γ' , where we just flip back each bit to what it was originally (or never flipping them at all). It is the simple diagonal, with no shift from **1** to **0** or **0** to **1**. γ' ’s **n-th** figure is $\alpha_n(\mathbf{n})$, not **1** minus that value. Let us now suppose we *do* have such a decision process for **SD** integers; we can create a Turing Machine, **D**, which, when supplied with the **SD** of a machine, **M**, tests it and if **M** is circle-free, prints an ‘**s**’ and if it is circular, a ‘**u**’. (Satisfactory or Unsatisfactory). It does its work, clears the tape, then leaves one of these two letters. Combining **D** with **U**, the universal machine, we can create the machine **H**, which will produce γ' . **H** will work by enumerating the positive integers, trivially converting them to **SD**s, then passing them to **D**, which will leave a **u** or an **s**. In the first **N-1** ‘sections’ of **H**’s progress, the numbers **1**, **2**, **3**, ..., **N-1** have been written down and tested by **D** as **SD**s, whether they’re valid or not. A certain number of these integers, **R(N-1)**, would be those of viable circle-free machines, that is, the number **R** accumulates only satisfactory machines, while **N** keeps climbing. In the **N-th** section, it tests if **N**

is a viable **SD** integer, and if it is, $\mathbf{R}(\mathbf{N}) = \mathbf{R}(\mathbf{N} - 1) + 1$, and if not, $\mathbf{R}(\mathbf{N}) = \mathbf{R}(\mathbf{N} - 1)$. Then, to continue to compute γ' , if $\mathbf{N}$ was satisfactory, we use $\mathbf{U}$ to write out the first $\mathbf{R}(\mathbf{N})$ figures of $\mathbf{N}$'s machine, then add the $\mathbf{R}(\mathbf{N})$ -th figure to our accumulation of γ' . γ' , again, is the diagonal of all the sequences produced by circle-free machines, as identified by $\mathbf{D}$.

From its construction, we can see that $\mathbf{H}$ is circle-free. For each $\mathbf{N}$, it will either be circle-free or not, then if it isn't, we move to the next number to test, and if it is, we calculate at most $\mathbf{R}(\mathbf{N})$ more figures from $\mathbf{U}$. Each step of $\mathbf{H}$ is terminating—it works on each example, finishes, then goes to the next example, ad infinitum, never becoming circular. Now, let $\mathbf{K}$ be the **SD** integer of $\mathbf{H}$ itself. What does $\mathbf{H}$ do in the $\mathbf{K}$ -th section of its work? As we just established, $\mathbf{D}$ will decide that $\mathbf{K}$ is a circle-free machine and it will not write $\mathbf{u}$. However, it *must not* be able to write $\mathbf{s}$ either.

Why?

If it would write $\mathbf{s}$ in the $\mathbf{K}$ -th section, then it would need to compute the first $\mathbf{R}(\mathbf{K} - 1) + 1 = \mathbf{R}(\mathbf{K})$ of the machine with $\mathbf{K}$ as its **SD** integer and to add the $\mathbf{R}(\mathbf{K})$ -th figure into the sequence γ' . Computing the first $\mathbf{R}(\mathbf{K} - 1)$ figures would happen seamlessly as discussed, however the instructions for calculating the $\mathbf{R}(\mathbf{K})$ -th figure amount to “*calculate the first $\mathbf{R}(\mathbf{K})$ figures of $\mathbf{H}$ and write down the $\mathbf{R}(\mathbf{K})$ -th*” Well that sounds exactly like we just did to get here! That figure would never be found, as $\mathbf{H}$ would be caught in an infinite loop. Calculating the first $\mathbf{R}(\mathbf{K} - 1)$ figures of $\mathbf{H}$ would amount to basically repeating all of its work up to this point, but then to compute the $\mathbf{R}(\mathbf{K})$ -th, it needs to do it all over again! This will never ever end, and thus $\mathbf{H}$ seems to be circular, contrary to our earlier findings. Neither ruling is possible and thus machine $\mathbf{D}$ is contradictory and impossible. $\mathbf{H}$ has a problem with its own **SD** integer. Could it just skip that one? Well, we know that a given sequence can be encoded by more than one machine, so $\mathbf{H}$ would need to avoid those machines as well. But, what about a machine that is equivalent to $\mathbf{H}$, but its output has its **27th** and **54th** digits swapped, for instance? What about the **3rd** and **19th** and **101st** all switched? There are an infinite set of such variations, and $\mathbf{H}$ screening for all of them is an impossible task. $\mathbf{D}$ is simply not possible. If it were, we could use it to create γ' which we could use to make γ , and we know this is impossible. Thus, the computable sequences are enumerable, however it is impossible to actually enumerate them. This is an incredible result, more commonly known now as *The Halting Problem* in computer science, which we will look at in another lens shortly.

First, however, we can further prove that there is no machine, $\mathbf{E}$, that can decide, in general, whether a given **SD** that corresponds to a machine that will ever print a given symbol, say $\mathbf{0}$. ($\mathbf{E}$ stands for *Ever print*.) The key is to show that if we had an $\mathbf{E}$, we could use it to decide whether a machine, $\mathbf{M}$, will ever print a symbol *infinitely often*. If we could do this for one symbol, we could then do it for all symbols in a machine's vocabulary and then see whether that machine prints symbols infinitely—whether it is circle-free. Let $\mathbf{M}'$ be the machine that prints exactly the same sequence as $\mathbf{M}$, but the first $\mathbf{0}$ (we are using this as the symbol for this example) is instead written as $\bar{\mathbf{0}}$. If $\mathbf{M}$ produces “**A101D0A10...**” then $\mathbf{M}'$ produces “**A101D0A10...**”. Then we can imagine a machine, $\mathbf{M}''$, that will produce “**A101D0A10...**”, and so on where $\mathbf{M}$ with $\mathbf{N}$'s will replace the first $\mathbf{N}$ $\mathbf{0}$ s with $\bar{\mathbf{0}}$.

Now imagine a machine, $\mathbf{F}$, that will sequentially write the **SD**s of $\mathbf{M}$, $\mathbf{M}'$, $\mathbf{M}''$, etc. Such a machine is possible. Remembering our $\mathbf{1/3}$ machine example, one would simply have to add some configurations for the beginning which print $\bar{\mathbf{0}}$ instead of $\mathbf{0}$, $\mathbf{N}$ times, then go back to normal. Using $\mathbf{F}$ and $\mathbf{E}$, we can now construct a machine, $\mathbf{G}$. $\mathbf{G}$'s first step would be to write out the **SD** of $\mathbf{M}$ (using $\mathbf{F}$) and test—with $\mathbf{E}$ —if $\mathbf{M}$ ever prints $\mathbf{0}$ or not. If it *never* does, $\mathbf{G}$ will print “**:0:**”. If it *ever* does, $\mathbf{G}$ will print nothing. Then it will move on to $\mathbf{M}'$ and do the same thing. Recognize that if $\mathbf{M}$ never prints $\mathbf{0}$ or only does so a *finite* amount of times, then $\mathbf{G}$ will print $\mathbf{0}$ infinitely often. If, however, $\mathbf{M}$ prints $\mathbf{0}$ *infinitely* often, then $\mathbf{G}$ will **never** print $\mathbf{0}$. Hopefully you see where this goes. What happens when we test $\mathbf{G}$ with $\mathbf{E}$? If $\mathbf{G}$ never prints $\mathbf{0}$, then we know that $\mathbf{M}$ prints $\mathbf{0}$ infinitely. If $\mathbf{G}$ *ever* prints $\mathbf{0}$, then we *know* that $\mathbf{M}$ does not print $\mathbf{0}$ infinitely often. A similar process can be done for $\mathbf{1}$, or any other symbol in an $\mathbf{M}$'s finite vocabulary. Tying these all together, we can create a machine that if $\mathbf{M}$ is circle-free, a proven impossibility. Therefore, we know that no machine $\mathbf{E}$ can exist.

Our question of the enumerability of computable sequences has led to three awesome realizations:

1. no machine, $\mathbf{D}$, can determine whether or not an arbitrary **SD** integer corresponds to a circle-free machine
2. no machine, $\mathbf{E}$, can determine whether an arbitrary **SD** integer corresponds to a machine that will print some symbol ever, or infinitely often
3. the set of computable numbers is enumerable in terms of cardinality, but not practically so

But it is enumerable, because you can just include them as a subset of the integers! It is confusing, yet it is

just a testament to the inherent incompleteness of any formalized system of truth and logic. It shows us that even mathematics and logic which we hold as so natural and fundamental, are just internally created abstractions for the world. These systems just break down in the edge cases, much like physics in the quantum world. It does not mean that we are wrong, it just means that there is no possible way to be *absolutely* correct. And this is a result that should comfort us and only emphasize the marvel that *any* of this stuff works. Ontologists cannot tell us how we are even here, or why, and the mysteries of consciousness emerging from purely randomly evolved physical machines still confound us, yet we have created an incredibly impactful and useful set of systems to model the world for our own advantage. The fact that we cannot prove them to be infallibly and universally true is a concession to the mystery of life that I am willing to make.

10.3 The Halting Problem in CS

Borrowing more from conventions of most programming languages, like **Python**, we will describe an analogue to Turing's circle-free problem that will decide if a given program will ever halt. Already pretty similar. Again as a review, while Turing needed the concept of circularity and non circularity for his machines and their purpose in his paper, most computer programs these days ideally do halt eventually and return an answer. If code went on forever, we would not really be able to do anything with it or gather information from it. As a construct for computation, infinity is fine. The main premise of the modernized halting problem is whether or not there can exist a program, an algorithm (a Turing Machine) that can take in the source code of a program, and an input to that program, and determine whether the program will halt on that input. Let us say we have a very simple program, **S(x)**, which returns the successor of **x**. In a programming language with math built in, we can define: "**S(x): return (x + 1)**".

Unsurprisingly, this function takes a number and returns that number plus one. In programming, it is important to realize that '**3**' is exactly identical to '**S(2)**' and **31** is exactly identical to **S(30)**. The function evaluates exactly to **x + 1**, as the return statement provides. So if we wanted to decide whether this program halts on a given input, we could just say that if **x** is a number, then this program will halt and return **x**'s successor. Great. There can be and are tons of functions/programs that are vastly more complicated than this one, sometimes returning a number, but sometimes having errors that allow them to run 'forever' and cause the program to be killed by the OS.

For instance, a function that may be more prone to running forever is one that, for any even number, finds two prime numbers that add up to the number. This is function is related to a famous unproven theorem (oxymoron, yes... unprovedn theorems are often called conjectures) by **Christian Goldbach** which says that all evens are the sum of two primes. On a given even input, this function will just search until it finds two primes that satisfy. For every input we have ever tested, the program halts, however, since the conjecture has not been proven, we do not have the *assurance* that the program will halt on *every* even input. Before we think about *any input*, however, we just need to focus on one input. So, we define the halting function,

H(P, i), to return **TRUE** if **P(i)** halts, and return **FALSE** if **P(i)** runs forever.

P is just the source code of the program (like '**S(x): return (x + 1)**'.....we can imagine an **SD**, bytecode, DNA, some text that represents a program's code), and **i** is just an input to the program **P**. If **P** comes to an answer when fed **i**, **H(P, i) = TRUE**, but if **P** never terminates when given **i** as an input, **H(P, i) = FALSE**.

10.3.1 Original Problem

Now for the proof:

1. Assume this function—**H**—exists
2. We can then construct **G(P)**, such that if **H(P, P) = TRUE**, then **G(P)** will run forever, and if **H(P, P) = FALSE**, then **G(P)** will halt immediately
 - **G** uses **H(P, i)** to test if a given program will halt when given its own source code as an input. If this does not make sense, you are in the right boat. This is a pathological function that is constructed exclusively for this proof... familiar, eh?
3. If **H** tells **G** that **P** halts on **P**, then **G** just does the opposite
4. Consider **G(G)** (that is, feed **G**'s own source code to **G**)
5. To solve this, **G** runs **H(G, G)**. If that returns **TRUE** (that is, if **G** halts on **G** as input), then **G(G)** will run forever.
 - Wait, that makes no sense, because **H** just said that **G** passed to **G** as an input will halt!
6. Suppose **H(G, G)** returns **FALSE** (that is, if **G** does not halt on **G** as input), then **G(G)** will immediately halt.
 - But that contradicts **H**'s ruling, and remember **H** is the function we are assuming is true and valid.

This is a clear contradiction, thus asserting that our assumption, of the possibility of **H(P, i)** being possible, is wrong. So no computer program of any kind can, in general, decide if a given program will halt on a given input. Wow.

Let us take it even further. We can show that no algorithm, **A(P)**, can determine whether a program, **P**, will halt on *all* inputs, or if *at least one* will cause it to run forever.

Notice that such a function would allow us to immediately prove or disprove the **Goldbach Conjecture**—and many similar unsolved proofs—because we could design a function to take in an even number, then run until it finds two primes that add up to it. If we had **A(Goldbach)**, we could immediately see whether all inputs halt, which would tell us that all evens are the sum of two primes.

To disprove **A(P)**, we will show that **A(P)** entails the possibility of **H(P, i)**, which we now know is impossible. Using the skills we have already seen now several times for constructing pathological, targeted functions to cause a contradiction, how could we use **A(P)** to make **H(P, i)**, which is arguably already a less powerful function because it does **A(P)**'s work but just on one input, not all inputs.

10.3.2 Expanded Problem

Here is the proof:

1. Assume this function—**A**—exists
2. Let **H(P, i)** be a function that immediately defines **newFunction(x)**
 - **newFunction(x)** will behave simply. If $x = i$, then it returns **P(i)**. Otherwise, halt immediately.
3. After defining it, **H(P, i)** simply returns **A(newFunction)**
 - **A** knows **P**'s behavior for *all* possible inputs, but **H(P, i)**—proven impossible—answers it for a specific one. Within our new **H**, we just obfuscate **A**'s vision by asserting that for all inputs except the one we care about—**i**—our new function halts. This now means that the determining factor of **A**'s answer with respect to **P** is—the only question **A** is actually even now answering within **newFunction**—is the behavior of **P(i)**
4. **H** has been proven impossible, so the assumption of **A(P)** has led directly to our ability to reconstruct an impossible function. A contradiction that demands the abandonment of the assumption.

Rather identical to Turing's original formulation of this problem, the emergent realization rings true: no finitely defined program or Turing machine can decide the ultimate fate, the end behavior of another program or machine. We cannot formulate all-knowing power. Again, a somewhat reassuring and humbling result.

11 The Entscheidungsproblem

With all the precedent covered, it is now time to investigate Turing's true intentions in writing his paper and inventing universal computing machines. Touched on with Halting and with Gödel, the issue at hand is the *Entscheidungsproblem*, the decision problem in first order logic, which also corresponds to the problem of assessing satisfiability of Diophantine equations. The problem is not concerned with how to solve such equations/predicates on a problem-by-problem basis, but *whether or not there can exist a general algorithm with which to determine the satisfiability of any given predicate in first order logic*.

Can we invent a Turing Machine that will be able to be fed *any* **PM**-style predicate on its tape, encoded in a way we will soon discuss, and come to a verdict in a finite amount of time about whether or not it has an assignment that leads to truth. In contrast to Gödel, who showed that in a system like **PM** there will always be statements like **G** and $\neg\mathbf{G}$ that are not provable, Turing here wants to show that there cannot be a procedure to come to such conclusions about the solvability of a statement in general. After all, our realizations and meaning derived from the Gödel-numbered strings we worked with were wholly in our minds, not at all explicitly in the system itself. If, in contrast to what Gödel showed, we could show that any statement **P** will either be provable or its negation, $\neg\mathbf{P}$, will be provable, immediately we would have a solution to the decision problem because we could have a Turing Machine crank out every possible theorem of **PM**, check each one for equivalence with **P** and $\neg\mathbf{P}$, eventually certainly reaching one of the two. In principle this makes sense, but before we see how Turing gets there, we must understand basically how such a machine could even work.

We will imagine a machine, **K**, that has configurations and vocabulary corresponding to a system like **PM**, which we will just call **PM**. We can imagine the Peano axioms being encoded on the tape as well as configurations corresponding to rules of inference with which we can mutate axioms, make theorems, then mutate theorems, eternally churning out tautologies. After all, the whole point of formalizing logical systems is to be able to mechanically derive one theorem from the next—no intuition needed on the part of the manipulator—which fits right into the wheelhouse of Turing Machines.

Before a Turing Machine can do this, we have to establish the symbols we have as vocabulary as well as a set of configurations to manage essentially equal phrases like

" $\forall x : \exists y : \mathbf{Sx} = \mathbf{y}$ " and " $\forall x : (\exists y : \mathbf{Sx} = \mathbf{y})$ " and " $\forall x : (\exists y : (\mathbf{Sx} = \mathbf{y}))$ "

It would need to classify

" $\forall x : \exists y : \mathbf{Sx} = \mathbf{y}$ " and " $\forall y : \exists x : \mathbf{Sy} = \mathbf{x}$ "

as the same as well. There are various alternative notations that could make these problems easier to solve, such as representing "**A**&**B** $\vee$ **C**&**D**" as " $\vee\&\mathbf{AB}\&\mathbf{CD}$ " where the connective is written first and 'defines' where

the statement is heading. We need not dwell on these minor details because it has been shown that such issues can be dealt with, and we might imagine how. The main point here is that a machine **K** could certainly be imagined to start with axioms on its tape, much like an **SD** on the tape of **U**, and in a **breadth-first** approach, generate all possible theorems in **PM**, which are defined by being derivable through rules of inference—encoded into configurations of **K**...while it would be a cumbersome list, it certainly could be done finitely—applied successively to axioms.

What is breadth-first? It is a paradigm for enumerating/searching spaces that involve several dimensions or directions. For instance, a given pair of axioms may be able to be manipulated by a rule, turned into a theorem that then can easily be multiplied/expanded by that same rule, ad infinitum. Imagine a repeated use of **LEM** to continue to make ever more complex statements essentially asserting the same thing with increasing conjunctions:

$$\mathbf{A} \rightarrow \mathbf{A} \vee \neg \mathbf{A} \rightarrow (\mathbf{A} \vee \neg \mathbf{A}) \vee \neg(\mathbf{A} \vee \neg \mathbf{A}) \rightarrow \dots$$

These are all theorems of **PM**, however if **K** got ‘started on this path’ of derivations, it would never be able to stop and move onto the next dimension or direction of proofs. What this would look like practically—and yes, this is a nightmare and a pure combinatorial explosion, but nonetheless effective on a machine with infinite memory and theoretically instant calculation—would be to apply to each theorem in our current pool on the tape just one transformation via a rule; loop through the pool of theorems and apply **LEM** when possible. Then do so for the next rule until we exhaust our rules on this ‘round’ of theorems. Then you have a new bigger pool or generation of theorems with which to repeat the process. Every single new theorem will always be a tautology, but this way, we concentrically expand our generated theorems, making progress once in every direction before continuing in this direction. This approach prioritizes making process in all directions, ie thinking **broad**. **Depth-first** would cause us to get stuck in an endless **LEM** typhoon. With this provision, we can imagine **K**. Each generation would get exponentially larger and more complex, but, again, in this theoretical machine that we can inductively imagine remaining consistent, we have all the time and space in the world.

Assuming we have **K**, Turing first introduces an idea of a machine $\mathbf{K}_\alpha$ that will be able to generate boolean sequences according to certain predicates, by using **K**. Suppose the predicate **IsOdd(x)**, which we want to be true for odd numbers. The boolean sequence for this predicate would ideally look like “010101010...” (interestingly the exact same as our first example **1/3** machine), because **0** is not odd, **1** is, **2** isn’t, **3** is, etc. Each index of the sequence corresponds to a natural number. How would $\mathbf{K}_\alpha$ work? We want to imagine a predicate $\mathbf{G}_\alpha(\mathbf{x})$ that is true if the **x-th** digit of α , the sequence, is **1**, so $\neg \mathbf{G}_\alpha(\mathbf{x})$ would mean the **x-th** digit is **0**. In our example, $\mathbf{G}_\alpha(\mathbf{x})$ is **IsOdd(x)**. Further, we need to conjoin all of the axioms necessary to ‘support’ whatever proposition **G** is. So in our case we would want the Peano axioms which allow for multiplication, because our **IsOdd(x)** predicate could look like: “ $\neg \exists \mathbf{y} : \mathbf{Product}(\mathbf{y}, \mathbf{SS0}) = \mathbf{x}$ ”. For simplicity, however, we will just imagine this conjunction of axioms, **P**, to be all of the Peano axioms necessary for all of **PM**. Again, infinite space on the tape so no need to mince axioms. On top of our predicate of axioms, **P**, we need to define $\mathbf{F}_n$, which is another conjunction of predicates for a given **n** that amounts to:

$$\mathbf{F}(\mathbf{0}, \mathbf{S0}) \ \& \ \mathbf{F}(\mathbf{S0}, \mathbf{SS0}) \ \& \ \dots \ \& \ \mathbf{F}(\mathbf{S}_{n-1}\mathbf{0}, \mathbf{S}_n\mathbf{0})$$

all the way up to **n**, essentially asserting that all the numbers up through **n** are direct successors, basically that **n** is a number. Redundant seeming, but necessary. Now we can propose

$$\forall \mathbf{n} : \mathbf{A}_n \vee \mathbf{B}_n$$

where

$$\mathbf{A}_n \iff \mathbf{P} \& \mathbf{F}_n \rightarrow \mathbf{G}_\alpha(\mathbf{n})$$

and

$$\mathbf{B}_n \iff \mathbf{P} \& \mathbf{F}_n \rightarrow \neg \mathbf{G}_\alpha(\mathbf{n})$$

That for any **n**—which we must remember is actually a 0 with **n** markers like **S** or *****—provided we have the Peano axioms and that successorship holds up through **n**, either **G** is true or false.

Back to our example, the following formulae will be provable in **PM** and by $\mathbf{K}_\alpha$:

- $\mathbf{B}_0 : \mathbf{P} \& \mathbf{F}_0 \rightarrow \neg \mathbf{IsOdd}(\mathbf{0})$
- $\mathbf{A}_1 : \mathbf{P} \& \mathbf{F}_1 \rightarrow \mathbf{IsOdd}(\mathbf{1})$

- $B_2 : P \& F_2 \rightarrow \neg \text{IsOdd}(2)$
- $A_3 : P \& F_3 \rightarrow \text{IsOdd}(3)$
-

Since we have conjoined the axioms necessary to define any binary predicate, we know that either that predicate or its negation is true for all values. Thus we can now show that K_α is computable. The motion of K_α would entail for each n writing on its tape both A and B . We know one of these will be provable formulae of PM , so we simply then run K until one of the two is reached, and when one is, we write either “:1:” or “:0:” for either A or B . Also, since for any n , we know this process will terminate and move freely to $n + 1$, K_α is circle-free. This establishes our ability not only to churn out every possible theorem of a logical calculus, but better yet, to compute truth sequences for any imaginable predicate in that calculus. The standardization of logic into symbols freed humans to rigorously prove and formalize, never letting the ambiguity of language stand in the way of binary truth or falsehood, and from there, such mechanization now lends itself to a mindless machine being able to thanklessly prove or disprove all of our questions. But can it do so for *any* question?

Returning to the Entscheidungsproblem, we recall that if Gödel had not shown his results, we’d think K would eventually find every single provable theorem, so the problem would not exist. Any formula we’d like to decide in terms of satisfiability, we just let K run until it hits that formula or its negation. However, due to Gödel’s results, we know that if we set K loose with the PM axioms, it would *never* reach statements like Gödel’s G *nor its negation*.

Now we will begin to flip what we just did—formalizing first-order logic into a Turing Machine—and formalize the motion of a Turing Machine into the language of first-order logic. Corresponding to each computing machine, M , we will construct a formula $Un(M)$ —for *Undecidable*—and we will show that if there is a general procedure to determine whether $Un(M)$ is provable, that we will have a method to tell whether an arbitrary machine will ever print 0 , something we have already disproven. This is yet another example of the beautiful collaborative and cumulative nature of proofs in math and logic: **oftentimes if we find an equivalence between two problems, one is much easier to solve, or one is already solved!** In this case, since we have already disproven some things, having to do with halting and ever-print, all we need now show is that the decision procedure for $Un(M)$ is equivalent to one of these disproven concepts. $Un(M)$ will function as a counterexample like G : a formula that no decision procedure could crack.

To define $Un(M)$, Turing first defines several necessary propositional functions. To define these functions, briefly we recall that Turing Machines have limited vocabularies of symbols, a limited set of configurations, an infinite tape of numbered or indexable squares, and a sequential numbering of *complete configurations* (the full tape snapshot and next machine configuration/state). For no reason other than avoiding any ambiguity, when we say *complete configuration number n*, we are referring to the n -th state that a machine has been through. Not the n -th configuration in the list of possible configurations, the n -th *complete* configuration means after $n - 1$ steps/states have been completed by the machine. When we say *machine configuration n* we are referring to Q_n .

- $R_{S_i}(x, y)$: “in the *complete* configuration number x (of M) the symbol on square y is S_i ”
 - It asks whether S_i is on y -th in complete configuration x .
- $I(x, y)$: “in the complete configuration x , square y is scanned”
 - It asks whether M is scanning y -th square in complete configuration x , the x -th complete configuration.
- $K_{Q_i}(x)$: in the x -th complete configuration the *machine* configuration is Q_i .
 - It asks whether the x -th complete configuration is the machine configuration named Q_i .
- $F(x, y)$: y is x ’s immediate successor
- $N(x)$: x is a positive natural integer
 - This proposition really does not add much because we already assume this.

So far, these propositions refer to specific aspects of the machine’s current state or the relationship between numbers on its tape. These are all obviously deterministic and testable, either returning true or false. Before

moving on, recall that each distinct machine configuration/state in $\mathbf{M}$ is determined not just by the configuration name/index, but by the scanned square it operates on (the machine configurations are overloaded). For instance, there are as many $\mathbf{Q}_1$ machine configurations as symbols in $\mathbf{M}$, because each one *could* be scanned in that configuration and we need a plan explicitly defined. Each configuration uniquely corresponds to one symbol scanned, one symbol to be printed, one move of the head, then one configuration to jump to next.

Using our previously defined propositions, we will now look at a much larger one tying things together, $\mathbf{Inst}(\mathbf{Q}_i, \mathbf{S}_j, \mathbf{S}_k, \mathbf{L}, \mathbf{Q}_1)$ (we will define for $\mathbf{L}$, but an analogous definition holds for $\mathbf{R}$ or $\mathbf{N}$):

$\mathbf{Inst}(\mathbf{Q}_i, \mathbf{S}_j, \mathbf{S}_k, \mathbf{L}, \mathbf{Q}_1) =$

$$\forall \mathbf{x} : \forall \mathbf{y} : \forall \mathbf{x}' : \forall \mathbf{y}' : (\mathbf{R}_{\mathbf{S}_j}(\mathbf{x}, \mathbf{y}) \& \mathbf{I}(\mathbf{x}, \mathbf{y}) \& \mathbf{K}_{\mathbf{Q}_i}(\mathbf{x}) \& \mathbf{F}(\mathbf{x}, \mathbf{x}') \& \mathbf{F}(\mathbf{y}', \mathbf{y}))$$

$$\rightarrow (\quad \mathbf{I}(\mathbf{x}', \mathbf{y}') \& \mathbf{R}_{\mathbf{S}_k}(\mathbf{x}', \mathbf{y}') \& \mathbf{K}_{\mathbf{Q}_1}(\mathbf{x}') \quad$$

$$\& \quad \forall \mathbf{z} : [\quad \mathbf{F}(\mathbf{y}', \mathbf{z}) \vee ([\mathbf{R}_{\mathbf{S}_0}(\mathbf{x}, \mathbf{z}) \rightarrow \mathbf{R}_{\mathbf{S}_0}(\mathbf{x}', \mathbf{z})] \& [\mathbf{R}_{\mathbf{S}_1}(\mathbf{x}, \mathbf{z}) \rightarrow \mathbf{R}_{\mathbf{S}_1}(\mathbf{x}', \mathbf{z})] \& \dots \& [\mathbf{R}_{\mathbf{S}_M}(\mathbf{x}, \mathbf{z}) \rightarrow \mathbf{R}_{\mathbf{S}_M}(\mathbf{x}', \mathbf{z})]) \quad])$$

You get all that?

- The **first** section of this proposition ensures that:
 - (with $\mathbf{R}$) the scanned symbol is $\mathbf{S}_j$
 - (with $\mathbf{I}$) the square scanned in the $\mathbf{x}$ -th complete configuration is the $\mathbf{y}$ -th square
 - (with $\mathbf{K}$) that in the $\mathbf{x}$ -th complete configuration, the machine configuration is $\mathbf{Q}_i$, and that $\mathbf{x}'$ directly succeeds $\mathbf{x}$
 - This section provides that ‘all is right’ in the machine during the $\mathbf{x}$ -th complete configuration and that the next complete configuration, $\mathbf{x}'$, is indeed next.
- The **next** section provides that:
 - (with $\mathbf{I}$) in the $\mathbf{x}'$ -th complete configuration, the $\mathbf{y}'$ -th square is scanned
 - (with $\mathbf{R}$) in the $\mathbf{x}'$ -th step, the $\mathbf{y}$ -th square contains (as was just printed in $\mathbf{x}/\mathbf{Q}_i$) $\mathbf{S}_k$
 - (with $\mathbf{K}$) the $\mathbf{x}'$ -th complete configuration is $\mathbf{Q}_1$
 - Minding that this is $\mathbf{Inst}(, , \mathbf{L},)$, now the square we are reading is the $\mathbf{y}'$ -th which is one **lower** than $\mathbf{y}$, that we previously in $\mathbf{x}/\mathbf{Q}_i$ printed $\mathbf{S}_k$ into square $\mathbf{y}$, and that our machine configuration in $\mathbf{x}'$ is $\mathbf{Q}_1$ (as provided by the quintuple for $\mathbf{Q}_i$).
- The **final** section ensures all other squares remain the same. That:
 - for all $\mathbf{z}$, either $\mathbf{z}$ is the successor to $\mathbf{y}'$ (meaning $\mathbf{z}$ is $\mathbf{y}$) or “there being a $\mathbf{S}_0$ symbol in square $\mathbf{z}$ in $\mathbf{x}$ implies that it is the same in $\mathbf{x}'$ ”, for all symbols in $\mathbf{M}$
 - We want all squares to either stay identical, or be square $\mathbf{y}$ to which we just printed $\mathbf{S}_k$ as per the machine configuration.

To visualize this, let us get a snapshot of what the tape could look like in the $\mathbf{x}$ -th step and $\mathbf{x}'$ -th ($\mathbf{Q}_i$ to $\mathbf{Q}_1$) to make $\mathbf{Inst}$ true:

$$\left[\quad \dots \mid \mid \overset{\mathbf{x}/\mathbf{Q}_i}{\mathbf{S}_j} \mid \dots \quad \right]$$

(where the scanned square is the $\mathbf{y}$ -th)

$$\begin{array}{c} x'/Q_i \\ [\dots | \text{ } | S_k | \dots] \end{array}$$

Inst simply asserts for a given machine configuration, in this case Q_i on scanning S_j , that all the changes to the tape and *complete* configuration are correct. In this case, the quintet that was being tested by **Inst** was $Q_i - S_j - S_k - L - Q_i$, so rightly, Q_i scanned S_j , then printed S_k , moved left, then went to Q_i .

Can you imagine how we will abstract on this proposition to characterize, with logical propositions, the entire behavior of a whole machine?

Recall that our goal in this exercise is to construct a predicate $\mathbf{Un}(\mathbf{M})$ on machine $\mathbf{M}$ that will entail $\mathbf{M}$ ever printing $\mathbf{0}$ (or some other symbol) and vice versa. If we can establish this relationship between $\mathbf{Un}$ and the printing of a symbol, then we will know that $\mathbf{Un}$ is an untenable proposition as it would suggest a general process to determine whether an arbitrary $\mathbf{M}$ will print a symbol, which has been disproven. Turing is not satisfied enough with his application of the imaginary Turing Machine to show the halting problem, to define computable numbers, (to eventually invent computers!), but now he is yet again using his construct as a basis to establish some immensely complex logical statements that are provably undecidable to solve Hilbert's **Entscheidungsproblem**. If we can turn “will $\mathbf{M}$ ever print $\mathbf{0}$ ” into a proposition, then we have found our proposition that disproves the possibility of the **Entscheidungsprocedure**, as I like to call it.

If **Inst** allows us to encapsulate and the validity of one complete configuration in $\mathbf{M}$ moving to the next, then naturally, we can look to chain these **Inst** statements together for all steps in $\mathbf{M}$ and for all **Inst** propositions, then just tack on “ $\exists x : \exists y : \mathbf{R}_{S_1}(x, y)$ ”, meaning ‘in some complete configuration number x and in some square number y , that square reads S_1 , which represents $\mathbf{0}$ ’. We will now build the further abstractions upon **Inst**. Our next proposition will characterize all of the machine configurations of $\mathbf{M}$ and how they relate to each other and essentially conjoin the entire sequential **Description** of $\mathbf{M}$ as a predicate.

We will call it $\mathbf{Des}(\mathbf{M})$. For our $\mathbf{1/3}$ example,

$$\begin{aligned} \mathbf{Des}(\mathbf{M}_{1/3}) = & \mathbf{Inst}(\mathbf{Q1}, \mathbf{S0}, \mathbf{S0}, \mathbf{R}, \mathbf{Q2}) \\ & \& \mathbf{Inst}(\mathbf{Q2}, \mathbf{S0}, \mathbf{S1}, \mathbf{R}, \mathbf{Q3}) \\ & \& \mathbf{Inst}(\mathbf{Q3}, \mathbf{S0}, \mathbf{S0}, \mathbf{R}, \mathbf{Q4}) \\ & \& \mathbf{Inst}(\mathbf{Q4}, \mathbf{S0}, \mathbf{S2}, \mathbf{R}, \mathbf{Q1}) \end{aligned}$$

For all the machine configurations in a machine, $\mathbf{M}$, $\mathbf{Des}(\mathbf{M})$ will contain a conjunction of all of such **Inst** propositions. If $\mathbf{Des}(\mathbf{M})$ is true, we have a machine with configurations that will not collide with each other, that will behave.

Finally we are ready for the proposition, $\mathbf{Un}(\mathbf{M})$, that will entail and be entailed by—in other words, have an **if and only if**, $\iff$, relationship—the machine printing $\mathbf{0}$.

$\text{Un}(\mathbf{M}) =$

$$\begin{aligned} \exists \mathbf{u} : [& \mathbf{N}(\mathbf{u}) \ \& \ \forall \mathbf{x} : (\mathbf{N}(\mathbf{x}) \rightarrow \exists \mathbf{x}' : \mathbf{F}(\mathbf{x}, \mathbf{x}')) \ \& \ \forall \mathbf{y} : \forall \mathbf{z} : (\mathbf{F}(\mathbf{y}, \mathbf{z}) \rightarrow \mathbf{N}(\mathbf{y}) \& \mathbf{N}(\mathbf{z})) \ \& \ \forall \mathbf{y} : \mathbf{R}_{\mathbf{S}_0}(\mathbf{u}, \mathbf{y}) \\ & \& \ \mathbf{I}(\mathbf{u}, \mathbf{u}) \ \& \ \mathbf{K}_{\mathbf{Q}_1}(\mathbf{u}) \ \& \ \mathbf{Des}(\mathbf{M}) \] \\ \rightarrow & \ \exists \mathbf{s} : \exists \mathbf{t} : \mathbf{N}(\mathbf{s}) \& \mathbf{N}(\mathbf{t}) \ \& \ \mathbf{R}_{\mathbf{S}_1}(\mathbf{s}, \mathbf{t}) \end{aligned}$$

- Starting with the **last** section, we simply state that:
 - there are two numbers, $\mathbf{s}$ and $\mathbf{t}$, such that $\mathbf{S}_1(\mathbf{0})$ appears in complete configuration $\mathbf{s}$ on square $\mathbf{t}$ of the tape
- The **beginning** section states that:
 - there is a number, $\mathbf{u}$, that denotes the first complete configuration and the first square in $\mathbf{M}$
 - for every $\mathbf{x}$ there is a successor, $\mathbf{x}'$
 - (with $\mathbf{R}$) in complete configuration $\mathbf{u}$, every square $\mathbf{y}$ (which is a number with a successor $\mathbf{z}$) on the tape is $\mathbf{S}_0$ (blank)
 - This simply describes the initial condition of the tape in the $\mathbf{u}$ -th (first) complete configuration.
- **Next**, we establish that:
 - (with $\mathbf{I}$) we are in the $\mathbf{u}$ -th square in the $\mathbf{u}$ -th complete configuration
 - (with $\mathbf{K}$) that the $\mathbf{u}$ -th *machine* configuration is $\mathbf{Q}_1$
 - (with $\mathbf{Des}(\mathbf{M})$) that we have a functional valid machine, $\mathbf{M}$

Seems simple enough: the tape starts empty, the head starts in the first square on the first configuration $\mathbf{Q}_1$, and we have non-conflicting **Inst** terms conjoined to make a proper machine. We propose that this scenario entails the machine printing $\mathbf{S}_1/\mathbf{0}$.

Recognize that this proposition, $\text{Un}(\mathbf{M})$, entails

1. our *ever-print* machine $\mathbf{E}$, which entails
2. our **SD**-based machine $\mathbf{D}$, that determines circularity, and
3. our Halting function $\mathbf{H}(\mathbf{P}, \mathbf{i})$

These have all been proven impossible!

While we are close, we must flesh out the entailment relationship in Un between the **first part of** the proposition, which we will redefine as $\mathbf{A}(\mathbf{M})$, and $\mathbf{M}$ ever printing $\mathbf{0}$.

$\mathbf{A}(\mathbf{M}) =$

$$\begin{aligned} \exists \mathbf{u} : [& \mathbf{N}(\mathbf{u}) \ \& \ \forall \mathbf{x} : (\mathbf{N}(\mathbf{x}) \rightarrow \exists \mathbf{x}' : \mathbf{F}(\mathbf{x}, \mathbf{x}')) \ \& \ \forall \mathbf{y} : \forall \mathbf{z} : (\mathbf{F}(\mathbf{y}, \mathbf{z}) \rightarrow \mathbf{N}(\mathbf{y}) \& \mathbf{N}(\mathbf{z})) \ \& \ \forall \mathbf{y} : \mathbf{R}_{\mathbf{S}_0}(\mathbf{u}, \mathbf{y}) \\ & \& \ \mathbf{I}(\mathbf{u}, \mathbf{u}) \ \& \ \mathbf{K}_{\mathbf{Q}_1}(\mathbf{u}) \ \& \ \mathbf{Des}(\mathbf{M}) \] \end{aligned}$$

Thus, $\text{Un}(\mathbf{M}) =$

$$\mathbf{A}(\mathbf{M}) \rightarrow \exists \mathbf{s} : \exists \mathbf{t} : \mathbf{N}(\mathbf{s}) \& \mathbf{N}(\mathbf{t}) \ \& \ \mathbf{R}_{\mathbf{S}_1}(\mathbf{s}, \mathbf{t})$$

To establish the factuality of $\mathbf{Un}(\mathbf{M})$, the dual entailment that $\mathbf{Un}(\mathbf{M})$ is true **if and only if** $\mathbf{S}_1/0$ is printed on the tape. We will work through

Two Lemmas:

1. If $\mathbf{S}_1$ appears on the tape in some complete configuration of M , then $\mathbf{Un}(M)$ is provable
2. If $\mathbf{Un}(M)$ is provable, $\mathbf{S}_1$ appears on the tape in some complete configuration of M

Starting with **Lemma 1**, we must first establish some variations of our propositions above into some functions. Whereas $\mathbf{R}$, $\mathbf{I}$, and $\mathbf{K}$ return booleans, $\mathbf{r}$, $\mathbf{i}$, and $\mathbf{k}$ will instead return the values of each that would make the uppercase propositions true. That is: $\mathbf{R}_{\mathbf{S}_{r_n, m}}(\mathbf{n}, \mathbf{m})$, $\mathbf{I}(\mathbf{n}, \mathbf{i}_n)$, $\mathbf{K}_{\mathbf{Q}_{k_n}}(\mathbf{n})$ are all tautologies.

- $\mathbf{r}_{n, m}$ returns the index of the symbol that appears on square $\mathbf{m}$ in the $\mathbf{n}$ -th complete configuration
- $\mathbf{i}_n$ returns the number/index of the scanned square in $\mathbf{n}$ -th complete configuration
- $\mathbf{k}_n$ returns the index of the machine configuration in the $\mathbf{n}$ -th complete configuration of $\mathbf{M}$

The reason for the tautological nature of the three above propositions is that we are asking, for instance, *is the character on the $\mathbf{m}$ -th square in complete configuration $\mathbf{n}$ the character on square $\mathbf{m}$ in complete configuration $\mathbf{n}$?*

So we can suppose that in the $\mathbf{n}$ -th complete configuration, the sequence of symbols on the tape is: " $\mathbf{S}_{r_{n, 0}}, \mathbf{S}_{r_{n, 1}}, \dots, \mathbf{S}_{r_{n, n}}$ " followed by only blanks, that the scanned symbol is the $\mathbf{i}_n$ -th, and that the machine configuration is $\mathbf{Q}_{k_n}$.

We may then propose the following statement,

$\mathbf{CC}_n =$

$$\mathbf{R}_{\mathbf{S}_{r_{n, 0}}}(\mathbf{n}, 0) \& \mathbf{R}_{\mathbf{S}_{r_{n, 1}}}(\mathbf{n}, 1) \& \dots \& \mathbf{R}_{\mathbf{S}_{r_{n, n}}}(\mathbf{n}, \mathbf{n}) \quad \& \quad \mathbf{I}(\mathbf{n}, \mathbf{i}_n) \quad \& \quad \mathbf{K}_{\mathbf{Q}_{k_n}}(\mathbf{n})$$

$$\& \quad [\quad \forall \mathbf{y} : \mathbf{F}(\mathbf{y}, 1) \vee \mathbf{F}(0, \mathbf{y}) \vee \mathbf{F}(1, \mathbf{y}) \vee \dots \vee \mathbf{F}(\mathbf{n} - 1, \mathbf{y}) \quad \vee \quad \mathbf{R}_{\mathbf{S}_0}(\mathbf{n}, \mathbf{y}) \quad]$$

I am borrowing Arabic numerals here for the sake of brevity and typing restrictions, but essentially what this proposition boils down to is: every square in the $\mathbf{n}$ -th complete configuration is the proper square for the $\mathbf{n}$ -th complete configuration of $\mathbf{M}$, the $\mathbf{i}_n$ -th square is the scanned square in $\mathbf{n}$ and $\mathbf{Q}_{k_n}$ is the machine configuration in the $\mathbf{n}$ -th complete configuration (these are all tautologies), and every $\mathbf{y}$ denoting a square number is properly succeeding the one that came before it, and if $\mathbf{y}$ is greater than $\mathbf{n}$, then it is blank. The **first** part is purely tautological and the **latter** part establishes trivial seeming propositions for the $\mathbf{n}$ -th complete configuration stating that all squares are either under $\mathbf{n}$ or blank.

$\mathbf{CC}_n$ essentially confirms once again that ‘all is correct’ in the $\mathbf{n}$ -th complete configuration of $\mathbf{M}$, just more concisely. Briefly, we can propose that $\mathbf{F}_n$ will stand for " $\mathbf{F}(0, 1) \& \mathbf{F}(1, 2) \& \dots \& \mathbf{F}(\mathbf{n} - 1, \mathbf{n})$ " as a shorthand. Now we define

$\mathbf{CF}_n =$

$$\mathbf{A}(\mathbf{M}) \quad \& \quad \mathbf{F}_n \quad \rightarrow \quad \mathbf{CC}_n$$

$\mathbf{CC}$ and $\mathbf{CF}$ are similar, however $\mathbf{CF}$ just includes $\mathbf{A}(\mathbf{M})$, which defines our machine, and $\mathbf{F}_n$, which very simply assures proper successorship between all successive complete configurations up to $\mathbf{n}$. Whereas $\mathbf{A}(\mathbf{n})$ defines the changes between states of the machine, $\mathbf{CF}$ explicitly defines the situation on the tape for specific configurations. Whereas $\mathbf{A}(\mathbf{M})$ defines the changes between all states/complete configurations of the machine, $\mathbf{CF}$ explicitly defines the situation on the tape for specific complete configurations.

We must now prove $\mathbf{CF}_n$ by induction. First to prove $\mathbf{CF}_0$, we need to show that $\mathbf{A}(\mathbf{M})$ entails $\mathbf{CF}_0$, which just reduces to:

$$\forall \mathbf{y} : \mathbf{R}_{\mathbf{S}_0}(\mathbf{0}, \mathbf{y}) \quad \& \quad \mathbf{I}(\mathbf{0}, \mathbf{0}) \quad \& \quad \mathbf{K}_{\mathbf{Q}_1}(\mathbf{0})$$

which contains three predicates that are directly taken from $\mathbf{A}(\mathbf{M})$. So clearly $\mathbf{A}(\mathbf{M}) \rightarrow \mathbf{CF}_0$. Now, by the definition of induction, to prove $\mathbf{CF}_n \rightarrow \mathbf{CF}_{n+1}$ would be to prove $\mathbf{CF}_n$ for all n because we have covered our base case of $\mathbf{0}$. We have three cases to consider, corresponding to the three possible changes of head position of $\mathbf{M}$ between complete configurations n and $n+1$. As usual, we will show it for $\mathbf{L}$ and assume similar demonstrations for $\mathbf{R}$ and $\mathbf{N}$.

We first define some variables.

- **a:** k_n
 - the index of the machine configuration in the n -th complete configuration
- **b:** $r_n(i_n)$
 - the index of the symbol scanned in the square i_n , which is scanned during complete configuration n
- **c:** k_{n+1}
 - the index of the machine configuration in the $(n+1)$ -th complete configuration
- **d:** $r_{n+1}(i_n)$
 - the index of the *new* symbol in the square i_n
 - (because the first argument is $n+1$, meaning the *next* complete configuration)

We now conclude that $\mathbf{Des}(\mathbf{M})$ must contain $\mathbf{Inst}(\mathbf{Q}_a, \mathbf{S}_b, \mathbf{S}_d, \mathbf{L}, \mathbf{Q}_c)$ as one of its terms. Hence,

$$\mathbf{Des}(\mathbf{M}) \rightarrow \mathbf{Inst}(\mathbf{Q}_a, \mathbf{S}_b, \mathbf{S}_d, \mathbf{L}, \mathbf{Q}_c)$$

and further,

$$\mathbf{A}(\mathbf{M}) \rightarrow \mathbf{Inst}(\mathbf{Q}_a, \mathbf{S}_b, \mathbf{S}_d, \mathbf{L}, \mathbf{Q}_c)$$

and, trivially, even further,

$$\mathbf{A}(\mathbf{M}) \quad \& \quad \mathbf{F}_{n+1} \rightarrow \mathbf{Inst}(\mathbf{Q}_a, \mathbf{S}_b, \mathbf{S}_d, \mathbf{L}, \mathbf{Q}_c) \quad \& \quad \mathbf{F}_{n+1}$$

But, we can also immediately see that

$$\mathbf{Inst}(\mathbf{Q}_a, \mathbf{S}_b, \mathbf{S}_d, \mathbf{L}, \mathbf{Q}_c) \quad \& \quad \mathbf{F}_{n+1} \rightarrow [\mathbf{CC}_n \rightarrow \mathbf{CC}_{n+1}]$$

This is because the fundamental provision of $\mathbf{Inst}$ is that the **transition** from a complete configuration with *this* machine configuration—in this case $\mathbf{Q}_a$ —preserves the whole tape correctly. Thus we see that

$$\mathbf{A}(\mathbf{M}) \quad \& \quad \mathbf{F}_{n+1} \rightarrow [\mathbf{CC}_n \rightarrow \mathbf{CC}_{n+1}]$$

From here, we can say

$$\mathbf{A}(\mathbf{M}) \quad \& \quad [\mathbf{F}_{n+1} \rightarrow \mathbf{CC}_n] \rightarrow \mathbf{A}(\mathbf{M}) \quad \& \quad [\mathbf{F}_{n+1} \rightarrow \mathbf{CC}_{n+1}]$$

to finally show

$$\mathbf{CF}_n \rightarrow \mathbf{CF}_{n+1}$$

Now, it is the assumption of **Lemma 1** that $\mathbf{S}_1$ appears on the tape. That is, for some $\mathbf{N}$ and $\mathbf{K}$, $\mathbf{CC}_N$ has as one of its terms: $\mathbf{R}_{\mathbf{S}_1}(\mathbf{N}, \mathbf{K})$ and thus that $\mathbf{CC}_N \rightarrow \mathbf{R}_{\mathbf{S}_1}(\mathbf{N}, \mathbf{K})$ is provable and so is then $\mathbf{A}(\mathbf{M}) \& \mathbf{F}_N \& \mathbf{F}_K \rightarrow \mathbf{CC}_N$.

We can now see that $\mathbf{A}(\mathbf{M}) \& \mathbf{F}_N \& \mathbf{F}_K \rightarrow \mathbf{R}_{S_1}(\mathbf{N}, \mathbf{K})$, which is a mirror image of our definition of $\mathbf{Un}(\mathbf{M})$ above. Because of the nature of $\mathbf{Un}(\mathbf{M})$, **Lemma 2** becomes very clear to show, because an implication is only true if the implied outcome is true, and the outcome implied in $\mathbf{Un}(\mathbf{M})$ is that $\mathbf{S}_1/0$ is printed on the tape. $\mathbf{Un}(\mathbf{M})$ is provable if and only if the tape of $\mathbf{M}$ contains $\mathbf{S}_1/0$ in some complete configuration.

$$\mathbf{Un}(\mathbf{M}) \iff \mathbf{M} \text{ prints } 0$$

Now we can refute the *Entscheidungsprocedure*. Suppose the contrary, then we have a mechanical, finite process to prove $\mathbf{Un}(\mathbf{M})$ for any $\mathbf{M}$ and any $\mathbf{n}$ (in $\mathbf{CF}_n$), but by **Lemma 1** and **Lemma 2**, we'd then have a finite process to infallibly determine whether a machine ever prints a symbol, and we'd have a procedure to determine whether any machine is circle-free, which have been proven impossible! A procedure to decide all instances of $\mathbf{Un}(\mathbf{M})$ would entail impossible power. Hence there can be no *Entscheidungsprocedure* and the *Entscheidungsproblem* can have no solution.

Recognize that it is fundamentally different from Gödel's results, because while Gödel showed an example of an undecidable string, Turing showed that there is an infinite class of these impossible predicates that are fundamentally unworkable. Whereas the assumed decision procedure would end up classifying Gödel's $\mathbf{G}$ and $\neg\mathbf{G}$ both as undecidable, Turing showed a scaffold for the kind of problem it could never solve. Having laid the groundwork by refuting the possibility of the machines $\mathbf{E}$ and $\mathbf{D}$, he needed only to show a predicate that would attempt to do what they did. And that he did.

It was pretty obvious, anyway, right? Clearly we intelligent humans who tamed fire, solved medicine and agriculture, and produce amazing art *absolutely could never* devise a process by which to solve any logical problem or simple algebraic equation! While I joke, this result gives us no choice but to introspect. Propositional logic is as airtight as it feels, but it is fundamentally limited. It deals in a domain of concepts that, if they behaved any other way, would entail a human population that was no more intelligent than one of dogs. While it feels like a fairly certain step, expanding the domain of logic to one where we could have shorthand formulae, quantifiers, and thus the means to define number theory—notably in a way that seems infallible! *The Peano axioms and their ability to permeate so many realms of mathematics as we know it with grace are incredible. They make so much sense and seem to spell no doom*—it instead endows a domain of utterly ungovernable complexity.

12 So Are Our Systems Wrong?

The temptation after reading Gödel and Turing may be then to ask where we went wrong in formalizing first order logic, numbers, and computability. Surely a self-referential paradox can be worked around as it had been by Russell. for a time.

The issue or shortcoming was never ever in the systems we created, the math we used, or the logic that underpins our sanity. There is not some error in our current formulations of these systems that could one day resolve the decision problem or decide on self-referential statements like Gödel's, or ever compute a non-computable number. The error, instead, was in expecting that we could ever formalize all truth above a level. The error was in expecting a computer program that could determine the fate of all other computer programs; by Turing's construction, even, this is more clearly seen as our faculties for understanding how a given imaginary program would work simply breaks down when it has to handle something to do with its own function or the function of a pathologically opposite program from it. In the context of the 'real' world outside of formal systems, there is an infinite amount of randomness, unpredictability, and an abyss of a trove of knowledge we will never grasp. In formalizing aspects of the world down onto a page or a tape and symbols, inevitably reduces it by (how many?) dimensions, yet it still works very well for many contexts. Our formal systems are nowhere near useless, they are just not all-knowing, and that is a somewhat humbling and reassuring result. In the context above propositional logic, which dealt with a very limited set of elements, we demand lots of representability by our formal systems—a gargantuan amount—and as soon as they start working, we expect that they have thus captured reality and distilled it in a bottle. Yet if we could so easily formalize all truths or compute randomness then I think we would be much more likely in a simulation.

The miracle is that out of a soup of pure chaos, patterns arose, order (perceptible by us) established itself, and we have been able to find a guiding light in the dark room in which we grasp for straws. To expect that a sophisticated physical machine could establish universal rules of any kind is silly, and we must be content to know our limits, that we can appreciate what we have done that much more. Gödel and Turing got famous by showing

what was impossible, yet in doing so, in how they did it, they tamed a new kind of fire for us that has since raged—within our meticulously scaped forests of silicon—like a thousand suns.

13 The Surging Wake of Turing’s Insight

Alan Turing, in developing a construct for solving a problem—the problem of solvable problems, no less—in first order logic and mathematics, precisely tapped his finger on fundamentally huge ideas in computing and information theory. He conceived, likely from the observation of how we humans compute problems on paper, with other contemporaries that there must be a primitive set of common actions involved in computing anything. With this understanding, he realized that programmability of these actions was the name of the game, with the physical medium having been established for executing primitive tasks. It begs a discussion on bootstrapping and the explosive innovation in digital computing that surfed in the wake of Turing’s work. In branching from constructs of arbitrary machines, via the Standard Description encoding, to the **Universal Turing Machine** that, if supplied with proper instructions, would use the same simple mechanisms, just in different ways to transform its function, Turing bridged from arcane computation to a real kind of system that could be implemented automatically. With **John Von Neumann** stimulating things from several angles, this concept of programmability was the defining principle of the evolution of computing.

To almost criminally gloss over details, modern **RAM**-based computers follow the constructs of a **UTM** closely with some important re-imaginings. The tape in Turing’s machines was infinite and the machine had no concept of being on or off, but in today’s computers the closest analogues are the **Random Access Memory** (which disappears when the computer is turned off) and the persistent, non-volatile memory on, say, a disk (where **RAM** can be transferred to in many cases). Using physical circuits called *latches* and *flip-flops*, data can be physically stored on command as a **0** or **1**. Wiring millions of these together in a uniform hierarchical array, programs and data may be stored persistently with electricity, to be read and rewritten on command (by *random access*). Whereas the **UTM** stores machine programs (the **SD** of another machine) in a separate section of the tape, instructions are often intermixed with data throughout these memory arrays. Instead of a read/write head which could access and interpret symbols on the tape, we have **Central Processing Units**, which are physical pieces of circuitry that can read and write values from memory using electricity. The **CPU**, really just a complex circuit, ‘reads’ bytes (handfuls of **8** bits at a time) in a certain section and those values read in (electrical signals!) propagate through the circuit and manipulate charge elsewhere, eventually then rewriting something back into memory. The **CPU** has a control unit that handles ‘finding’ different data, again, by feeding an ‘address’ (a binary representation of different slots in the memory array) to a decoding circuit that will immediately ‘unlock’ the channel/pathway to the desired data or memory unit, allowing its information to flow into the **CPU**. Instead of a tape that moves, we have an integrated circuit that opens and closes to different areas of stored memory based on electrical signals. Instead of machine configurations, we have **Operation Codes** that are built into the **CPU** by the manufacturer and denote a limited set of primitive physical operations that the **CPU** can do. To make these operations happen, the **CPU** has specialized subsections, such as the **Arithmetic Logic Unit** which can perform logical (**AND**, **OR**, **NOT**) operations and **arithmetic** on binary data. Whatever is ‘done’ by the **CPU** is easily rewritten to memory through a circuit that unlocks the specific desired memory slot. Once this architecture is set up, as Turing showed, code could be written in the proper encoding (for computers, to start, binary), then put into the tape/memory and then used by the controlling unit (**head/CPU**). Both computing paradigms ultimately amount to storing and primitively manipulating data many many times. The physical machine does nothing but obey physics that, by its construction, manifests in manipulations of data in very simple ways. If we read the list of **opcodes** for a given computer and its **CPU** (such as **LOAD** data from a cell, **WRITE** to a cell, **MOVE TO** this program cell.right out of the **UTM!**), we could imagine similarly encoding a machine to write alternating **0s** and **1s** to places in memory, as we did with our first Turing Machine. We will assume that the bridge between typing in binary digits and those digits being stored in memory is established, so we can just think cleverly about how to program, not how to physically implement binary code.

Obviously (since I am typing this on a computer, not thinking at all about binary or circuits to do so), there is much more that can be done. We can compute anything computable on these things, so how do we abstract away from binary? First of all, recognize that any kind of abstract encoding or translation we could do certainly falls well within the realm of computing, so all of this work should be done on the computer. No new hardware needs to be introduced, we just have to bite the bullet and start developing shorthands for this tedious machine language.

The first abstraction up from machine code (in binary) was the invention and implementation of **assembler** languages. Along with our assumption of the input/output functionality in the operating system, we assume that there is a trivial process or program with which to turn plain text (that you may see on a computer’s screen)

into binary representations with **Unicode** (in order for you to read these letters, that functionality is already in place and it just amounts to a lookup table converting symbols to standardized binary). If we can type in English which gets converted into binary, and we can write code in binary into memory, we can begin programming a new language. Suppose we wanted to first make 'plain text' shorthands for all the **opcodes** from machine code. Whereas each **opcode** may be **8 digits of binary**, we can turn them into, say, **4** or **5** letter english key words.

Suppose this was as far as we wanted to go in our first new language. We want to write a program file using binary to represent all data and memory addresses, but English text to dictate **opcodes** to be done on different pieces of data, then have this run on the computer as if it was just all written in machine code in the first place. What do we need to do? Suppose a simple program in our assembler language looks like “**WRIT** **10110110 00001000**” and it translates to a program that **writes** the second byte (*the binary representation of the number 8*) to the memory address specified by the first byte. Suppose there is an **opcode**—again, a binary code specific to our **CPU**—for this kind of operation that is “**10101101**” and it acts in the same way on the two following bytes of data. Suppose **Unicode** representations of the letters “**W R I T**” are, however, “**1111 1100 0110 1010**” and joined into two bytes to become “**11111100 01101010**”. Our program file in memory will look like “**11111100 01101010 10110110 00001000**” once it is typed in. We need those first two bytes to be somehow translated into “**10101101**”, the actual machine number for the instruction **WRIT**.

All we have at our disposal as of yet is machine code, so we must write (in binary machine code!) a program that can take a file like this and turn **Unicode** sequences into their proper **opcodes**. Suppose also in our assembly language you have to prepend instructions by “**11111111**” and data/arguments by “**00000000**” to make it easier for things to be translated, so our program is actually: “**11111111 11111100 01101010 00000000 10110110 00001000**”.

What we need is a **compiler**, a program, in machine code, that can take in a file in **assembler** and then turn it into a machine code set of instructions that the computer can actually use (i.e. this program would become “**10101101 10110110 00001000**”). It would need to recognize full “**1**” bytes and “**0**” bytes and—in our example—only focus on the “**1**” bytes and translate them properly. This process is hard to imagine constructing even for a set of **10** possible machine instructions written in plain text, but this is the process that must be taken if we want convenience, if we want to eventually free ourselves from the primitive machine code to actually use the machine. We know binary machine code, with the hardware endowment, is capable of such a task, because we could easily do this task on paper, so we just must bite the bullet and work through that translation.

This compiler program, we will call it **compiler A**, is written in binary, typed into the computer and stored somewhere specific, that it may be found and executed. It must be an executable sequence of machine code. Supposing we have written this compiler, now we may write programs with plain text **opcodes** and binary data and addresses, and our lives are that much easier.

While our binary compiler is great and represents a gargantuan leap from tedious physical code to something more readable and conceivable, suppose we wanted to break free completely from writing in machine code and we want to now rewrite our compiler in the new assembly language. Can we do it? What is the task? Keeping in mind that we need ultimately to use machine byte code to do anything, we want to write a compiler (we will call it **compiler B**) *in our assembler language* and use **compiler A** to turn it into an executable file of byte machine code. This way, we can forever kiss goodbye **compiler A** and only use **B**. Whereas we wrote **A** in machine code to translate from **Unicode** to **opcode** (doubtless using operations like *writing*), now we get to use our English assembly codes to stand in for all that extra binary, and writing **B** is conceivably much easier than writing **A**. Once **B** is written and finally compiled by **A**, we have now completely cut ties with ever writing programs in machine code ever again. We can use all the assembly language we want, then just use the compiled **B** to compile assembly programs.

This process is called **Bootstrapping**, referring to the concept of ‘*picking yourself up by your own bootstraps*’, which is quite apt. It is a seminal feat of **abstraction**, using lower levels (in terms of abstract-ness and complexity of concepts, certainly a *high* level of complication) of systems to construct a higher level language that maps down to the lower levels, so that we can interpret things better from above. It is an objective fact that reading binary is a much more taxing process for humans than plain text, so while we need the binary level to function properly below as it is the medium in which anything ultimately happens, to work with it, to read it and write it, it would be much better to seal ourselves off with shorthands and abstract constructs. Think of driving a car or even walking: you are absolutely not thinking about each electrical pulse to your muscles to keep you balanced or keep you in your lane, your body and brain have abstracted such primitive operations away from you, so that you need only think in a higher-level context. Turn right, speed up, look both ways before you cross. This kind of abstraction, sealing off lower levels with proper mappings up, is fundamental to computing and even more so to consciousness.

Bootstrapping has since occurred so many times that now, to encode a Neural Network program that classifies

hand-drawn digits on a 28x28 pixel grid, one needs *maybe* **100** lines of Python code that reads like English and simple matrix multiplication notation from linear algebra class. Computations governed by a couple words in English get rapidly translated into binary and executed *billions* of times per second. It is honestly not possible for humans to comprehend the speed at which we let these translations and electrical manipulations happen, but we know, from our construction, what is happening at each vibration of the crystals imbuing our **CPUs** with life. Alan Turing's mechanisms of primitive symbol manipulation being stacked and stacked and stacked to make a high-level program have, with ever improving hardware, been allowed to explode in speed, (shrink in) space, and thus applicability, all while preserving and depending on the basics at the foundation.

14 Turing Machine Learning

15 LLM Embeddings Through the Lens of Gödel-Numbering

16 Complexity from Simplicity

At this point, a brief discussion on complexity seems necessary. While we have traversed from the basics of computation to its fundamental limits, the most pervasive takeaway I still feel from this content is the marvel of what can arise when simplicity at the foundation of a system is given the time, space, and ability to scale up. I am certain that many readers will still not quite believe—or at least fully appreciate and *feel*—that a Turing Machine with its **5**-part configurations and *highly* limited and limiting (both crucial!) set of rules can do all the computation that any other computer in the world could. *Well surely it couldn't run IOS, ChatGPT, the programs in, say, an MRI.* If you zoomed into the computer on which you're reading this or on which this was typed or on which you will go and Google Alan Turing or Kurt Gödel and its basic rules were presented to you as Turing's were here, you'd have the same disbelief. Turing Machines today are faster and smaller than Alan could have ever imagined, but they are still doing exactly what he professed. No **CPU** is ever *ever* doing more than one thing at a time. No **CPU** is solving any problem more simple than **AND**, **OR**, or **NOT**. The complexity of our modern technology, the biological technology that has given life on Earth, even of the celestial bodies in space all arise from simple rules and a ton of scale

Some examples of complexity arising from utter simplicity include

- **4** kinds of bases make up **DNA**, which dictates exactly how every protein in the body is built and thus behaves
- Ants in a colony follow chemical signals and tend to behave how their neighbors behave, yet colonies have built functional cities underground, sacrifice themselves to build a bridge for their colony, and effectively transport foods multiple of their weight all day every day to the center of their colony
- Neurons in the brain either fire or do not fire, based on a threshold of electrical input either being surpassed or not, and from such simple **AND** gates we get consciousness, on top of all the primitive functionality below
- A handful of rigid axioms and rules of substitution and implication endow all the logic, math, and science we can conjure, which we have used to build the world we have today
- Turing Machines can only read, write, and shift one window at any given step in their motion, yet the entire corpus of electronics that have revolutionized every area of human activity, even the internet and Artificial Intelligence is carried out by machines that are *at most* as complex as Turing Machines at their foundation
- The wrapper of skin on your body which feels like a continuous glove of membrane, a seal on your innards through which nothing we can see can get is made up of *discrete cells* which have gaps between them
- There are **26** primitive symbols we use to encode any possible thought in language
- Taking a quantity of anything, then imagining adding a single element or taking a single element away is all you need to define all of math
- Neural Networks, which underlie all of the most advanced Artificial Intelligence of this current time operate by the simple rules: take an input, take a handful of weighted sums with a handful of sets of stored numbers, compare that product to what the 'correct' answer is, adjust stored numbers slightly to get a more correct answer next round, repeat with lots of data

This nowhere-near-exhaustive and utterly disorganized list is meant to be kindling in a hopeful unending wildfire of thought—*hey, **thought** itself is a highly complex outcome from simple physics—about—wildfire of thought* is also a high-level idea made up of smaller more primitive pieces...what are the traits of a wildfire?...how can I apply that to the idea of *thought*...ah, I see what it means now—the beautifully complex life you’re living and the simple and seemingly not-powerful-enough ideas that make up its base.

We are programmed this way, yes, but in your next conversation or the next time you stand up and walk or eat something or get over a cold or even read this next word, try to make a list of all the processes that your body does to make it happen. Could you even make a rough list of parts for a human body, forgetting even the implementation and design, and come *anywhere* close to what we’ve got?

What happens when you feel hungry or you feel you need to go to the restroom or you feel kinda sick? *Oh, well my body is telling me something.* How? Some kind of electrochemical signal made enough noise in your brain to breach out of the subconscious. Where did it come from? How did it learn to speak when it needed your attention?

The next time you listen to an audio recording or look at a photograph or smell something familiar, think about the decoding your brain is doing. *Well this voice sounds like that singer’s voice, so it must be him.* How did you even know it was a voice versus the bark of a dog or the hum of an air conditioner or the strumming of a guitar? How do you take in a heap of pixels and *literally be unable to see the trees in the forest* of the now-contextualized and identified image. Could you write a program that could do that process in even a whole day of computation? And, uh, yea, it took you a slice of a second that you could never even measure.

We owe existence a bit more appreciation. *Aren’t we so lucky to be here with friends and family whom we love tonight?* Aren’t we lucky to be sane at all?! Aren’t we lucky to be not just alive but aware that we’re alive, enough to enjoy a sunset or gulp down some refreshing water or sleep soundly and know we’ll wake up in the same sane machine?!

What kicked off this process of the formation of planets, of the formation of elementary particles and atoms, of the self-organizing marvel of biological—did you catch even the simple pieces that make up that *word*?? **Bio, life, Logy, logic**—life, we will never know. Who wrote the configurations and laid the tape and built the head of this vast Turing Machine, we will never know. What an added zest to the absurd mystery of life! What even more to appreciate!

17 The Tails of Reality II

The Tails of Reality refers to the edges of this model we have charted of the world, where we are overwhelmingly aware that things no longer make sense. *Humans are intelligent because we have language, we have math and science, we have free will*—we have a model of the world around us. When you picture someone’s face or a place you’ve visited or you hum yourself a song, it appears in your head. *You can almost actually see it, right?* But you know it is not there. Somehow, the idea of it is being computed and displayed within your mind. Dreams give us even more evidence. We’ve all awoken from dreams that felt truly real; felt like our real lives, but we know they are not. When you *really* see something are you not just getting a far more dense heap of evidence and data for your brain to synthesize into a coherent moment?

Turing and Gödel mapped out important edges of the collective human model of the world, of the laws of logic and science, so much so that they found where the sidewalk ends. They showed us—by digging head-on, deep into the systems they sought to figure out—that at the end of the day, at best we have an incomplete and inconsistent mapping. Surely not a **one to one** mapping of the world, but one that works pretty damn well most of the damn time. But a model all the same.

Stepping outside the brain and the human, do you know how songs and images are stored on your computer or phone? *Well somehow it gets recorded...I know how analog cameras work, so I am sure it’s some advanced version of that...well the microphone records the sound, and hey, I know how vinyl works.* The sound you’re saving or the image you’re fossilizing never actually makes it into digital storage. When you record something, a microphone is simply taking note of volume **44,100** times per second and writing those volumes down. It is agnostic to what it is listening to; it is not even listening, it is being taken for a ride by the sound that’s coming in. A pressurized ride through space and time, and it is just taking very very comprehensive notes. Can you believe that you’re favorite most emotional, raw, *what have you* song is being stored as a rough sketch, but to your ears it is all the same? The digital representation is simply holding your brain’s hand as much as is needed so that when the list of **44,100 * Seconds** air pressure levels gets reanimated by a pressurizer that we call a speaker we hear it again as that voice or instrument we know so well. You can see and hear the band playing the song, yet you’re being fed a list of numbers made manifest as air pressure.

How about the utopian internet service that now underlies everything we do and all that we are? As I am writing these words and saving them *to my account on this writing website*, the atmosphere around me is being warped, modulated precisely so as to encode a message that gets—in less than a second—transmitted to space and back down to some data center. You cannot see it and you cannot hear it, but information is being streamed all around you all day every day *in the air you breathe and swim through* and you are none the wiser. And that is great, because it thus feels like magic. Did you know that fiber optic cables transmit their information as pulses of light? Billions or even trillions of pulses per second carry your thoughts hopes dreams movies pictures songs and homework through glass cables all around you. *Ahh, sorry I'll resend it, the WiFi is bugging.* What an odd thought that everything that exists on the internet—clearly to us now, the bulk of what is important to humans—is encoded as binary and transmitted as pulses of light moving at the fastest speed we have ever measured. Is it a testament to human ingenuity, certainly. Does it however also give us a peak into the fabric of our intelligence? It is simple enough to be turned into pulses of light?

If you were told to imagine a reality where all that felt real to you—in your head, on your computer, in your memory—was a crude sketch, a map of the world around you, you'd fear that life was a simulation. The systems that endow human intelligence and consciousness—the neural activity in the brain, the logical systems therein, the higher level apparatuses thereof that pave the way for language, math, science, and art, the digital world—are rough approximations that look smooth from a far lens. They work so well so much of the time, that we choose to take them at their most abstract level. If you thought about a song on your phone as a stream of rigid levels of amplitude, you'd certainly be missing the forest for the trees, but a forest *is made up of trees*. Once a system like the Peano axioms or Euclid's Geometry or Turing's Machine are shown to be useful, we begin to use them. And with use comes intuition and with intuition comes smoothed averaging of facts and admission of details in the name of understanding and functionality, and we forget where it all came from. And we should forget where we came from if we want to be productive human beings. Whatever that means!

Biological evolution can be seen as a gradual parallel process. Though it really is one process. The hardware of these self-organizing machines—that arose from a chaotic soup of matter and rules—improves iteratively with more data and more time. *The white rabbits fare well in the arctic tundra, so they gradually have fewer gray children.* Perception mechanisms, muscle-memorized models of physical structures, resiliency improves physically as life trudges on and babies are made. But in tandem, as things evolve to be *conscious*, the acting manager of the system, the mind of the animal that lives in the brain, zooms further out from the physical. An ant, a plant, we see, are not thinking about what they're doing, how, why—they are so far below that level that they are essentially robotic devices. They receive physical signals and—unbeknownst to them—adapt, respond, react, and live. It is a wholly physical process with no control along the way, just particles following orders handed down by the universe. As you start to think about some intelligent birds, though, maybe some mammals, we tend to believe that they have a bit more than perception of signals—they have some primitive awareness. You see a crow try a couple methods of picking at a raccoon carcass on the street until it finds a way that works best, your cat paws at a toy stuck under the dresser until he figures it out! While they're not living, talking, thinking human beings, *they're here*. They enjoy and depend on all the primitive robotics that the plants and ants do, and they take the same rough samples of the world to build a system that works, but we can see that they are a bit more removed from the purely static physical. They have things zoomed a bit more out from their base than the ants do. There is more room for intuition in the cat's mind, more room to learn *in this life*, not just over the course of hundreds of generations of succeeding cats.

When you get to humans, everything that comes in as a physical signal, anything physical in the machine of our body has a name, it has a reason, it has a row in the spreadsheet of our map of the world. You're not a robotic data-receiver that gets taken for a ride by the elements, you're a person. Yea the cones in your eye are being uncontrollably activated by energy coming in, but you're *seeing*. The basilar membrane and all the hairs in your ear are not just being perturbed by ripples in the atmosphere you swim in, you're *hearing*. You're body is not thoughtlessly, programmatically slinging electrical charge through a vast system of electrical ropes and cords when you trip over a rock and stumble, you're *using your balance*. You're not manipulating arbitrarily defined terms and symbols on the page that are held up by a set of unproven assumptions, you're *doing math*. You're not a computational circuit that can store encodings of information of all kinds for meaningful retrieval, that builds categories of categories of categories things by chemically favoring and reinforcing certain patterns of electricity—representing everything from the idea of water to the memory of the feeling of petting your first dog to your favorite sentence from your favorite book, governing everything from your instinctual parsing of air pressure signals to hear a loud and potentially threatening noise to the not-so-instinctual parsing of the clues to a crossword puzzle—running through your soft brain tissue, you're *you*!

We have such a sweet set of physical features—from the preceptors we share with an ant or a plant to the

nervous system of the cat to the neural system that makes it all make sense for us—that we forget what we are. Math, science, computers work so well that we forget that they’re rough models of the world and its rules, ones that we have proven to be unable to wrangle. Our ears, eyes, nerves, nose, tongue, and **brain** work so well at creating an omni-modal, singular and continuous stream of reality that we forget that it is a vast program of rough notes, bits of data extracted from without. We have evolved to forget about evolution. We are sophisticated enough that things feel simple. It is almost emerging as a standard for a successful system for its constituents or users to not realize that it is even a system at all. Since you were in kindergarten, you have not considered a number like **5** as an abstract label for, say, a pile of an apple and another apple and another apple and another apple and another apple, **5** apples! It has taken on such a useful role that we care not even to think of the specifics of what gives it meaning. Good systems become implicit the better they get.

Yet. At the end of the day—*what an overused, explicit-meaning-divorced category of a phrase!*—they are just systems and they are not the truth. Hell, think of Gödel’s **G** and substitute **TRUE** as the predicate instead of **PROOF**...see if you get an even more confounding answer. When you make a rough sketch of the world, when you make arbitrary but rigid decisions at base, when you formalize something as chaotic as life itself, you are bound to have a truth limit.

These are the tails of reality that Turing and Gödel wrestled with. When we are faced directly with the incompleteness of our tapestry of reality, the best bet is just to zoom back out.